A BOOK APART

Les livres de ceux qui font le web

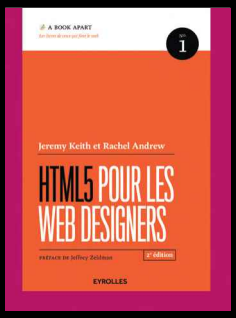
No. 17

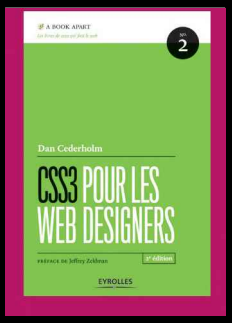
David Demaree

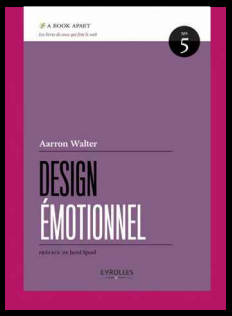
GIT PAR LA PRATIQUE

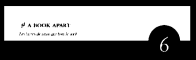
PRÉFACE DE Mandy Brown

EYROLLES



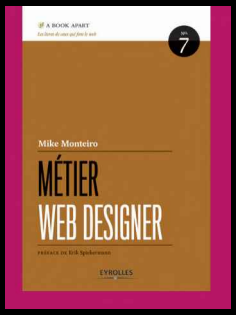






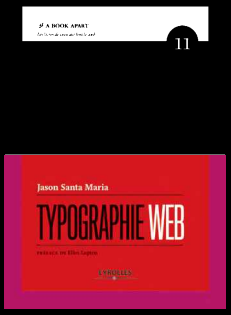
Luke Wroblewski

FIRST



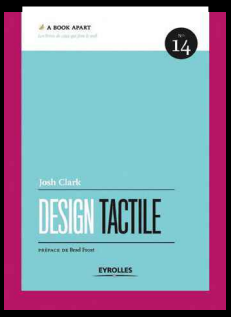
A BOOK APART
9
Erika Hall
LA PHASE DE RECHERCHE EN WEB DESIGN
EYROLLES

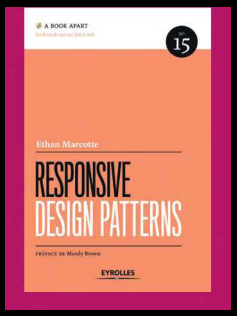
A BOOK APART
10
Dan Cederholm
SASS POUR LES WEB DESIGNERS
EYROLLES

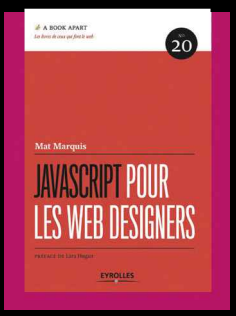

A BOOK APART
11
Jason Santa Maria
TYPOGRAPHIE WEB
EYROLLES

A BOOK APART
12
Mike Monteiro
WEB DESIGNER CHERCHE CLIENT IDÉAL
EYROLLES

A BOOK APART
13
Scott Jehl
DESIGN WEB RESPONSIVE ET RESPONSABLE
EYROLLES

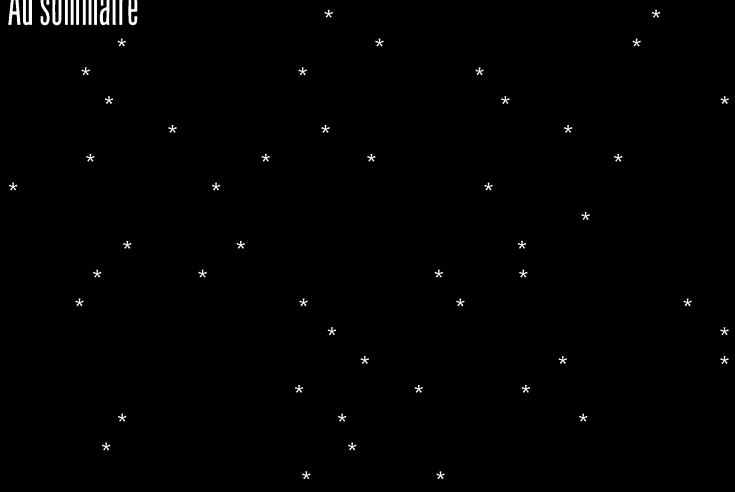

A BOOK APART
14
Josh Clark
DESIGN TACTILE
EYROLLES


A BOOK APART
15
Ethan Marcotte
RESPONSIVE DESIGN PATTERNS
EYROLLES


A BOOK APART
20
Mat Marquis
JAVASCRIPT POUR LES WEB DESIGNERS
EYROLLES

Grâce à son modèle de gestion de versions, Git est devenu un puissant outil de collaboration, indispensable dans tous les types de projets informatiques. Prenez les rênes de Git sous la houlette de David Demaree, qui fait la lumière sur le déroulement du processus en ligne de commande, les subtilités des repositories et des branches, les ingrédients d'un message de commit pertinent et bien d'autres aspects… Avec lui, vous découvrirez les tâches courantes du suivi de version et des conseils avisés pour les scénarios plus complexes. Avec ce petit guide pratique, vous avez entre les mains un outil précieux pour faire de Git votre allié et celui de toute votre équipe.

Au sommaire



Les livres de ceux qui font le web

www.editions-eyrolles.com

David Demaree

GIT PAR LA PRATIQUE

EYROLLES

ÉDITIONS EYROLLES
61, bld Saint-Germain
75240 Paris Cedex 05
www.editions-eyrolles.com

Traduction autorisée de l'ouvrage en langue anglaise intitulé *GIT for Humans* de David Demaree (ISBN : 978-1-9375573-9-3), publié par A Book Apart LLC

Adapté de l'anglais par Anne-Sophie Gagné Fradier
Avec la collaboration technique de Céline Aumard

© 2016 David Demaree pour l'édition en langue anglaise

© Groupe Eyrolles, 2018, pour la présente édition, ISBN : 978-2-212-67441-5

Dans la même collection

HTML5 pour les web designers - n°1, 2e édition, Jeremy Keith et Rachel Andrew, 2016.

CSS3 pour les web designers - n°2, 2e édition, Dan Cederholm, 2016.

Stratégie de contenu web - n°3, Erin Kissane, 2011.

Responsive web design - n°4, 2e édition, Ethan Marcotte, 2017.

Design émotionnel - n°5, Aarron Walter, 2012.

Mobile first - n°6, Luke Wroblewski, 2012.

Métier web designer - n°7, Mike Monteiro, 2012.

Stratégie de contenu mobile - n°8, Karen McGrane, 2013.

La phase de recherche en web design - n°9, Erika Hall, 2015.

Sass pour les web designers - n°10, Dan Cederholm, 2015.

Typographie web - n°11, Jason Santa Maria, 2015.

Web designer cherche client idéal - n°12, Mike Monteiro, 2015.

Design web responsive et responsable - n°13, Scott Jehl, 2015.

Design tactile - n°14, Josh Clark, 2016.

Responsive design patterns - n°15, Ethan Marcotte, 2016.

JavaScript pour les web designers - n°20, Mat Marquis, 2017.

En application de la loi du 11 mars 1957, il est interdit de reproduire intégralement ou partiellement le présent ouvrage, sur quelque support que ce soit, sans autorisation de l'éditeur ou du Centre Français d'Exploitation du Droit de Copie, 20, rue des Grands Augustins, 75006 Paris.

TABLE DES MATIÈRES

5 *Préface*

7 *Introduction*

11 CHAPITRE 1
Penser en versions

27 CHAPITRE 2
Fondamentaux

61 CHAPITRE 3
Branches

84 CHAPITRE 4
Repositories distants

107 CHAPITRE 5
Historique

137 *Conclusion*

138 *Ressources*

147 *Remerciements*

149 *Références*

151 *Index*

PRÉFACE

Je vais vous le dire franchement : Git est exaspérant.

Attendez ! Ne partez pas tout de suite. Oui, Git est insupportable. Mais il est également indispensable pour deux raisons aussi différentes qu'excellentes : tout d'abord, sur le plan pratique, Git est indispensable pour développer un site web ou une application en équipe – et si vous tenez ce livre entre vos mains, c'est probablement quelque chose que vous avez coutume de faire. Deuxièmement, Git est devenu une sorte de modèle pour la collaboration à notre époque : des équipes distribuées qui travaillent de façon asynchrone sur un volume de travail commun.

Vous n'êtes donc pas obligé d'aimer Git, mais vous devez le connaître. De nombreux tutoriels font des pieds et des mains pour proposer des analogies entre les pratiques ésotériques de Git et des phénomènes réels, au point que les lecteurs se retrouvent souvent accrochés à un arbre à se demander quelle branche va casser. Dans cet ouvrage, David Demaree nous épargne ces absurdités et nous invite à découvrir Git en parlant son langage. Plutôt que de recourir à des métaphores capillotractées, il explique, avec patience et dans des termes simples, les principes fondamentaux de Git que tout bon designer, développeur, spécialiste de la stratégie de contenu et chef de produit se doit de connaître. David Demaree nous livre ainsi un ouvrage clair et concis que vous pourrez lire en quelques heures avant de revenir à votre console, prêt à manier « pull » et « merge » en toute confiance.

Mais ce n'est pas tout, et il serait dommage que vous passiez à côté du reste ! Outre les commandes et la syntaxe, ces pages abordent également le travail en équipe et donnent quelques précieux conseils. Un « commit » de modification n'est pas seulement un outil permettant de modifier du code, c'est aussi une pratique qui, judicieusement employée, permet de

communiquer et de partager son travail. C'est un processus remarquablement puissant. Alors si les incongruités de Git poussent souvent les débutants dans leurs retranchements, son influence sur tous ceux qui bâtissent des sites web est bien méritée. Je vous encourage bien sûr à dévorer les chapitres qui suivent pour comprendre comment gérer les conflits de fusion et interpréter un log. Mais n'oubliez pas qu'au bout du compte, Git ne s'adresse pas à des machines, mais à des êtres humains.

Mandy Brown

INTRODUCTION

Quand j'ai commencé à concevoir des sites web en amateur en 1995, il suffisait de connaître HTML pour se dire développeur. C'est tout. JavaScript et CSS n'allaient pas débarquer dans le navigateur avant encore un an, et Flash ne verrait pas le jour avant la fin de la décennie. Le Web commençait tout juste à se révéler comme média riche et rempli de contenu passionnant, et quiconque était doté d'un éditeur de texte et capable de retenir une douzaine de balises pouvait participer. C'était une chouette époque.

Vingt ans plus tard, le développement web n'est plus aussi simple. HTML, CSS et JavaScript restent le fondement de notre travail, mais au fil du temps, et ces dernières années en particulier, ils ont évolué : ces langages conçus pour élaborer des documents, assez simples pour que la plupart des designers puissent les manier de mémoire, sont devenus une plateforme pour produire des applications. On a l'impression de ne plus faire de pages web, mais des thèmes ou des templates voire, pour les plus ambitieux, des applis. Nous produisons des milliers de lignes de code de plus en plus complexe, et nous partageons la responsabilité de la gestion de ce code avec de plus en plus de gens, et de manière toujours plus variée. Nous avons le pouvoir de réaliser des choses extraordinaires pour nos utilisateurs, des choses qui étaient inimaginables dans la prime jeunesse du Web, mais au prix de nous sentir comme des hamsters coincés dans la roue de la technologie.

Frank Chimero le formule très bien (http://bkaprt.com/gfh/00-01/) :

> *Soyons honnêtes : GitHub est impénétrable, Git l'est encore plus, je ne comprends pratiquement plus rien à ce qui compose la pile web d'aujourd'hui, et personne ne l'explique correctement parce qu'il semblerait que ce soit un savoir élémentaire que tout le monde tient pour acquis sans jamais le documenter, comme s'il*

s'agissait d'un secret qui s'était répandu dans la communauté par une belle journée de 2012, et à côté duquel j'étais passé parce que, vous voyez, j'avais une vie à vivre. Alors j'ai entièrement renoncé à percer le mystère, à comprendre ce sur quoi tous les autres travaillent et qui justifie une telle complexité, tout à coup effrayé d'être un has been. Je me raccroche à l'illusion que ma valeur réside dans mes « idées » et dans ma capacité à « comprendre les choses », quand bien même je ne comprends rien à tout ce dont je viens de parler — et il est bien possible que j'aie raison, parce qu'il y a effectivement beaucoup trop de choses à savoir. Faisons place aux jeunes.

Git est loin d'être la plus complexe des nouvelles technologies du Web, mais c'est une partie intégrante d'un bloc de base commun à tous les éléments de la pile. Impossible d'échapper à Git si vous voulez participer au nouveau Web des plates-formes. Vous devrez forcément vous y confronter, que ce soit directement ou à titre de mécanisme de transport employé par un autre outil. Et cela pourrait bien expliquer pourquoi Git est l'emblème du changement radical qui affecte le milieu du développement web.

On ne compte plus les livres, les articles de blog et autres contenus en ligne qui tentent d'enseigner aux utilisateurs de tous les niveaux comment utiliser Git. Mais malgré ces nombreux tutoriels, il y a des jours où tout le monde semble se plaindre que Git n'a absolument aucun sens. Et pourtant, nous l'utilisons. Nous nous sentons obligés de l'utiliser sans trop savoir comment, en ayant l'impression de courir avec les yeux bandés et une grosse paire de ciseaux.

Et cela ne concerne pas seulement des designers comme Frank Chimero. Les nouveaux arrivants du Web et ceux qui travaillent dans des domaines indirectement liés au développement web (comme la rédaction ou les données ouvertes) sont également contraints de vivre sur la planète Git, comme des milliers d'entre nous qui aiment le Web technique mais se sentent complètement dépassés par Git.

Ayant passé une bonne partie de la dernière décennie à utiliser Git dans pratiquement tous les projets, explorant parfois les

méandres les plus tortueux de son comportement, c'est en toute confiance que je peux vous dire : ce n'est pas vous, c'est Git. Si vous trouvez que Git est difficile, ce n'est pas parce que vous n'êtes pas assez intelligent ou que vous avez raté une réunion importante. C'est parce que Git est difficile.

Git est difficile en partie parce qu'il représente ce que Joel Spolsky appelle une « abstraction percée » (http://bkaprt.com/gfh/00-02/). Les abstractions, au sens logiciel du terme, sont des choses qui rendent une tâche plus simple à manipuler conceptuellement, en dissimulant les éléments qui la rendent difficile. Les interfaces sont des abstractions : il n'y a absolument aucun lien entre le geste qui consiste à faire glisser un fichier sur une icône de poubelle pour le supprimer et la suppression réelle du fichier sur votre disque dur. C'est simplement qu'un designer a pensé que cette métaphore rendrait le concept de suppression de fichiers plus compréhensible. Et ça marche ! J'ai utilisé des ordinateurs pendant plus de dix ans sans savoir comment fonctionnait la touche Maj (je ne plaisante pas), mais je n'ai jamais eu aucun problème à me débarrasser d'un fichier.

Les abstractions sont là pour nous protéger de la complexité. Une abstraction percée ne remplit pas sa mission et laisse entrevoir la complexité sous-jacente, tout comme un parapluie percé ne remplit pas sa mission de vous tenir au sec. Pour citer un exemple donné par Spolsky :

> *Vous ne pouvez pas rouler à la même vitesse que d'habitude quand il pleut, même si votre voiture est équipée d'essuie-glaces, de phares, d'un toit et d'un radiateur, tous ces accessoires qui vous évitent d'avoir à vous soucier de la pluie (qui vous font faire abstraction de la météo) : il faut quand même faire attention à l'aquaplaning, et parfois la pluie est si dense que vous manquez de visibilité — et donc vous roulez moins vite sous la pluie.*

L'interface de Git est « percée » parce que son interface en ligne de commande ne vous protège pas, en tant qu'utilisateur, de ce qui se passe sous le capot. Et l'une des raisons qui font que Git est aussi angoissant, c'est qu'il possède sa propre logique, souvent assez différente de la façon dont nous, êtres humains, avons l'habitude d'organiser les informations. Vous êtes donc parfois

contraint de savoir comment il fonctionne pour parvenir à vos fins. Pour utiliser Git, vous devrez parfois appliquer la logique de Git à des situations où la logique humaine (et les abstractions prétendument compréhensibles de Git) échoue. En d'autres termes, pour maîtriser Git, vous devrez penser comme Git.

Je veux vous aider à comprendre comment Git pense.

Croyez-le ou non, le modèle conceptuel difficile de Git est une fonctionnalité, pas un bug. Si l'on a l'impression de courir avec des ciseaux en l'utilisant, c'est parce que c'est un outil puissant qui permet de déformer le temps et l'espace à volonté : lourde responsabilité pour les simples mortels que nous sommes. Mais Git croit en vous et vous croit capable de manier un tel pouvoir. Et moi aussi.

Lançons-nous !

1 PENSER EN VERSIONS

Si vous êtes dans le métier depuis longtemps, vous vous souvenez peut-être d'un temps où les écrivains gravaient leurs romans dans la pierre. Outre l'énergie que cela demandait, la taille, le poids et le coût du matériel faisaient qu'il était peu pratique pour l'auteur d'apporter des modifications à son œuvre une fois celle-ci terminée. Corriger une coquille ou, pire, clarifier un message ou fluidifier la langue, imposait d'enlever des morceaux de roche ou de trouver une surface libre dans la grotte pour y écrire. Et dans les rares occasions où un changement s'avérait absolument indispensable, il était difficile – physiquement difficile – de conserver d'anciennes versions, et donc quasiment impossible de comparer la version finale d'un poème, d'une recette ou d'une peinture rupestre avec la version précédente, ou d'expérimenter différentes ébauches. Le simple fait d'écrire était déjà magique.

Au fil des siècles, la notation écrite est devenue plus simple et les auteurs ont ainsi pu commencer à explorer différentes approches ou changer d'avis au cours du processus créatif et après sa finalisation. Dans le même temps, les idées et le langage

se sont diffusés plus facilement, au point que chacun devienne un écrivain, du moins potentiellement.

Mais jusqu'à l'introduction des ordinateurs, la meilleure manière d'enregistrer ou de distribuer une idée était encore de l'inscrire sur un objet physique comme un morceau de papier, ce qui demandait du temps et de l'argent. La création de versions supplémentaires avait un coût, au point que la première ébauche de quoi que ce soit — roman, plan, tableau ou même photo — avait déjà un air de produit fini.

Beaucoup d'entre nous ont encore cette vision de notre travail. Prendre le temps de clarifier et d'améliorer les choses en réalisant plusieurs ébauches est un vrai luxe. Grâce aux machines et aux réseaux, le coût de la diffusion de l'information a considérablement baissé, mais l'itération nécessite encore deux choses : du temps et de la discipline.

Quand j'étais à l'école, quelques enseignants ont tenté de nous montrer, à mes camarades et moi, l'intérêt de l'itération (et de commencer les exposés sans attendre la veille de la remise) en nous demandant de remettre non seulement la version finale du devoir mais aussi les brouillons l'ayant précédée. Au lieu de faire de nous d'habiles penseurs itératifs, cette méthode n'a eu qu'un effet : démultiplier le nombre de nuits passées à terminer dans la précipitation. Plutôt que de passer du temps à produire trois versions d'un devoir au lieu d'une, même en sachant que la nouvelle sera toujours meilleure que la précédente, nous préférons suivre la voie de la paresse, nous contenter d'un travail médiocre ou incomplet et revoir quelques vieux épisodes de *Friends*.

Mais il reste encore deux domaines de notre culture écrite où les changements incrémentiels et le suivi de ces changements sur plusieurs versions ne sont pas seulement utiles mais cruciaux : le droit et (et c'est plus important pour l'histoire qui nous intéresse) le code source des logiciels.

Comme d'autres types d'écrits, le code source a eu une jeunesse que l'on pourrait qualifier d'analogique. Pour programmer les premiers ordinateurs, il fallait percer des trous dans des cartes, insérer celles-ci dans les machines qui allaient ensuite exécuter les instructions encodées dans les cartes, pour finalement

produire un résultat. (La tradition attribue l'utilisation du mot *bug* — « insecte » en anglais — à Grace Hopper, qui avait découvert que des problèmes de fonctionnement de l'ordinateur Harvard Mark II avaient pour origine une colonie de papillons de nuit installée dans les relais de données.)

Les premiers codeurs ont fait face aux mêmes problèmes que les premiers écrivains : modifier un support physique comme une carte perforée était aussi chronophage que coûteux. Il fallait des heures voire des jours pour exécuter leurs programmes, et la moindre erreur nécessitait de redémarrer toute la séquence à partir de zéro : il fallait donc viser juste du premier coup autant que possible.

Les langages informatiques doivent être compris par des machines qui — à moins de faire dans la science-fiction — restent bien plus stupides que nous. Alors qu'un humain peut lire « ont » à la place de « on » (ou l'équivalent en code) et pousser simplement un soupir désabusé devant l'inattention de l'auteur, l'ordinateur, lui, plante. Un système informatique qui plante n'est pas très utile, si bien que les développeurs de logiciels ont tenté de simplifier les choses en recourant au moyen qu'ils maîtrisent le mieux : développer d'autres systèmes.

LES BASES DE LA GESTION DE VERSIONS

Ils ont fini par mettre au point un système de gestion de versions. Le principe de base de la gestion de versions est le suivant : au lieu de conserver uniquement la dernière version d'un objet, on garde également une trace des différentes révisions au fur et à mesure, de manière à pouvoir, au besoin, revenir à une version antérieure. Bien qu'il existe pour cela des outils logiciels — dont l'un est précisément le sujet de ce livre, la gestion de versions est avant tout une pratique. C'est une méthode, au-delà des outils que l'on emploie pour la mettre en application.

Il vous est certainement arrivé de conserver des versions antérieures de votre travail sur un projet particulier, en utilisant l'option Enregistrer sous... de votre application pour donner un nouveau nom à chaque nouvel exemplaire. Vous avez peut-être

ajouté la date (projet_2014-04-15.doc) ou un numéro de version (maquette-1a.psd) au nom de fichier. Ces deux approches sont des formes rudimentaires mais parfaitement valables de gestion de versions.

Les systèmes de gestion de versions comme Git fonctionnent en conservant un exemplaire de chaque version successive d'un projet dans ce que l'on appelle un *repository* (ou *dépôt*), dans lequel vous *validez*, ou *committez*, des versions de votre travail qui représentent des pauses logiques, comme des points de sauvegarde dans un jeu vidéo. Chaque *commit* (ou *validation de transaction*) inclut des méta-données utiles comme le nom et l'adresse e-mail de la personne qui en est l'auteur, ce qui permet de savoir qui féliciter (ou blâmer) pour une modification particulière. Ces commits sont organisés en *branches*, dont chacune représente une piste d'évolution dans l'historique de votre projet, et parmi lesquelles une branche, appelée *tronc* ou *branche maîtresse*, représente la version officielle et principale. Lorsqu'on a bâti un historique de commits, il est ensuite facile de récupérer n'importe quelle version précédemment committée du projet, d'annuler des modifications ou de comparer deux versions ou plus à des fins de débogage.

Pour enregistrer des changements dans le repository, il faut une version du projet qui puisse être modifiée sans risque. Les systèmes de gestion de versions comme Git appellent souvent cela une *copie de travail*. Celle-ci joue le rôle de bloc-notes pour toutes les modifications que vous voulez apporter au projet. Vous committerez ensuite ces changements dans le repository à titre de version officielle enregistrée. De notre point de vue, une copie de travail est généralement facile à repérer : c'est l'exemplaire qui se trouve sur notre disque dur, sous la forme de fichiers conventionnels.

La gestion de versions peut sembler laborieuse parce que dans un processus de travail ordinaire, il faut enregistrer les modifications deux fois : dans la copie de travail, et dans le repository. En tant que jeune développeur web débutant, je trouvais ça ennuyeux au point d'éviter toute forme de gestion de versions. Mais au bout du compte, j'en suis venu à apprécier ses avantages, à savoir que chaque version importante de mes projets était

stockée, annotée et soigneusement organisée dans un lieu sûr. Cela m'a également aidé à voir les commits comme des changements significatifs, par opposition aux centaines de petites modifications enregistrées en l'espace d'une heure. Le commit exige des étapes supplémentaires : on s'arrête brièvement de coder pour rédiger un message descriptif, il faut parfois résoudre des conflits entre sa version et celle d'un collègue. Et ce sont précisément ces étapes qui m'ont permis d'acquérir une méthode de travail plus réfléchie et plus judicieuse.

Adopter une pratique de base de la gestion de versions représente un petit supplément de travail, mais ce n'est pas difficile. Toutefois comme tout ce qu'on fait pour rester organisé, la gestion de versions nécessite une pratique régulière pour porter ses fruits et devenir ce que David Allen, auteur de *Getting Things Done*, appelle un « système de confiance ». D'un côté, une fois que vous avez committé une version de votre travail dans un repository, vous devez avoir la certitude que, la prochaine fois que vous la consulterez, elle sera exactement dans l'état où elle était quand vous l'avez committée. (Et comme nous le verrons, Git se charge de cela très bien.) Mais vous devez également avoir la certitude que la version que vous recherchez a bien été committée : vous devez donc vous engager à committer vos modifications à intervalles réguliers pendant que vous travaillez.

PROJETS COMPLEXES

Les versions de fichiers uniques, comme des documents Photoshop, sont faciles à gérer : chaque version représente une copie complète du projet tel qu'il était à un certain point de son histoire et, si vous utilisez des numéros ou des dates pour identifier les versions, vous n'avez pas de difficulté à savoir de quel point il s'agit, simplement en regardant la liste des noms de fichiers. Mais si certaines choses que nous faisons sont soigneusement encapsulées dans des fichiers uniques, d'autres – les sites web et les applications, notamment – se composent de répertoires entiers de fichiers source. Comment appliquer la gestion de versions à des projets de ce type ?

C'est là que les systèmes de gestion de versions de logiciel comme Git gagnent vraiment leur place dans notre boîte à outils : ils gèrent les projets plus complexes tels que les sites web ou le code source d'une application, et coordonnent les modifications apportées par de nombreux collaborateurs.

La méthode la plus simple de gestion de versions est celle que nous utiliserions pour un fichier Photoshop : copier l'intégralité du répertoire du projet en ajoutant la date ou un numéro de version incrémentiel à son nom au fur et à mesure. Le répertoire nommé `notre-site/versions/v12` contient la douzième version de notre projet. Comme on le ferait avec un fichier seul, on crée une nouvelle copie numérotée du projet entier à chaque modification importante du site web : après v12 il y aura v13, suivie de v14, etc. Ici, le répertoire `versions` joue le rôle de repository et chaque répertoire numéroté est une version committée. Dans les cas simples, cela fonctionne très bien. Et effectivement, avant d'utiliser des systèmes de gestion de versions, c'est comme cela que je gérais les projets web de mes clients.

Comme un site web comprend de nombreux types de fichiers différents, nous avions besoin de copier une copie de travail sur le disque dur, dans son propre répertoire : `notre-site/copie-de-travail`. Dans ce type de projet, le processus de validation des modifications est un peu plus complexe que dans le cas de notre fichier Photoshop hypothétique, mais la différence est minime : les modifications sont effectuées et testées dans le répertoire `copie-de-travail` puis, une fois qu'elles sont prêtes à publier, on les committe en créant une copie dont le nom inclut le numéro de version suivant dans la séquence, comme `versions/v13`.

Mais que se passe-t-il quand on essaie de partager ce système de gestion de versions avec d'autres personnes ?

De nos jours, le partage de fichiers tels que ceux de nos versions committées ne pose pas de difficultés. Le repository peut être un dossier partagé dans un service comme Dropbox ou Google Drive, ou encore un disque réseau ou un serveur de fichiers dans vos locaux. Pour inviter un nouveau collaborateur à votre projet, il suffit souvent de lui donner accès à ce dossier. Tous

ceux qui ont accès au dossier partagé peuvent potentiellement committer des modifications.

Les choses se compliquent quand il faut non seulement synchroniser les modifications des différents membres de l'équipe, mais également coordonner ces changements de manière à ce que le système de gestion de versions reste fiable et viable. Un système basé sur des versions numérotées de répertoires et de copies de travail peut sembler évident, mais il ne faut jamais partir du principe que deux personnes vont interpréter ou suivre des règles simples de la même façon. Pour maintenir un système de confiance, il est indispensable que les règles soient toujours suivies exactement de la même manière. Si vous travaillez dans une petite équipe et que vous pensez « Allons, ça n'a rien d'effrayant », imaginez que vous deviez expliquer — je ne parle même pas de mettre en place — un système comme celui-ci dans une grande entreprise ou un projet open source comme Linux, qui compte des milliers de contributeurs. Je n'ai pas choisi cet exemple au hasard : Git a été inventé par Linus Torvalds précisément pour répondre aux besoins exigeants du projet Linux, après un conflit de licence sur l'outil commercial de gestion de versions utilisé auparavant (http://bkaprt.com/gfh/01-01/).

Mais, pour la forme, imaginons que tous les membres de votre équipe comprennent les règles du système et qu'ils soient tous capables de committer une nouvelle version de façon fiable. C'est là que les choses se compliquent vraiment : que se passe-t-il quand deux personnes veulent committer une nouvelle version au même moment ?

Ici, la difficulté ne réside pas tant dans l'application des règles que dans la communication. Quand on atteint le point où deux collaborateurs peuvent avoir besoin de travailler sur les mêmes fichiers au même moment, on court le risque que les modifications de l'un écrasent — ou, pour employer mon terme technique préféré, « matraquent » — le travail d'un autre.

Imaginez que nous travaillions tous les deux sur un site web. Vous vous demandez de quoi le site aurait l'air si tous les liens s'affichaient en vert plutôt qu'en rouge, tandis que, de mon côté, j'envisage de tous les mettre en bleu. Selon les règles de notre système de gestion de versions, je devrais effectuer et enregistrer

mes modifications dans la copie de la feuille de styles qui se trouve dans le répertoire. Malheureusement, les règles disent aussi que vous devez faire vos modifications à cet endroit. Si l'on suit cette méthode, dans la prochaine version numérotée du site, la couleur finale des liens — vert ou bleu — dépendra entièrement de l'ordre dans lequel nous aurons enregistré nos modifications. Si j'enregistre mon fichier après que vous avez enregistré le vôtre, mes modifications l'emportent.

Un seul moyen d'éviter ce matraquage : que les personnes souhaitant soumettre des modifications au même moment — vous et moi en l'occurrence — se concertent pour éviter les conflits, ou bien que notre système de gestion de versions, d'une façon ou d'une autre, réconcilie ou rejette automatiquement les changements conflictuels.

Certains systèmes primitifs de gestion de versions résolvent ce problème en imposant aux contributeurs « d'extraire » un fichier pour le modifier, comme on emprunte un livre à la bibliothèque. Quand un fichier est extrait, il ne peut plus être édité par quiconque tant que celui qui l'a extrait n'a pas terminé de le modifier puis explicitement rendu, ou « archivé ». On évite ainsi tout risque d'écrasement accidentel mais un autre problème apparaît : si c'est moi qui ai extrait le fichier de la page d'accueil, vous devez attendre que j'aie terminé avant de pouvoir faire votre travail.

COLLABORATION DISTRIBUÉE

Au lieu d'avoir une même copie de travail partagée par tous, on peut demander à tous les membres de l'équipe d'avoir une copie de travail stockée sur son ordinateur. En théorie, du moins, cela permet à chacun de travailler de son côté jusqu'à ce qu'il soit temps d'enregistrer une nouvelle version officielle.

Bien sûr, il reste encore possible que deux personnes tentent de committer des versions au même moment, mais on committe son travail bien moins souvent qu'on ne l'enregistre, et c'est plus facile à coordonner, par e-mail ou via un outil comme Slack : « Je vais pousser la version 34 du site, tout le monde est d'accord ? »

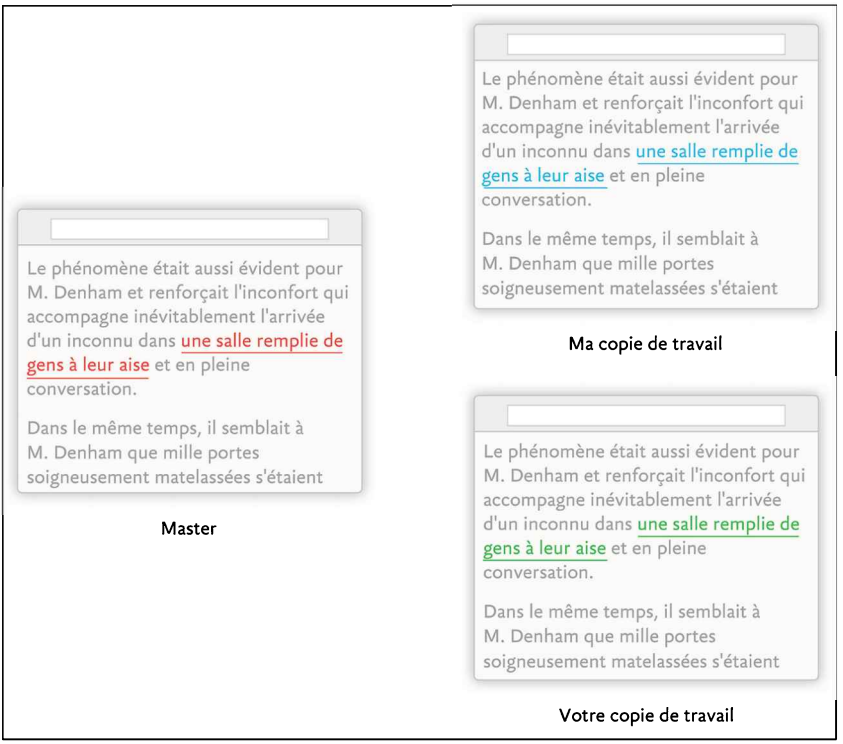


FIG 1.1 : Les liens ont des couleurs différentes sur les trois versions de cette page web. Comment savoir laquelle est correcte ?

Avec ce système, nous n'avons absolument pas tenté de fusionner différentes versions, avec différentes modifications, en un tout cohérent. Nous avons tout simplement créé des copies nommées des répertoires, en partant du principe que notre copie de travail est une source fiable et canonique pour la prochaine version. Mais quand cela cesse d'être le cas et que nos copies de travail ne sont plus synchrones, c'est un tout autre problème qui se pose.

Dans le schéma de la **FIG 1.1**, les liens sont verts dans votre version du site, bleus dans la mienne et rouges dans la dernière version officielle (v12). Les deux copies de travail sont plus

récentes que la copie maîtresse partagée, mais au-delà de cela, comment savoir de quelle couleur devront être les liens dans la prochaine version officielle ?

Et surtout, comment savoir quels autres aspects peuvent avoir été modifiés ? Que faire si, en plus d'une autre couleur de liens, votre copie du site inclut une toute nouvelle page expliquant la mission de l'entreprise, que cette page est absente de ma version, qui comprend, quant à elle, un correctif pour un bug de JavaScript (correctif dont votre copie est dépourvue) ?

C'est là que les systèmes de gestion de versions « maison » s'effondrent. Bien sûr, on pourrait se concerter pour combiner nos deux versions de travail en une même version, mais cela prendrait un temps précieux qu'il vaut mieux consacrer à écrire, concevoir, coder, faire du café ou parcourir Tumblr en quête de GIF de chat. Rechercher les modifications conflictuelles dans les fichiers de travail n'a rien d'amusant, et devient surtout ingérable à grande échelle. Si l'on multiplie les fichiers, les collaborateurs et les changements, on augmente le risque d'introduire des problèmes graves.

N'oubliez pas que le contrôle de versions ne s'arrête pas aux versions à proprement parler : il comprend aussi les règles et les processus de gestion de ces versions. Face à un grand nombre de fichiers et de collaborateurs, il peut être épuisant de faire appliquer ces règles sans arbitre, et ce n'est qu'en appliquant systématiquement les règles que vous pourrez avoir suffisamment confiance en votre système de gestion de versions pour qu'il fasse son office.

Heureusement, tous ces aspects — application des règles, suivi des versions dans un repository de travaux précédents, circulation des modifications entre le repository et la copie de travail, fusion des répertoires et arbitrage des conflits — sont plus vite (et bien mieux) traités par les ordinateurs que par nous autres humains. En apprenant et en adoptant un système de gestion de versions *automatisé* tel que Git, on préserve la bonne organisation de notre travail et une saine coordination des modifications des différents collaborateurs, et ce sans grand effort — une fois le système adopté et maîtrisé, s'entend.

L'HOMME ET LA MACHINE

Git fait pour la gestion de versions ce que les standards du Web ont fait pour notre code ou, dans les faits, ce que Microsoft Word a fait pour le traitement de texte. Parce que Git est omniprésent, une fois que vous savez vous en servir, vous pouvez envoyer du code quasiment partout. Git est parfait pour synchroniser des changements entre différents ordinateurs, qu'il s'agisse de serveurs comme GitHub ou du portable de votre collègue. Les membres dispersés d'une équipe peuvent utiliser Git pour conjuguer leurs efforts sur un projet, pousser leurs modifications vers un relais central où chacun peut récupérer sa propre copie, consulter le travail en cours, puis utiliser Git pour pousser des modifications vers un serveur web en vue de leur déploiement.

La façon dont Git indexe les changements et gère les branches vous offre un contrôle inégalé et tout en souplesse sur la commit et l'organisation des modifications de vos projets. Ces atouts font de Git l'outil idéal pour des projets tels que le noyau du système d'exploitation Linux (http://bkaprt.com/gfh/01-02/), qui compte des milliers de contributeurs et des centaines de milliers de commits. Mais Git sait aussi s'adapter avec grâce aux projets et aux équipes de petite envergure. Que vous souhaitiez ajouter une gestion de versions à votre site personnel ou partager du code avec toute votre agence, apprendre Git vous fera véritablement entrer dans la cour des grands.

Il faut rappeler que Git n'est pas le seul outil qui permette d'automatiser la gestion de versions ou la synchronisation des fichiers entre collaborateurs. Les gens emploient parfois des services de synchronisation plus simples, comme Dropbox, qui proposent des dossiers partagés et la possibilité de consulter et de restaurer les versions précédentes d'un fichier. Si l'essentiel de votre travail emploie des fichiers uniques — documents Word, feuilles de calcul, PSD, etc. — et qu'un dossier partagé Dropbox convient à vos besoins, vous n'avez sans doute pas besoin de Git.

PRINCIPE DE GIT

Git conserve votre projet dans un repository local (généralement un dossier caché de votre disque dur). Cette caractéristique le distingue des systèmes de gestion de versions plus anciens comme Subversion ou le système basé sur Dropbox que nous évoquions auparavant. Ces processus sur serveur sont centralisés : le seul lieu où vous avez accès à l'intégralité de l'historique des versions précédentes est un espace partagé et distant, et seule votre copie de travail est accessible hors ligne.

Git est un système de gestion de versions décentralisé. Votre copie de travail ainsi qu'une copie intégrale de l'historique complet du projet se trouvent sur votre machine, sur le serveur, et sur tous les autres ordinateurs hébergeant une copie du projet. Par défaut, les dossiers de repository invisibles de Git se trouvent dans un dossier visible, celui de votre copie de travail. Si vous parcourez ce répertoire, vous ne verrez que les fichiers et les dossiers attendus dans une copie de travail de votre projet. C'est dans ce dossier de travail que vous faites vos modifications.

Dès que vous êtes prêt, vous pouvez exporter ces modifications dans le repository sûr et stable en les committant. Dans le processus semi-manuel décrit plus haut, on committe une nouvelle version en dupliquant notre version de travail et en la renommant en appliquant le numéro de version séquentiel. Le commit des modifications dans Git relève d'un processus similaire, sur le plan conceptuel du moins. À chaque commit, Git enregistre l'état précis de nos fichiers tels qu'ils se trouvent dans le repository, afin de pouvoir y accéder ultérieurement. Contrairement à notre système manuel où nous avions la fâcheuse (et délicate) responsabilité de veiller à ce que les nouvelles versions soient copiées sans écraser les efforts d'un collègue, Git automatise toute cette bureaucratie. Mieux encore, Git copie les nouvelles versions de façon incrémentielle en référençant les copies existantes des fichiers inchangés, de manière à économiser l'espace disque.

Non seulement Git se charge en toute sécurité de la copie des données entre la copie de travail et le repository (et entre le

repository local et celui du serveur), mais il offre un système robuste permettant de référencer différentes versions du projet. Lorsque l'on met en place des pratiques de gestion de versions au sein d'une équipe, il faut notamment déterminer puis communiquer la bonne manière d'identifier les versions. Allez-vous utiliser un numéro (comme v12), la date (28/07/2014) ou un autre critère ? Git vous permet d'attribuer vos propres noms et numéros aux versions si nécessaire, mais affecte également un identifiant unique et fiable à chaque commit. Si vous n'avez pas besoin d'attribuer des noms ou des numéros personnalisés à vos versions, vous pouvez laisser Git s'en occuper.

Enfin, Git propose de puissants outils pour fusionner en toute sécurité les modifications apportées par différentes versions d'un projet, et non seulement les versions de différents collaborateurs, mais également les multiples variantes créées par un même contributeur sur son ordinateur.

DES DÉFIS À RELEVER

La gestion de versions peut être ardue pour les débutants, non seulement parce qu'elle complique les opérations, mais parce que le changement est, en soi, une complexité. L'utilisation d'un outil tel que Git vous pousse à remettre en question votre propre vision des rouages du changement.

Par exemple, la gestion de versions nous impose d'avoir une compréhension nuancée de l'*état*. En tant qu'humains travaillant dans un espace virtuel, nous avons l'habitude d'employer des métaphores physiques à titre de raccourcis cognitifs pratiques pour comprendre les objets numériques. Revenons à votre exemple consistant à modifier la couleur des liens d'une page web. Avant de s'être initié à la philosophie des versions, on pense être en possession d'un fichier — assimilable à un objet physique se trouvant par hasard de l'autre côté de l'écran de l'ordinateur — et modifier ce fichier. Le fichier CSS restait lui-même, seule la couleur des liens avait changé.

En réalité, du point de vue de l'ordinateur (et de Git), il y a au moins trois fichiers : la copie sauvegardée avant toute

modification (avec les liens rouges), la copie de travail où nous avons remplacé la ligne contrôlant la couleur des liens, et (enfin) la nouvelle copie enregistrée qui remplace la précédente.

Mais dans toute cette mécanique sous-jacente à la modification d'une ligne de CSS, rien ne vient altérer notre impression que styles.css est un seul et même fichier. Sur le plan sémantique, considérer le tas d'octets appelé styles.css comme un objet unique est très pratique, car cela nous aide à comprendre où se trouvent nos données. Il n'y a pas grand intérêt à passer trop de temps à s'inquiéter des différences qui existent entre les versions de ce fichier : il est préférable de s'appuyer sur le nom du fichier pour savoir de quoi il s'agit, et de disposer d'un autre moyen d'appréhender son évolution.

Plus exactement, on ne parle pas de trois fichiers différents, mais d'un même fichier dans trois états différents. C'est le même fichier, car bien que son contenu change, son nom reste le même ; sur le plan logique, c'est donc le même fichier.

Une différence potentiellement déroutante distingue notre exemple basé sur des noms de répertoires et de fichiers numérotés d'un véritable système de gestion de versions comme Git : dans ce dernier, vous ne trouverez pas de dossier géant contenant toutes les versions précédentes. En tant qu'utilisateurs de Git, nous sommes tenus de savoir que derrière chaque *exemplaire logique* d'un fichier de notre arborescence de travail, Git conserve soigneusement toutes les anciennes versions — tous les états précédents — dudit fichier.

On n'aura aucun mal à comprendre un système dans lequel deux fichiers ou répertoires sont des copies l'un de l'autre, où l'un est un peu plus récent ou plus évolué que l'autre, parce qu'il existe des équivalences dans le monde réel : les manuscrits traversent plusieurs ébauches, les livres sont réédités, et ainsi de suite. La méthode rudimentaire de gestion de versions que je décrivais plus haut était relativement facile à comprendre parce qu'il s'agissait juste de déplacer des fichiers sur un ordinateur — rien de bien exotique, donc. Ce nouveau modèle a moins à voir avec la rédaction de brouillons qu'avec le voyage dans le temps. Et quiconque a vu *Retour vers le futur II* sait que le voyage dans le temps est une affaire d'une grande complexité.

En ce qui concerne les fichiers enregistrés sur notre disque dur, les applications et les systèmes d'exploitation savent très bien comment masquer (autrement dit, « abstraire ») toute cette complexité. Au lieu d'une nuée de versions allant et venant entre le disque dur et la mémoire, nous ne voyons qu'une icône. Parfois, son contenu change, et l'heure du « dernier enregistrement » change en conséquence, mais sur les plans visuel et sémantique, il se comporte toujours comme un même fichier.

Git est très mauvais pour dissimuler ce type de complexité. Pour tout dire, il ne fait pas beaucoup d'efforts. Git souffre de ce que j'appellerais un excès de simplicité.

Contrairement à de nombreux outils du quotidien, Git n'essaie pas de représenter ce qu'il fait par des métaphores familières, ce que fait OS X en représentant la suppression d'un fichier par le déplacement de son icône dans la corbeille. La conception de Git part du principe que vous savez comment fonctionne un système de gestion de versions et, plus spécifiquement, comment fonctionne Git. Vous êtes tenu d'interagir avec Git selon ses propres termes.

Git propose plus de 75 fonctions en ligne de commande, chacune remplissant une tâche spécifique, avec ses paramètres et ses résultats. Il n'existe pas de commande « Enregistrer nouvelle version » dans Git. Au lieu de cela, pour faire un commit — qui s'apparente, sans équivalence exacte, à l'enregistrement d'une nouvelle version —, vous devez effectuer deux à trois opérations différentes. Chacune de ces opérations remplit une fonction légitime dans l'univers de Git, mais aucune ne correspond à ce que vous pourriez faire avec un fichier ou un document dans le monde réel.

Mais Git est également le programme le plus terre à terre qui soit. Il ne fait jamais rien de plus que ce que vous lui demandez (même si l'on peut facilement lui en demander plus que prévu par accident). D'un côté, cela signifie qu'il faut sans doute lui parler dans des termes plus laborieux et guindés que d'habitude, idée quasi radicale à une époque où un logiciel est capable de vous recommander un film ou de vous appeler un taxi. De l'autre, le fait qu'une commande donnée ait une portée définie impose également une limite aux dégâts que vous pouvez

provoquer à un moment précis. Et si vous rencontrez un jour un réel problème Git-esque, parce que Git ne parvient pas à réconcilier des modifications conflictuelles ou ne sait pas où committer votre travail, il y aura toujours une commande pour vous sortir d'affaire, souvent sans le moindre dommage pour le travail que vous essayiez de sauvegarder.

Comme on l'a vu, même les petites équipes ont tout intérêt à utiliser la gestion de versions. Grâce à Internet, des personnes situées de part et d'autre du globe peuvent travailler ensemble sur tout type de projet. Le mouvement open source a donné toute son ampleur à cette possibilité en permettant à des milliers d'inconnus de proposer des changements à des projets consultés et utilisés par des millions de personnes, élevant la collaboration à une échelle jusque-là jamais vue. Quand vous essayez d'accepter les contributions d'une communauté de milliers de participants, la gestion de versions devient une nécessité absolue.

Alors si la gestion de versions a fait ses premiers pas comme une forme d'assurance contre les erreurs, les outils comme Git ont permis d'en faire une pratique beaucoup plus importante. En tant que pratique et ensemble d'outils, la gestion de versions nous offre un cadre commun pour collaborer et partager tous types de travaux avec n'importe qui, n'importe où — mais également une nouvelle façon de comprendre et de gérer les modifications. Penser et travailler en versions ne nous aide pas seulement à comprendre comment un projet évolue au fil du temps, mais nous donne aussi notre mot à dire sur ce processus d'évolution.

Nous allons maintenant voir comme intégrer ce mode de pensée dans nos méthodes de travail.

2 FONDAMENTAUX

Si vous débutez avec Git, je vous recommande de vous en tenir à la ligne de commande, dans un premier temps du moins. La ligne de commande est la langue maternelle de Git. Saisir des commandes et lire les réponses renvoyées par Git est un excellent moyen d'apprendre le fonctionnement de Git, ce qui vous rendra service quand vous serez face à un problème, ce qui ne manquera pas d'arriver. La ligne de commande a également l'avantage d'être la même, quelle que soit la plate-forme. Si vous savez interagir avec Git avec des commandes, vous pourrez utiliser Git sur n'importe quelle machine.

L'interface en ligne de commande vous donne un accès total à l'intégralité des capacités de Git, mais il existe également des applications proposant une interface graphique à Git, dont certaines sont excellentes. Et bien que je vous recommande de commencer par le terminal, vous n'êtes pas obligé de vous y cantonner par la suite. Vous pouvez utiliser le terminal et une appli Git conjointement, par exemple en committant et en poussant les changements dans l'appli où ces opérations

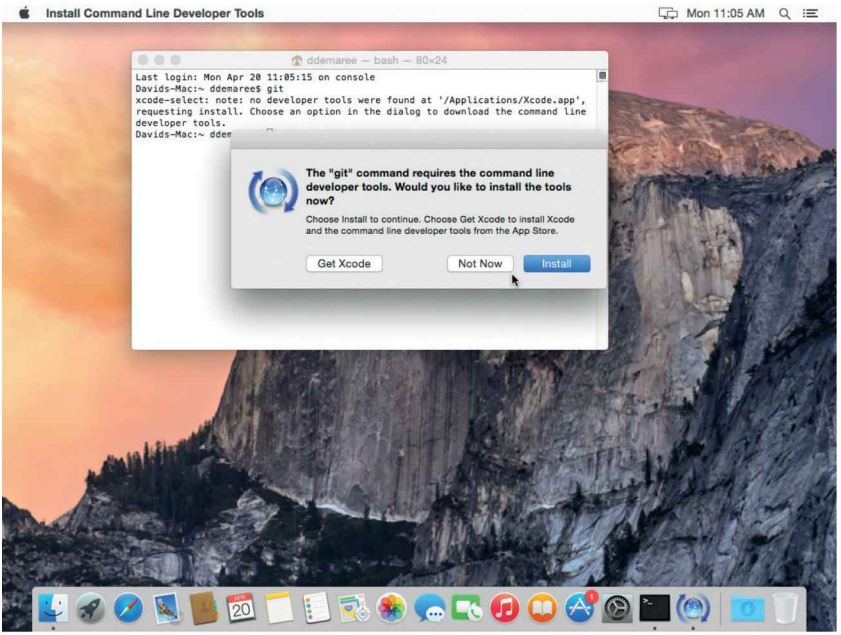


FIG 2.1 : Si vous ouvrez une fenêtre de terminal et que vous saisissez `git`, votre Mac vous propose d'installer automatiquement un paquet de logiciels maintenu par Apple qui comprend Git.

peuvent être plus simples (ou plus à votre goût), et en utilisant la ligne de commande pour tout le reste.

Pour le moment, commençons par quelque chose de simple : installer Git sur votre ordinateur.

INSTALLATION ET EXÉCUTION SOUS MAC

Depuis la version 10.9 (Mavericks), sortie en 2013, OS X télécharge et installe automatiquement les outils en ligne de commande comme Git la première fois que vous essayez de les utiliser. Si vous ouvrez simplement l'application Terminal et que vous saisissez une commande Git, votre Mac vous proposera d'installer un paquet appelé Apple Command Line Tools, qui

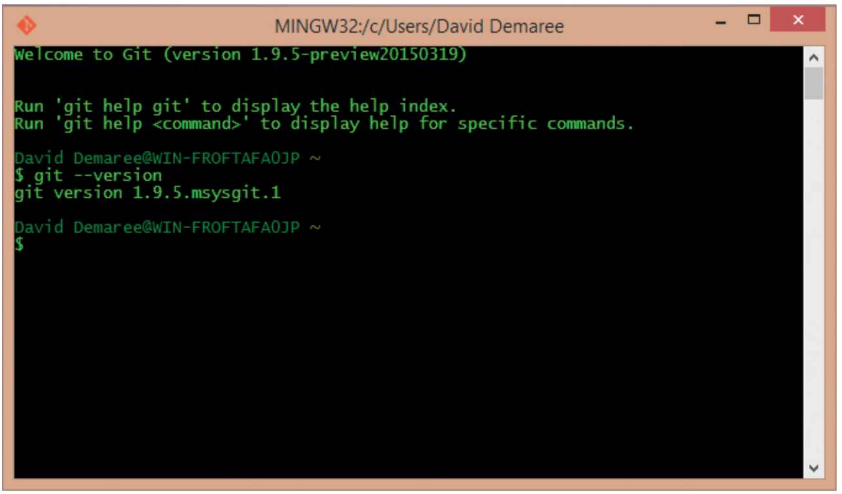


FIG 2.2 : Application Git Bash fournie avec les distributions de Git pour Windows. Celle-ci émule une console Unix complète offrant les mêmes commandes qu'une console dans Linux ou OS X.

comprend Git et d'autres utilitaires. Après l'installation, OS X téléchargera et installera automatiquement les mises à jour de Git via l'application Mac App Store.

INSTALLATION ET EXÉCUTION SOUS WINDOWS

L'équipe de développement de Git maintient un installeur pour Windows simple d'utilisation que vous pouvez télécharger sur le site officiel de Git (http://git-scm.org/). L'assistant d'installation vous posera plusieurs questions sur la configuration et l'utilisation de Git. Quand vous ne savez pas, optez simplement pour les paramètres par défaut.

Si vous êtes à l'aise avec l'un des deux environnements de ligne de commande standard de Windows, à savoir l'invite de commande ou PowerShell, l'installeur Git vous permettra de configurer Git pour l'un ou l'autre. Par défaut, l'installeur Git fournit sa propre application de terminal, appelée Git Bash, qui émule

(qui fonctionne de façon similaire à) un système Unix comme OS X ou Linux et prend en charge non seulement Git mais toutes les commandes que nous utiliserons dans ce chapitre et dans ce livre. Si vous avez l'habitude d'utiliser Git sur l'une de ces plates-formes, ou si vous souhaitez bénéficier de l'expérience de ligne de commande la plus cohérente sur différentes machines, Git Bash est un excellent outil.

Bien que Windows et Linux/OS X prennent en charge les mêmes encodages pour les fichiers de texte (ASCII et Unicode), ces systèmes utilisent des caractères de fin de ligne différents. De nombreux éditeurs de texte Windows ou multi-plates-formes – Atom et Sublime Text, par exemple – gèrent déjà très bien les caractères de fin de ligne, mais Git ne sait pas toujours quel format employer.

Git pour Windows nous rend ici service en proposant automatiquement de convertir les fins de ligne Windows en fins de ligne Unix lorsque vous committez, et vice versa quand vous archivez un fichier.

Même si tous vos collègues et vos serveurs sont sous Windows aujourd'hui, il est généralement plus sûr à long terme de committer avec des fins de ligne Unix pour que vos fichiers puissent être extraits sans risque sur les systèmes non Windows. Une dernière petite différence : si les chemins d'accès Windows utilisent des antislashs et des lettres pour nommer les disques, comme dans `C:\Users\David\monprojet`, les chemins Unix s'écrivent avec des slashs conventionnels. Sous Mac, l'équivalent serait `/Users/David/monprojet` (sans lettre de disque en tête puisque les systèmes Unix comme OS X ne les emploient pas). Ici encore, Git Bash nous rend service en remplaçant automatiquement les slashs et en présentant ce chemin Windows sous la forme `/c/Users/David/monprojet`. (Le `/c/` de tête indique le disque `C:/`, conformément à la convention Unix qui référence les disques montés comme des répertoires à la racine du système de fichiers de l'ordinateur.)

COMMANDES DE BASE

Si vous n'êtes ni programmeur, ni administrateur système, ni informaticien, et que c'est la première fois que vous interagissez avec la ligne de commande, vous risquez de trouver effrayant de taper des petits morceaux de jargon derrière une invite. C'est tout à fait compréhensible, car cela peut sembler un peu effrayant. À partir de la ligne de commande, votre ordinateur vous permet de faire des choses qu'il vous interdirait immédiatement dans Finder ou dans le Panneau de configuration, comme supprimer votre système d'exploitation voire l'intégralité de votre disque dur. Gardez toutefois à l'esprit que si la ligne de commande est puissante, elle est aussi un peu idiote à un autre niveau : elle ne fait que ce que vous lui dites de faire, et la plupart des commandes sont conçues pour accomplir une seule tâche simple.

Avant de nous plonger dans les détails de l'utilisation de Git, je vais passer en revue le fonctionnement de la ligne de commande et vous expliquer comment lire les exemples qui apparaissent tout au long de ce livre.

À quoi sert la ligne de commande

La ligne de commande est une rémanence d'une époque où les systèmes d'exploitation entièrement graphiques comme Windows et OS X n'étaient pas assez puissants pour régir tous les aspects d'un ordinateur. Si vous êtes aussi vieux que moi, vous vous souvenez peut-être avoir dû quitter l'environnement Windows et repasser sous MS-DOS pour une raison quelconque. Vous vous souvenez peut-être aussi avoir regardé *Retour vers le futur II* sur une VHS toute neuve.

L'interpréteur de ligne de commande est un *shell*, terme anglais qui signifie « coquille ». Comme bien d'autres termes informatiques, c'est une métaphore. De même que certains animaux, comme les tortues, possèdent une coquille qui dissimule leurs parties les plus fragiles ou dangereuses, tout en ménageant des ouvertures pour qu'ils puissent se déplacer et interagir avec leur environnement, le shell couvre et protège l'ordinateur — ainsi

que vous, utilisateur — des commandes susceptibles d'entraîner un comportement anarchique ou une panne totale du système. Si nous parlons essentiellement de ligne de commande ici, les interfaces graphiques que nous connaissons tous aujourd'hui sont aussi des coquilles, faites d'images plutôt que de texte.

L'ouverture de notre coquille métaphorique est l'invite de ligne de commande, une chaîne de caractères qui signale que l'ordinateur est prêt à recevoir vos instructions. La plupart des invites de ce livre auront la forme suivante :

```
$: _
```

Le tiret bas (_) représente le curseur ; le `$:` est l'invite à proprement parler, qui indique que la console attend que vous lui demandiez quelque chose. Bien que le shell soit notre moyen principal d'interaction avec certains types de systèmes, il ne s'agit que d'un programme. Le shell utilisé par défaut dans OS X et de nombreux systèmes Linux, de même que celui qui est inclus dans le paquet Git pour Windows, s'appelle *bash*. Bash est le shell le plus couramment utilisé par les machines de type Unix, et même s'il est loin d'être le seul, nous partirons du principe que c'est le vôtre, au moins dans le cadre de ce livre.

Vous avez sans doute déjà vu, dans un film ou une série télévisée, un hacker aux lunettes noires aller puiser des données dans des serveurs et infiltrer le Pentagone en tapant du charabia sur un écran d'ordinateur. S'il y a une chose que ces films représentent souvent correctement, c'est le fonctionnement d'une invite de commande. On commence par taper l'instruction qu'on veut donner à l'ordinateur dans une langue qu'il comprend :

```
$: whoami_
```

On appuie ensuite sur Entrée pour envoyer la commande. Ici, il s'agit du nom d'un autre programme qu'on demande au shell d'exécuter. Ce programme `whoami` (« qui suis-je ») est très simple : il indique simplement sous quel nom d'utilisateur vous êtes connecté. Une fois que le programme a terminé de s'exécuter, généralement immédiatement, il renvoie (ou « imprime ») une

réponse dans la fenêtre du terminal, sous la ligne où vous avez saisi la commande, comme ceci :

```
$: whoami
david
```

Le terminal affiche alors à nouveau les caractères de l'invite, prêt à répondre à une autre question :

```
$: _
```

Les programmes en ligne de commande manquent souvent de cérémonie. Ils impriment les informations que vous demandez sans faire beaucoup d'effort pour les mettre en forme ni même en contexte. Alors que vous espériez peut-être une étiquette ou un peu de contexte (par exemple, « Votre nom d'utilisateur est : david »), la commande `whoami` vous répond directement. Vous avez demandé `whoami`, elle répond `david`, puis le terminal vous donne une nouvelle ligne pour poser une autre question. De nombreuses commandes (autant présentes dans le système d'exploitation que dans Git) ne renvoient aucune réponse.

Bien que de nombreuses commandes, dont plusieurs commandes Git, puissent être un peu trop bavardes dans leurs réponses, `whoami` est si laconique qu'elle frôle l'impolitesse. Ce n'est pas le signe d'un problème. Au contraire : l'absence de réponse indique que le système et le shell pensent que vous savez ce que vous faites. Quand un programme se termine (ou « quitte ») sans réponse, cela signifie presque toujours que la tâche que vous lui avez demandée a été accomplie sans erreurs.

Prenons par exemple la commande `mkdir`, qui crée un nouveau répertoire :

```
$: mkdir javascripts
$:
```

Si vous n'êtes pas certain de ce qui vient de se passer : `mkdir` n'a fait aucun commentaire, ce qui pourrait laisser penser que

la commande de création n'a pas fonctionné du tout. Mais regardez ce qui se passe si l'on ressaie :

```
$: mkdir javascripts
mkdir: javascripts: File exists
$:
```

Dans ce cas, la commande a échoué pour une raison toute simple : elle ne peut pas créer un répertoire `javascripts` parce qu'il existe déjà.

Navigation en ligne de commande

Comme une fenêtre Finder sur un Mac, le terminal observe toujours un seul répertoire ou dossier actif à la fois. C'est ce qu'on appelle le *répertoire de travail*. Pour afficher votre emplacement actuel sous la forme d'un chemin absolu, utilisez la commande `pwd` (acronyme de « print working directory », soit « afficher répertoire de travail ») :

```
$: pwd_
/Users/david
```

On peut ensuite utiliser la commande `cd` (« change directory », soit « changer de répertoire »), qui fait du chemin qu'on lui indique le nouveau répertoire de travail — l'équivalent, en ligne de commande, d'un double-clic sur l'icône du dossier. C'est souvent la première commande qu'on lance dans une nouvelle session de terminal, pour se rendre du point de départ par défaut du shell (généralement la racine du répertoire utilisateur) au répertoire contenant le projet Git.

```
$: cd Projets/notre-site
$: pwd
/Users/david/Projets/notre-site
```

Nous avons donc saisi le chemin d'accès au répertoire du projet (`Projets/notre-site`), relativement au répertoire de départ où nous avons entamé cette session de terminal. Les chemins

relatifs se distinguent des chemins absolus par l'absence de slash initial (/) qui représente ici le répertoire racine (ou de plus haut niveau) de votre système de fichiers local.

Petite astuce pratique si c'est votre première expérience avec le terminal : vous pouvez utiliser le tilde (~) comme alias pour votre répertoire utilisateur. Les chemins absolus `/Users/david/Projets/notre-site` et `~/Projets/notre-site` sont équivalents, et vous pouvez utiliser l'un ou l'autre pour accéder à votre répertoire de projet depuis n'importe quel autre point de votre disque dur.

Pour afficher une liste de tous les fichiers et répertoires contenus dans le répertoire en cours, utilisez la commande `ls` (« liste ») :

```
$: ls
css          index.html
```

Ce répertoire contient deux choses : une page web (`index.html`) et un sous-répertoire contenant nos feuilles de styles (`css`). Nous ne sommes pas dans Finder, et aucune icône ne permet de différencier les catégories des objets que nous avons sous les yeux : impossible de savoir si `css` est un répertoire ou un fichier. Mais `ls` propose quelques options pour demander plus d'informations. Par exemple, si vous ajoutez l'option `-F`, `ls` affiche des slashs de fin pour distinguer les répertoires des fichiers conventionnels :

```
$: ls -F
css/         index.html
```

Pour explorer un répertoire sans entrer dedans, il suffit de passer le nom du répertoire en question à la commande `ls`, à titre d'argument :

```
$: ls css
styles.css
```

Bien sûr, on peut aussi entrer dans un sous-répertoire de notre projet en utilisant `cd`.

```
$: cd css
$: ls
styles.css
```

Comme sur le Web, le double point (. .) représente le répertoire parent du répertoire en cours et, comme vous vous en doutez, il fonctionne dans une invite de shell.

```
$: ls ..
css          index.html

$: cd ..
$: pwd
/Users/david/Projets/notre-site
```

Il peut être utile de garder à l'esprit que les outils en ligne de commande et les interfaces graphiques comme Finder remplissent essentiellement la même fonction. Tous observent les mêmes fichiers sur le même disque dur, ce qui signifie que vous pouvez utiliser Finder (ou la boîte de dialogue Enregistrer... d'une application) pour enregistrer des fichiers ou créer des répertoires si vous préférez une interface graphique. Si vous êtes plus habile avec la ligne de commande ou que vous n'aimez pas basculer entre le terminal et l'interface graphique, il peut être plus rapide de faire quasiment tout dans le terminal. D'un autre côté, selon votre expérience et vos préférences, vous travaillerez peut-être plus vite et plus efficacement en utilisant le terminal et Finder en tandem plutôt que l'un ou l'autre seul.

L'invite

Pour rester simple, les invites de commande utilisées dans le reste de cet ouvrage auront la forme suivante :

```
(master) $:
```

La branche Git en cours, ici `master`, est indiquée entre parenthèses. Nous explorerons plus tard les autres branches, mais `master` sera notre branche de travail pour ce chapitre et le suivant.

Dans nos exemples, un astérisque placé après le nom de la branche indiquera que notre copie de travail contient des changements non committés, comme ceci :

```
(master *) $:
```

Chaque système d'exploitation et chaque shell a son propre format d'invite par défaut et, si vous ne l'aimez pas, vous pouvez la personnaliser à votre goût. Il y a fort à parier que les invites de votre ordinateur seront différentes de celle de ce livre.

Je mentionne ce point principalement pour vous rassurer : ne vous inquiétez pas si les exemples de ce livre et les invites de votre terminal ne sont pas identiques. Les commandes Git et leurs résultats sont ce qui compte le plus, et ils devraient être les mêmes, quel que soit l'aspect de votre invite.

LE LANGAGE DE GIT

Maintenant que vous avez pris vos repères dans le terminal, il est temps d'interagir vraiment avec Git en lui envoyant des commandes, qui seront toutes plus ou moins de la forme suivante :

```
(master) $: git commande paramètre1 paramètre2 »
  --option
```

Le nom de la commande (`commande` dans l'exemple) représente la fonction spécifique à exécuter par Git, parmi plus d'une centaine. En coulisses, chacune de ces commandes est un programme distinct, chargé d'accomplir une tâche très précise.

Si certaines fonctions de Git se suffisent d'un nom de commande (comme `git status`), la plupart nécessitent des paramètres pour faire leur travail, de même qu'on passe des

informations à des fonctions dans un langage de programmation comme JavaScript ou Ruby.

Bon nombre de commandes Git se lisent comme une sorte de phrase : *Git, fais ceci à cela*. Par exemple, la commande `git checkout master` signifie essentiellement : « Git, extrais la branche nommée `master`. »

Les options sont des paramètres spéciaux signalés par au moins un tiret de tête. Elles sont rarement nécessaires et changent souvent la manière dont Git gère une tâche particulière. Beaucoup d'options ont une forme longue, comme `--global` et une forme raccourcie, comme `-g`.

Certaines options prennent même une valeur, comme `git commit --message="Bonjour"`.

Au fil des pages, je vous présenterai des options importantes ainsi que leur fonction et les types de valeurs qu'elles peuvent recevoir.

CONFIGURER GIT

Git est un animal complexe qui présente des centaines d'options de configuration, mais seules deux sont absolument nécessaires à son fonctionnement : votre nom et votre adresse e-mail. Git ajoute à chaque commit un attribut `Author` (« auteur ») comprenant votre nom et votre adresse e-mail, afin que les collaborateurs d'un projet puissent savoir qui a apporté une modification donnée. Le nom que vous saisissez servira à vous identifier dans les *logs* (ou *journaux de modification*) et partout où Git affichera l'auteur d'un changement, tandis que votre adresse e-mail permettra non seulement à d'autres participants de vous joindre, mais indiquera également votre identité à un service hébergé comme GitHub.

Commençons donc par dire à Git qui vous êtes en utilisant la commande `git config`. Contrairement à de nombreuses commandes Git, qui ne fonctionnent qu'au sein d'un projet Git, celle-ci peut être exécutée depuis n'importe quel répertoire.

Saisissez chacune de ces lignes sur une invite, en indiquant votre nom et votre adresse e-mail :

```
$: git config --global user.name "David Demaree"
$: git config --global user.email "david@demaree.me"
```

Ici, on indique à Git qu'une certaine propriété de configuration (`user.name`) doit prendre la valeur (`David Demaree`) que nous lui fournissons.

L'option `--global` indique à Git qu'il doit s'agir des valeurs par défaut pour tous les projets de cet ordinateur. Vous pouvez, si vous le souhaitez, utiliser la commande `git config` pour définir des options de configuration comme `user.name` propres à des projets spécifiques, tout simplement en omettant l'option `--global`. Mais pour le moment, nous allons définir ces valeurs globalement parce que Git a besoin qu'elles soient définies quelque part, et que, de cette façon, vous n'aurez pas à le faire à chaque fois que vous créerez un nouveau projet.

Une brève remarque au sujet de la confidentialité, parce que le partage d'informations personnelles de ce type peut être un sujet sensible : Git utilise le nom et l'adresse e-mail que vous fournissez pour attribuer le crédit des commits que vous effectuez. Dans le cas d'un repository local, ou tant que les commits n'ont pas été poussés sur un serveur, ces informations résident uniquement sur votre ordinateur. Mais les commits sont faits pour être partagés : sachez donc que les commits partagés avec d'autres (que ce soit dans votre équipe ou dans un projet open source) incluront ces informations. De plus, comme vous le verrez, les commits ne sont pas faits pour être modifiés une fois appliqués.

Je précise cela pour vous avertir, et non pour vous effrayer. Git exige un nom et une adresse e-mail pour attribuer les commits, mais se fiche bien de savoir si les valeurs que vous saisissez reflètent bien la vérité. D'ailleurs, des binômes de programmeurs travaillant sur le même problème et sur la même machine modifient souvent la configuration de Git afin que leurs commits aient une attribution commune.

Quand vous indiquez ces valeurs, vous êtes libre de fournir uniquement les informations personnelles que vous souhaitez — et personne ne vous en demandera plus.

CRÉER UN NOUVEAU PROJET

Une fois que vous avez adopté Git comme système de gestion de versions, l'une des premières choses à faire est de créer une nouvelle base de données Git à l'aide de la commande `git init`. Mais ce sera rarement la toute première chose, car la plupart des repositories Git sont conçus pour se trouver dans un dossier de votre disque — le répertoire de travail — avec les fichiers de votre projet.

Plus simplement : pour suivre les modifications au sein d'un dossier de projet, il faut d'abord avoir un dossier de projet. Il peut s'agir d'un dossier existant dans lequel vous travaillez déjà et dans lequel vous ajoutez Git, ou bien d'un tout nouveau projet pour lequel vous allez utiliser Git dès le départ.

Pour plus de clarté dans ce chapitre, nous allons entamer le développement d'un nouveau site web. Créons d'abord un répertoire. Nous pourrions le faire dans Finder ou Windows Explorer, mais nous allons ici utiliser la ligne de commande.

```
$: mkdir notre-site
```

Un nouveau dossier intitulé `notre-site` est créé à l'intérieur du répertoire actuel. Entrons maintenant dans ce nouveau répertoire à l'aide de la commande `cd` :

```
$: cd notre-site
```

Nous allons initialiser un nouveau repository Git dans ce dossier de projet à l'aide de la commande `git init`.

```
$: git init
Initialized empty Git repository in /Users/david/ »
  travail/notre-site/.git/
(master) $:
```

Voilà ! Un tout nouveau projet Git vide sur notre ordinateur. Il est vide à deux égards : il ne contient aucun fichier (n'oubliez pas que nous venons tout juste de créer ce répertoire) et aucun commit.

CLONER UN PROJET EXISTANT

Si vous rejoignez un projet Git que vous n'avez pas vous-même initié, la première étape consiste à récupérer une copie du repository entreposé sur un serveur quelconque. C'est ce qu'on appelle *cloner* : ce qui est enregistré sur votre ordinateur est une réplique de tout ce qui a été stocké sur le serveur.

Le clonage d'un repository est un processus séquentiel que Git, toujours serviable, enveloppe dans une seule et même commande : `git clone`. Comparable à une macro, `git clone` est une commande pratique qui exécute plusieurs commandes connexes simultanément : elle crée un nouveau répertoire de travail (portant le nom du repository présent sur le serveur par défaut), initialise un nouveau repository Git, ajoute un serveur distant *(remote)* appelé `origin` et extrait les modifications de ce serveur distant.

```
$: git clone https://gitforhumans.info/ »
  notre-site.git
Cloning into notre-site...
remote: Counting objects: 11, done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 11 (delta 1), reused 11 (delta 1)
Unpacking objects: 100% (11/11), done.
Checking connectivity... done
$: cd notre-site
(master) $:
```

Suivre la convention consistant à nommer le répertoire de travail comme le repository est quasiment toujours la façon la plus simple de procéder. Mais si vous tenez à donner un autre nom à votre répertoire, pour lui ajouter le nom d'un client par exemple,

il suffit de passer un autre nom de dossier comme argument à la commande `git clone`. Ici, au lieu de nommer le projet de notre client `notre-site`, appelons-le `site-client` :

```
$: cd ~/travail
$: git clone https://clientco.co/notre-site.git »
  site-client
Cloning into 'gfh-website'...
remote: Counting objects: 11, done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 11 (delta 1), reused 11 (delta 1)
Unpacking objects: 100% (11/11), done.
Checking connectivity... done
$: cd site-client
(master) $:
```

Dans tous les cas, une fois que vous avez cloné le repository distant, Git fonctionne exactement comme si vous aviez vous-même créé le projet. De plus, comme Git ne se soucie généralement que de ce qui se trouve dans votre projet et non dans le dossier qui le contient, peu importe le nom que vous lui avez donné quand vous avez cloné le projet, vous pourrez le renommer par la suite sans crainte.

Le clone que nous venons de créer contient tout l'historique de ce repository, y compris toutes les modifications partagées par toutes les autres copies. Cette idée qu'on ne clone pas seulement le projet dans son état en cours, mais également tout son historique, prouvera toute son utilité un peu plus tard.

PRÊT À COMMITTER

Git s'attend à ce que chaque modification effectuée au sein d'un répertoire sous son contrôle soit enregistrée et stockée dans le repository, bien à l'abri, et cela inclut la modification qui fait naître un projet du néant. Sur le plan pratique, Git s'inquiète plus de la gestion de vos commits que des fichiers que vous modifiez à l'aide de ces commits. Ce n'est pas tant que Git est indifférent

au contenu de vos fichiers : c'est juste que son modèle d'organisation et de gestion de votre travail est axé sur les commits.

Les commits sont un type de données Git appelé *objet*. À l'intérieur, chaque information connue par Git — le contenu des fichiers, la structure des dossiers et, surtout, les commits qui signalent les autres comme versions d'un projet — est stockée sous la forme d'un objet, et chaque objet possède un nom unique dérivé de son contenu. En réalité, le nom d'un objet est plutôt un identifiant court, lisible par la machine, qui le distingue de manière fiable des autres objets.

Les commits sont un type parmi d'autres d'objets avec lesquels vous travaillerez au quotidien. Sur le plan sémantique, chaque commit représente un *snapshot* (ou *instantané*) complet de l'état de votre projet à un moment précis ; son identifiant unique permet de distinguer cet état de l'aspect qu'avaient les fichiers de votre projet à tout autre moment.

Git procède par addition. Bien que des fichiers de votre projet puissent être créés, supprimés ou modifiés, les commits qui consignent tous ces changements sont toujours ajoutés. Quand vous supprimez un fichier, vous ajoutez un commit. Si vous modifiez une ligne de texte ou de code, ou que vous renommez un fichier, vous modifiez l'état du projet et vous ajoutez un commit qui signale ce changement et le propage au reste de votre équipe.

C'est une des beautés de la conception de Git : les éléments de sa base de données sont inaltérables, à savoir qu'ils ne peuvent jamais être vraiment modifiés, seulement augmentés. Git est un système d'accumulation. Git additionne chaque changement que vous lui signalez pour que vous puissiez revenir en arrière et explorer l'historique par la suite.

Lorsque vous committez une modification dans votre travail, il se passe des choses assez géniales en coulisse, et nous nous y plongerons en détail dans le prochain chapitre. Pour le moment, faisons un commit pour voir comment ça marche.

STATUT GIT

Nous venons de créer un répertoire pour accueillir notre projet de site web. Nous allons maintenant ajouter un fichier qui servira de page d'accueil. Ouvrez un nouveau document dans votre éditeur HTML préféré et ajoutez-y ceci :

```
<!DOCTYPE html>
  <html>
    <head>
      <title>Notre site</title>
    </head>
    <body>
      <h1>Notre site</h1>
    </body>
</html>
```

Terminé. Enregistrez le fichier dans votre répertoire de projet sous le nom index.html. Avant de continuer, demandons à Git ce qu'il sait de l'état de notre projet en utilisant la commande `git status`.

```
(master *) $: git status
# On branch master
#
# Initial commit
#
# Untracked files:
#   (use "git add <file>..." to include in what will
  be committed)
#
# index.html
nothing added to commit but untracked files present
(use "git add" to track)
```

On trouve ici plusieurs informations très intéressantes. Tout d'abord, nous nous trouvons dans la branche `master` du projet (nous en parlerons plus en détail au chapitre 4). Pour le

moment, sachez simplement que nous sommes dans la bonne dimension de l'invraisemblable multivers qu'est Git. Comme c'est un nouveau repository vide, notre prochain commit sera le commit initial, le tout premier de notre chronologie. Enfin, et surtout, nous avons une liste de fichiers *untracked* ou « non suivis », dont notre fichier index.html.

AJOUTER DES FICHIERS À GIT

Avant de pouvoir committer un fichier, celui-ci doit être suivi. Et pour suivre un fichier, il faut l'ajouter à la base de données de Git. C'est une source de confusion pour beaucoup de débutants : pourquoi faire la distinction entre l'ajout et le commit d'un fichier ?

Parce que ce n'est pas la même chose ! Un commit enregistre les modifications apportées aux fichiers figurant dans la base de données de Git ; il indique par exemple qu'un fichier est passé de la version A à la version B (ou, dans le cas d'un commit initial, il définit ce qu'est la version A). En toute logique, pour savoir comment la version A a changé pour devenir B, Git doit connaître les versions A et B à titre individuel. Pour des raisons de simplicité (ne riez pas), il est normal que nous autres, êtres humains, considérions les commits comme un raccourci pour désigner des versions spécifiques des fichiers et des dossiers. Mais en tant que tels, les commits ne sont que des références, de même qu'une adresse postale référence une maison.

Quand on ajoute un fichier, on construit la maison. La commande `git add` prend un snapshot du fichier en question et l'enregistre dans le repository pour qu'il soit référençable ultérieurement, dans un commit.

Git enregistre ainsi parfois des snapshots qui ne seront jamais committés, et c'est très bien comme ça. Ils prennent très peu de place sur le disque et, de temps à autre, Git fait ce qu'on appelle une *collecte des ordures :* il détecte tous les objets qui ne sont référencés dans aucun commit et les supprime. Si ça semble drastique, essayez de le voir de cette façon : vous n'avez suivi aucune des pistes représentées par ces objets épars. S'ils valaient la

peine d'être conservés, Git part du principe que vous les auriez committés quelque part.

Mais là je sais que vous vous dites : « Tout cela est fort intéressant, mais comment faire pour mettre la première version du fichier dans la base de données de Git pour le committer ensuite ? »

Comme le suggérait la commande `git status`, nous allons utiliser la commande `git add`.

Pour ajouter le fichier de la page d'accueil, saisissez ce qui suit :

```
(master *) $: git add index.html
```

En général, Git ne répond rien et ne vous dit pas qu'il s'est passé quoi que ce soit. Et quand vous aurez fait cela plusieurs fois, vous n'aurez besoin d'aucune réponse. Mais comme c'est votre premier commit, vérifions à nouveau avec `git status` :

```
(master *) $: git status
# On branch master
#
# Initial commit
#
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#
# new file:   index.html
#
```

Notre fichier ne figure plus dans la liste des `Untracked files`, mais dans la liste `Changes to be committed`. Nous avons maintenant indexé notre fichier et nous sommes prêts à le committer.

```
(master *) $: git commit --message "Commit initial"
[master (root-commit) 600df9f] Commit initial
1 file changed, 9 insertions(+)
```

```
create mode 100644 index.html
(master) $:
```

Magie. Vous venez de committer votre premier fichier.

Si vous y regardez de près, sur la première ligne de la réponse, juste avant le message de commit que nous avons fourni, vous trouverez l'identifiant unique de ce commit : `600df9f`. La plupart des identifiants de commit que vous verrez (et vous allez en voir une bonne quantité au fil de ces pages) ressembleront à ça.

L'e-mail et le nom que nous venons d'ajouter indiquent notre identité à Git, et l'option `--message` précise à vos collaborateurs (ou votre moi du futur) la nature des changements réalisés. Tant que nous y sommes, laissez-moi vous montrer une petite astuce pour économiser des caractères. Beaucoup d'options des commandes Git sont disponibles sous une forme raccourcie (généralement d'un seul caractère), signalée par un tiret unique plutôt qu'un double tiret. Ainsi, l'argument `--message` peut également être saisi sous la forme `-m`.

```
(master *) $: git commit -m "Commit initial"
```

Nous avons ainsi fait notre premier pas sur la longue route de l'historique de ce projet. Passons maintenant au changement suivant et à notre deuxième commit.

INDEXATION

Avant de pouvoir committer une nouvelle version de vos fichiers, cette nouvelle version doit être ajoutée à la base de données de Git, à l'aide de la commande `git add`. Cette procédure s'appelle également *indexation* : la *zone d'indexation* est le lieu où se trouvent ces nouvelles versions, entre le moment où vous mettez à jour vos fichiers de travail et celui où vous les committez.

L'indexation d'un fichier a deux conséquences en coulisses. Tout d'abord, Git fournit un snapshot de ce fichier à sa base de données afin de pouvoir y faire référence dans le prochain

commit. Le fonctionnement des références de Git est tel qu'un fichier doit déjà se trouver dans le repository pour qu'une référence soit possible, et il faut pouvoir le référencer pour le committer. Tant qu'aucune version d'un fichier n'est indexée, Git ne sait pas comment désigner cette version et ne peut donc pas la committer.

Git démarre également une ébauche locale de votre prochain commit en référençant tous les fichiers et répertoires contenus dans son dossier, y compris les fichiers qui n'ont pas été modifiés depuis le commit précédent : les références sont alors copiées. Chaque commit est autonome : il ne référence pas seulement ce qui a changé, mais aussi tout ce qui compose l'état de votre projet à un moment donné. La plupart du temps, vous n'aurez pas besoin de connaître ces mécanismes, mais il me semble qu'on comprend mieux le déroulement du commit en sachant ce qui se passe réellement.

Contrairement à la plupart des données de votre repository Git, la zone d'indexation n'est pas synchronisée ni partagée avec les autres membres de votre équipe : elle n'existe que sur votre ordinateur.

DEUXIÈME COMMIT

Le premier commit vous aura sans doute paru très long, avec toutes mes digressions sur les magasins de données et la nature sémantique des objets Git, mais vous verrez progressivement que ces trois commandes — `status`, `add` et `commit` — composeront l'essentiel de votre interaction avec Git. Vous pouvez utiliser ce processus générique de commit des dizaines de fois par jour et, hormis certaines situations un peu délicates, ces trois commandes vous suffiront.

Pour illustrer cela, réalisons notre deuxième commit en ajoutant un fichier CSS de base qui sera utilisé par notre page d'accueil. Tout d'abord, dans le répertoire de votre projet, utilisez la commande `mkdir` pour créer un nouveau sous-répertoire appelé « css ». Ouvrez ensuite votre éditeur de texte, créez un nouveau fichier et saisissez ce qui suit :

```
body {
  font-family: 'source-sans-pro', Arial, sans-serif;
  font-size: 100%;
}
```

Nommez ce fichier « style.css » et ajoutez-le au sous-répertoire `css/`. Ajoutons maintenant un lien vers cette nouvelle feuille de styles dans notre fichier index.html :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Bonjour</title>
     <link href="css/styles.css" type="text/css" »
       media="all">
  </head>
  <body>
    <h1>Bonjour</h1>
  </body>
</html>
```

Récapitulons ce qui vient de se produire dans le système de fichiers : on a d'abord ajouté un répertoire (`css/`) contenant un fichier (styles.css), puis on a modifié un fichier existant. Vérifions maintenant `git status` :

```
(master *) $: git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be
  committed)
#   (use "git checkout -- <file>..." to discard
  changes in working directory)
#
# modified:    index.html
#
# Untracked files:
```

```
#   (use "git add <file>..." to include in what will be
 committed)
#
# css/
no changes added to commit (use "git add" and/or
  "git commit -a")
```

Sous l'intitulé `Changes not staged for commit`, on retrouve le fichier HTML que Git connaît déjà (parce que nous l'avons committé auparavant), et qu'il présente maintenant (à juste titre) comme modifié. En dessous se trouve à nouveau la liste `Untracked files` mais à la place du fichier styles.css ajouté précédemment, figure le répertoire `css/` qui le contient. Git indique de cette manière qu'il ne connaît pas du tout ce sous-répertoire.

Ces deux changements peuvent être indexés à l'aide d'un simple `git add`. Nous allons maintenant indiquer à la fois le fichier HTML et tout le répertoire `css/` comme arguments en les séparant par une espace pour indiquer qu'il faut indexer séparément chaque élément de la liste. En français, cela revient à dire : « Git, ajoute ceci et cela. »

```
(master *) $: git add index.html css/
```

Vérifions à nouveau `git status` :

```
(master *) $: git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# new file:    css/styles.css
# modified:    index.html
#
```

Les deux fichiers sont indexés et prêts à être committés. Git a remarqué que index.html a été modifié par rapport à une version précédente, tandis que styles.css (et le répertoire `css/`

où il se trouve) sont de nouveaux éléments dans ce commit. Committons.

```
(master *) $: git commit -m "Ajouté feuille de styles"
```

Grâce à ce deuxième commit, notre projet a maintenant un historique. Nous nous pencherons plus en détail sur le commit log de Git dans le prochain chapitre, mais on peut déjà voir la chronologie prendre forme en exécutant `git log`.

Le processus d'indexation et de commit des fichiers couvre une très grande partie de nos besoins en matière de gestion de versions. Et jusqu'ici, tout est (relativement) sensé. On peut trouver un peu étrange de devoir indexer les fichiers avant de les committer, mais comme nous le verrons plus tard, cela peut aussi être très puissant. Quoi qu'il en soit, pour le moment ce n'est qu'un désagrément mineur et non une transformation profonde de la façon dont on gère les fichiers.

D'autres changements, en revanche, sont moins intuitifs. Nous en verrons quelques-uns sous peu.

RETIRER DES FICHIERS DE GIT

Si l'on supprime un fichier de la copie de travail d'un projet, il faut également le retirer du repository. Certes, la commande Git qui sert à supprimer les fichiers, `git rm`, s'applique à faire croire qu'elle se contente de supprimer un fichier.

Cela dit, rappelons deux choses que nous savons déjà : Git est un système d'accumulation, et Git ne soucie des changements que dans le contexte d'un commit. Et c'est là qu'on rencontre notre premier paradoxe logique sérieux dans l'utilisation de Git.

Pour ceux qui débutent avec la ligne de commande, `rm` (qui signifie *remove*, ou supprimer en anglais), est la commande Unix standard pour la suppression des fichiers. Dans un terminal, taper `rm chemin/de/mon/fichier` supprime le fichier désigné. `git rm` se comporte de façon similaire, avec une fonction supplémentaire : en plus de supprimer le fichier, la commande

indexe également un nouveau commit indiquant la suppression du fichier. En d'autres termes, pour supprimer un fichier, il faut ajouter un commit.

Ce processus peut sembler contradictoire : voyons un exemple pour illustrer la suppression d'un fichier.

Imaginons que depuis notre dernière séance de travail sur le projet web, un collègue a ajouté un fichier robots.txt pour indiquer aux moteurs de recherche de ne rien indexer sur le site. (Nous verrons plus tard comment les changements effectués par d'autres personnes sont répercutés dans nos repositories. Pour le moment, disons que le temps a passé et que notre projet a intégré ces changements.)

Nous avons finalement changé d'avis et il faut désormais qu'il soit indexé. Nous devons donc retirer le fichier robots.txt.

Pour cela on utilise `git rm` :

```
(master) $: git rm robots.txt
rm 'robots.txt'
[master *] $:
```

En exécutant `git status`, on voit que la suppression du fichier a été indexée pour être incluse dans le prochain commit :

```
[master *] $: git status
# On branch master
# Your branch is ahead of 'origin/master' by 1
  commit.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# deleted: robots.txt
#
```

Les snapshots de Git, sur lesquels les commits sont basés, incluent le contenu des fichiers et les structures de répertoires

qui les contiennent. Quand on supprime un fichier avec `git rm`, Git crée un nouveau snapshot du projet sans robots.txt et indexe cette version comme la prochaine à committer. Procédons maintenant au commit :

```
[master *] $: git commit -m "Retiré robots.txt"
[master 983024f] Retiré robots.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 robots.txt
```

Git et la corbeille

Vous avez sans doute l'habitude de supprimer un fichier en faisant simplement glisser son icône vers la Corbeille. Ou, si vous utilisez un outil d'édition tout-en-un comme Coda, vous supprimez peut-être les fichiers en utilisant son gestionnaire intégré. Même si vous savez que Git possède une commande « supprimer fichier », il peut être difficile de surmonter des années de mémoire musculaire quand l'outil de suppression est juste ici.

Sachez que je compatis parce que même moi, en tant que programmeur qui utilise la ligne de commande et Git depuis très longtemps, je supprime pratiquement toujours les fichiers à l'ancienne (autrement dit, de la façon que je trouve la plus pratique) avant d'informer Git de la suppression.

Et la commande pour ce faire est exactement la même : `git rm`. Si vous supprimez un fichier, la tâche de `git rm` se limite à indexer une nouvelle modification qui le supprime de l'index de Git.

Donc, après avoir supprimé le fichier robots.txt à l'ancienne, il suffit d'exécuter :

```
[*] $: git rm robots.txt
rm 'robots.txt'
```

Git nous informe qu'il a exécuté `rm 'robots.txt'` pour supprimer le fichier du disque dur, même si ce n'était pas nécessaire. Ce n'est pas un problème : la commande `rm` ne fait rien de plus si le fichier a déjà été supprimé.

RENOMMER DES FICHIERS, OU L'ATTACHEMENT DÉRAISONNABLE DE GIT AUX NOMS DE FICHIERS

Il s'agit maintenant de renommer la feuille de styles principale de notre site, styles.css, en screen.css (en vue de l'ajout ultérieur d'un print.css). La mémoire musculaire étant ce qu'elle est, on risque d'utiliser la commande de renommage que l'on connaît le mieux. Disons que nous avons renommé ce fichier dans Finder d'OS X, et vérifions maintenant `git status` :

```
[master *] $: git status
# On branch master
# Your branch is ahead of 'origin/master' by 2
  commits.
#   (use "git push" to publish your local commits)
#
# Changes not staged for commit:
#   (use "git add/rm <file>..." to update what will
  be committed)
#   (use "git checkout -- <file>..." to discard
  changes in working directory)
#
# deleted:    css/styles.css
#
# Untracked files:
#   (use "git add <file>..." to include in what will
  be committed)
#
# css/screen.css
no changes added to commit (use "git add" and/or
  "git commit -a")
```

Nous avons maintenant un changement non indexé indiquant la suppression totale de la feuille de styles. Mais pourquoi ? Où est-elle passée ?

Vous la retrouverez si vous regardez sous `Untracked files`, mais Git nous dit qu'il considère notre fichier renommé comme un fichier entièrement nouveau et non suivi. Cela semble peut-

être complètement fou pour un être humain, mais Git a une vision du monde extrêmement littérale et basée sur les noms de fichiers (et plus spécifiquement, sur votre système de fichiers). Et donc tout est parfaitement logique.

Git indique que le fichier qu'il suivait sous le nom « css/style.css » n'est plus présent sous ce nom. Il indique également qu'il ne suit pas le fichier appelé « css/screen.css », puisque nous ne lui avons jamais demandé de le faire.

Bien sûr, nous savons que ces deux noms désignent le même fichier. Mais Git ne le sait pas, parce que Git s'appuie uniquement sur les noms des fichiers pour savoir s'il les connaît ou non. Pour nous, les choses sont simples et logiques : ce n'est qu'un changement de nom. Mais pour éviter de faire de mauvaises suppositions sur une modification et risquer ainsi un commit erroné, Git ne tire aucune conclusion quand vous faites une modification par une méthode autre qu'une commande `git`.

Et comme on pourrait croire que Git peut facilement gérer ce genre de choses à notre place, ce comportement peut s'avérer très frustrant la première fois qu'on le rencontre. Mais en réalité, je considère que c'est une nouvelle démonstration de la simplicité de Git. Sa tâche consiste à suivre les modifications et les commits. C'est tout. Git est capable de reconnaître que le fichier styles.css supprimé et le nouveau fichier screen.css non suivi ont exactement le même contenu, mais il n'a aucune idée de ce que cela signifie. Il n'est pas capable de passer à l'étape suivante et de décider que les deux chemins désignent le même fichier, car cet état de fait relève de la sémantique : c'est le sens que vous lui aviez attribué et non une caractéristique intrinsèque aux données.

Par exemple, que se passe-t-il si vous voulez créer un deuxième exemplaire de ce fichier, sous un autre nom ? Et si vous supprimez styles.css par accident ? Git doit prévoir ces possibilités, même si cela nous semble un peu ridicule.

Mais revenons à `git status` : nous avons un fichier manquant et un nouveau fichier mystérieux. Nous savons qu'il s'agit du même fichier, mais Git ne le sait pas. Essayons d'abord la commande `git mv` (raccourci de *move*, déplacer en anglais),

l'outil de renommage conventionnel de Git. Voici ce qui se passe :

```
[master *] $: git mv css/styles.css css/screen.css
fatal: bad source, source=css/styles.css,
  destination=css/screen.css
```

Contrairement à `git rm`, qui se fiche bien de savoir si le fichier supprimé l'a effectivement été ou non (et qui n'a même pas besoin que ce soit le cas), la commande `git mv` ne renomme un fichier que s'il a l'autorisation de le déplacer ou de le renommer dans la copie de travail. Si nous avions utilisé `git mv` et non Finder pour ce faire dès le départ, cela aurait très bien fonctionné. Mais ce n'est pas le cas, et il faut trouver un autre moyen d'indexer et de committer ce changement.

Git reconnaît deux changements non committés : une suppression (`css/style.css`) et un ajout (`css/screen.css`). Chacun d'eux doit être traité séparément avant de committer.

Utilisons d'abord `git rm` pour mettre fin au suivi de styles.css :

```
[master *] $: git rm css/styles.css
```

On utilise ensuite `git add` pour lui demander de suivre le fichier sous son nouveau nom :

```
[master *] $: git add css/screen.css
```

Une fois ces deux opérations effectuées, vérifions le statut :

```
[master *] $: git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    css/styles.css -> css/screen.css
    modified:   index.html
```

C'est exactement la réponse qu'on aurait obtenue en utilisant `git mv` dès le départ. Vous pouvez supprimer, déplacer et renommer les fichiers à l'aide des commandes intégrées de Git pour économiser un peu de saisie, mais ce n'est pas indispensable. C'est un exemple typique de situation où Git semble devenir vraiment problématique, alors qu'il s'agit en fait d'un désagrément mineur. C'est la fréquence à laquelle vous renommez les fichiers qui déterminera si vous préférez faire les choses à la manière de Git ou selon vos habitudes, en sachant que vous pourrez toujours vous expliquer avec Git un peu plus tard.

SUPER BANCO : git add --all

En cas de doute (ou de manque de temps), Git possède une arme d'indexation massive qui prend en compte tout ce qui a changé dans votre copie locale : `git add --all` (ou `git add -A` pour les intimes). L'option `--all` est idéale quand vous voulez committer plusieurs changements simultanément. Par exemple, on a commencé à ajouter des comportements JavaScript à la page en créant un nouveau fichier (ainsi qu'un répertoire) appelé `js/site.js` et en le référençant dans le document HTML. Notre statut est le suivant :

```
[master *] $: git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be
  committed)
  (use "git checkout -- <file>..." to discard
  changes in working directory)

    modified:   index.html

Untracked files:
  (use "git add <file>..." to include in what will
  be committed)

    js/
```

Le répertoire `js` (qui contient le fichier de script `site.js`) est nouveau dans le projet et s'affiche donc comme non suivi. Nous avons également modifié index.html pour ajouter une balise script référençant le nouveau fichier. Il se présente comme modifié mais non indexé.

On pourrait informer Git de ces changements séparément en saisissant les chemins de chaque fichier et répertoire à indexer. Mais nous allons plutôt utiliser `git add --all` et vérifier le statut :

```
[master *] $: git add --all
[master *] $: git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   index.html
    new file:   js/site.js
```

Grâce à cette simple commande, tout ce que nous avons fait est prêt à committer.

Cet exemple est plutôt simple, mais on peut le rendre plus complexe. Avant de committer, nous décidons de donner un nom plus descriptif, `scripts`, au répertoire contenant les scripts, `js`, en utilisant la commande `mv` (déplacer) et en éditant notre document HTML afin de référencer le fichier JavaScript sous son nouveau nom :

```
[master *] $: mv js scripts
[master *] $: git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   index.html
    new file:   js/site.js

Changes not staged for commit:
```

```
  (use "git add/rm <file>..." to update what will be
  committed)
  (use "git checkout -- <file>..." to discard
  changes in working directory)

    modified:   index.html
    deleted:    js/site.js

Untracked files:
  (use "git add <file>..." to include in what will
  be committed)

    scripts/
```

Il se passe beaucoup de choses, mais pour une fois, ce statut complexe est relativement facile à suivre et à expliquer. On voit d'abord les deux modifications déjà indexées : elles restent indexées bien que d'autres modifications apportées à la copie de travail semblent les avoir remplacées. Si l'on committait maintenant, le répertoire de script serait nommé `js` dans le repository bien que son nom soit « scripts » dans la copie de travail.

Outre les modifications que nous avons indexées, Git mentionne également les modifications plus récentes et encore non indexées. Comme avant, comme il ne voit plus le fichier `js/site.js` sous ce nom, il le présente comme supprimé, et le fichier `scripts/site.js` qui le remplace apparaît entièrement nouveau. On voit également index.html listé deux fois, dans deux états différents à la fois : modifié et indexé, mais aussi modifié et non indexé. `git add` enregistre une copie de l'état d'un fichier dans la base de données de données de Git en vue de son inclusion dans un commit ; ici, Git essaie de nous dire qu'il existe une version encore plus récente d'index.html que celle que nous avons indexée auparavant.

Heureusement, `git add --all` vient mettre de l'ordre dans ce joyeux bazar :

```
[master *] $: git add --all
[master *] $: git status
```

```
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   index.html
    new file:   scripts/site.js
```

Tous ces changements apparemment contradictoires ont été condensés en deux modifications compréhensibles. Il ne reste plus qu'à committer :

```
[master *] $: git commit -m "Ajouté JavaScript"
[master 4af326c] Ajouté JavaScript
 2 files changed, 1 insertion(+)
 create mode 100644 scripts/site.js
```

C'est très bien, et c'est bien plus simple. D'où la question : pourquoi ne pas toujours utiliser `git add --all` ? Très sincèrement, dans la plupart des cas, `--all` est plus qu'une option acceptable : c'est la meilleure. Non seulement elle vous fera gagner du temps, mais en réduisant le nombre de commandes on réduit également les risques de transmettre accidentellement à Git une instruction erronée, cause de bien de confusions et de chagrins. En utilisant cette option, vous dites à Git que la version du projet qui figure dans votre copie de travail est un reflet exact de ce que vous souhaitez committer.

Et de toute façon, le fait que vous puissiez committer tout ce qui a changé dans votre copie locale, sans y être obligé, est l'une des fonctions les plus puissantes de Git. C'est tout l'intérêt de la zone d'indexation : on exerce un contrôle précis sur la portée de chaque commit qui sera aussi large ou réduite que souhaité, selon ses habitudes de travail ou les besoins du projet concerné.

3 BRANCHES

Nous avons beaucoup parlé de « versions » jusqu'ici, mais de quoi s'agit-il exactement ? Pour le dictionnaire, c'est « une forme particulière de quelque chose, différente à certains égards d'une forme antérieure ou d'une autre forme du même type de chose. » Autrement dit, les versions peuvent être, d'une part, *séquentielles* ou *itératives*, représentant alors la forme d'une chose dans son évolution au fil du temps ; et d'autre part, elles peuvent différer selon d'autres critères. L'important est qu'une version n'est pas qu'une copie d'un objet, c'est une copie qui diffère ou varie d'un autre exemplaire à un ou plusieurs égards.

Dans Git, la gestion de versions séquentielles — qui consiste à suivre la différence entre un snapshot de votre travail et ses formes précédentes — est l'une des nombreuses tâches assurées par le commit. Chaque commit inclut une référence à la précédente, ou *commit parent* ; à partir de cette référence, Git peut revenir en arrière et expliquer toute la chaîne de commits à rebours. En ce sens, un commit représente à la fois votre travail dans une forme particulière et son évolution par rapport à sa forme précédente.

Ce type de relation entre les versions permet de comprendre d'où vous venez mais pas toujours de savoir où vous allez ni pourquoi. C'est là que les branches deviennent utiles. Une branche est une copie virtuelle de votre projet — un projet dans le projet — où vous pouvez committer librement des opérations de façon isolée par rapport au reste de votre repository. Les branches permettent de gérer et d'utiliser d'autres types de versions dans Git, de faire des expériences, d'essayer d'autres pistes, le tout séparément de la copie « officielle » du travail que représente la branche `master`.

Beaucoup ont vaillamment tenté d'expliquer les branches de Git en les comparant à des objets réels, mais aucune analogie ne peut rendre entièrement justice à ces magnifiques et puissants réservoirs d'information pure. La meilleure manière de comprendre les branches est de le faire dans leurs propres termes : elles sont un moyen d'organiser et de décrire le travail. Et pour bien l'expliquer, le mieux est encore de se jeter à l'eau et de faire pousser quelques branches.

PRINCIPE D'EMBRANCHEMENT

Chaque repository Git commence avec une branche principale à laquelle Git donne le nom de `master` par défaut. Sur le plan technique, `master` est une branche comme les autres ; ce qui la rend spéciale est seulement son rôle conventionnel de version primaire, stable, du projet qui se trouve dans un repository. Après, les mots « primaire » et « stable » ont le sens que vous leur donnez, et chaque équipe utilise ses branches `master` de différentes manières. La seule chose qui est gravée dans le marbre, c'est que la branche `master` est la première avec laquelle vous travaillerez. Mais ce ne sera pas la dernière.

Avant de créer notre première branche, nous allons passer en revue la liste de toutes les branches du repository local à l'aide de la commande `git branch` :

```
(master) $: git branch
* master
```

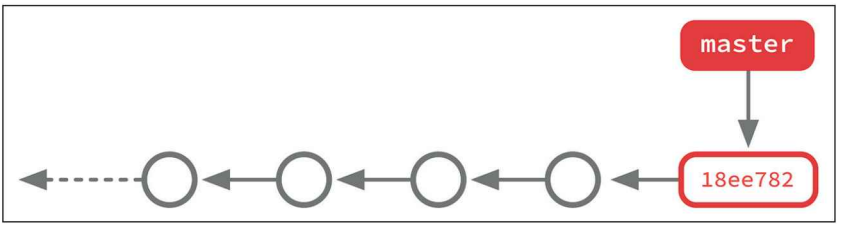


FIG 3.1 : Le commit en cours est encadré en rouge, et le nom de la branche actuelle pointe vers ce commit.

Pour le moment, il n'y a qu'une seule branche, `master`, précédée d'un astérisque qui signale qu'il s'agit de la branche en cours.

En coulisse, une branche n'est rien de plus qu'un surnom lisible pointant vers un commit particulier. Le schéma de la **FIG 3.1** montre le commit en cours (`18ee782`) encadré en rouge, ainsi que le nom de notre branche actuelle, `master`, pointant vers le commit. Cette branche n'est qu'une pile de commits, à la tête de laquelle se trouve `18ee782`.

(Pour filer la métaphore arboricole, le commit de la tête d'une branche est parfois appelée « sommet ».) On reconstitue la pile en remontant la chaîne de commits parents à partir du commit de tête aussi loin que possible, jusqu'au premier commit du repository.

Quand on parle des branches, on est parfois tenté de les désigner comme des lieux : quand on ajoute un commit, on « l'envoie » vers la branche `master`. C'est effectivement une métaphore pratique au quotidien. « Où est-ce que j'ai committé les nouveaux liens du pied de page ? » : une question raisonnable dont la réponse est souvent un nom de branche. Il n'y a pas de mal à traiter les noms de branche comme des dossiers dans le gros classeur de notre projet.

Mais cette analogie a naturellement ses limites. En effet, si l'on ne peut mettre une feuille de papier que dans un seul dossier à la fois, les commits de Git peuvent appartenir à plusieurs branches — et donc se trouver à plusieurs endroits — à la fois. Mais ce n'est pas un problème ! Au bout du compte, le rôle le

plus important d'une branche est celui de panneau de signalisation ou de signet pointant vers une version particulière de votre travail, distinguant la branche `master` d'une autre branche nommée, par exemple, `nouvelle-page-accueil`. À cet égard, les noms de branches ont une fonction comparable à celle des libellés de Gmail. De même qu'un e-mail peut être à la fois sous les libellés « Boîte de réception » et « Messages de Maman », un même commit peut se trouver à la fois dans les branches `master` et `nouvelle-page-accueil`. Les noms de branches ne sont pas tant des destinations que des étiquettes ou des panneaux permettant de retrouver certains commits.

OUVRIR UNE NOUVELLE BRANCHE

Notre projet suivant est une mission de grande importance : la refonte de la page d'accueil de notre site. Il faudra sûrement du temps et beaucoup de commits pour y parvenir. Mais il n'est pas question de publier notre travail avant qu'il ne soit terminé, et nous ne voulons pas non plus empêcher nos collègues d'apporter des modifications au site ou à la page en place pendant que nous progressons.

Il faut donc disposer d'un endroit sûr pour apporter des changements potentiellement importants sans perturber le reste des opérations.

Le lieu idéal pour ça ? Une branche distincte. En jargon Git, c'est une *branche de travail* ou une *branche thématique*. Les branches thématiques se distinguent de la branche `master` par le fait qu'elles ont effectivement un thème : le travail qui s'y déroule relève d'un objectif particulier que décrit généralement le nom de la branche. Comme il s'agit de produire une nouvelle page d'accueil, créons une branche baptisée `nouvelle-page-accueil`.

Pour créer une nouvelle branche, il suffit de passer son nom dans la commande `git branch`, comme suit :

```
(master) $: git branch nouvelle-page-accueil
```

On demande ainsi à Git de créer une nouvelle branche appelée `nouvelle-page-accueil` en utilisant le commit en cours comme point de départ. Peu importe quelle branche est définie comme branche actuelle : `git branch` ne se soucie que du commit en tête de la branche. Pour le moment, c'est le commit `18ee782` qui est en tête de la branche `master`, et notre toute nouvelle branche `nouvelle-page-accueil` va donc également démarrer avec le commit `18ee782` à sa tête. Toutefois, ces deux branches n'ont aucun lien entre elles, excepté qu'elles possèdent toutes les deux le commit `18ee782`.

Chose agaçante s'il en est, Git ne bascule pas automatiquement dans la nouvelle branche à l'exécution de la commande `git branch <nombranche>`. Git crée une nouvelle branche mais `master` reste la branche en cours ou (en termes Git) la branche extraite. Si l'on reste dans cet état, Git a créé la nouvelle branche, mais le prochain commit sera fait dans la branche `master`.

L'extraction d'une branche a deux effets. Tout d'abord, elle réinitialise la copie de travail qui devient le reflet du commit de tête de la branche. Elle définit ensuite la branche choisie comme branche actuelle, de sorte que chaque nouveau commit sera ajouté à la branche en question.

Pour changer de branche, il faut donc utiliser la commande `git checkout`.

```
(master) $: git checkout nouvelle-page-accueil
Switched to branch 'nouvelle-page-accueil'
(nouvelle-page-accueil) $:
```

Une fois que vous aurez l'habitude de travailler avec des branches, la petite ritournelle consistant à créer une branche puis à y basculer peut devenir fatigante. Heureusement, Git propose un raccourci pratique : vous pouvez utiliser la commande `git checkout` pour créer une nouvelle branche et y basculer en une seule opération, grâce à l'option `-b`, comme ceci :

```
(master) $: git checkout -b nouvelle-page-accueil
Switched to a new branch 'nouvelle-page-accueil'
(nouvelle-page-accueil) $:
```

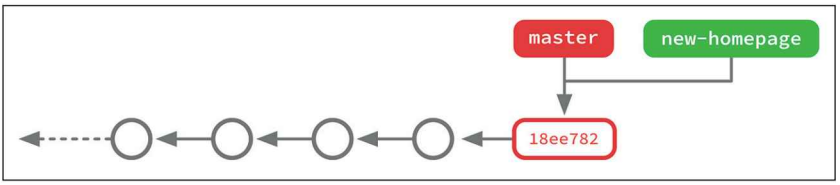


FIG 3.2 : En l'absence de nouveaux commits, ces deux branches sont littéralement identiques.

Dans tous les cas, une fois que vous avez créé une nouvelle branche et que vous vous y êtes déplacé, vous devriez la voir en tapant la commande `git branch` :

```
(nouvelle-page-accueil) $: git branch
  master
* nouvelle-page-accueil
```

L'astérisque indique que la branche `nouvelle-page-accueil` est désormais la branche en cours (comme prévu, donc), et c'est donc là que seront envoyées nos prochains commits.

Pour le moment, comme nous n'avons ajouté aucun commit aux branches `master` ni `nouvelle-page-accueil`, ces deux branches sont littéralement identiques : les deux noms pointent vers le même commit.

Mais l'un des grands avantages de la gestion des branches par Git réside dans le fait que, la plupart du temps, vous n'avez pas à vous soucier de cela : bien qu'ils soient parfaitement identiques, ces deux objets sont distincts sur le plan logique. Bien que les branches `master` et `nouvelle-page-accueil` aient des contenus identiques, ce sont deux exemplaires logiquement différents de notre travail, et non pas deux noms différents pour le même exemplaire.

Vous n'avez pas à vous demander quelle est la branche en cours, car bien que ces deux branches soient identiques pour le moment, cela ne sera sans doute pas longtemps le cas. En extrayant la branche `nouvelle-page-accueil`, on manifeste notre intention de diverger de la chronologie officielle et

d'apporter des modifications qui ne se trouveront pas nécessairement dans la version de production du code.

Prenons quelques instants pour savourer quelques détails sémantiques. Comme je l'ai dit, les branches `master` et `nouvelle-page-accueil` sont actuellement identiques. En pratique, cela signifie que `18ee782` est le commit de tête des deux branches. On peut donc dire que les deux branches contiennent ce commit ainsi que son commit parent immédiat ainsi que chaque commit qui le précède directement, jusqu'au commit initial de ce repository. Comme l'une des branches en question est la branche `master` (et que nous n'avons rien committé ailleurs que dans `master`), tout l'historique de notre projet se trouve dans l'une et l'autre de ces branches.

Je fais ce rappel parce que nous sommes sur le point de faire notre premier commit dans une branche autre que `master`. Après avoir extrait la branche `nouvelle-page-accueil`, tout ce qu'on fait appartient à la branche thématique et ne figure pas dans la branche `master`.

Pas encore.

SUR LE FIL

Quelles que soient nos ambitions pour la refonte de la page d'accueil, il faut bien commencer quelque part. Pour le moment, commençons par quelque chose de simple : remplacer le dégradé bleu de l'en-tête du site par un gris uni, parce que l'heure est au flat design.

On modifie la CSS et on committe le changement :

```
(nouvelle-page-accueil *) $: git commit -am "Modifié
  couleur de fond en-tête"
[nouvelle-page-accueil b26b038] Modifié couleur de
  fond en-tête
1 file changed, 1 insertion(+), 1 deletion(-)
```

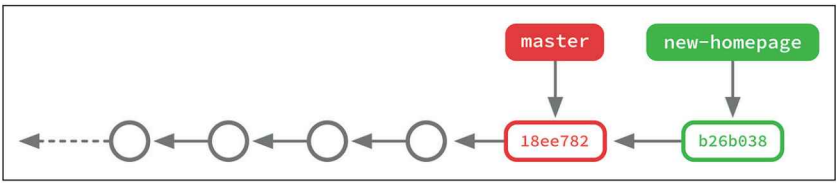


FIG 3.3 : Ces deux branches se superposent parfaitement à un commit près.

Avant de poursuivre, je tiens à attirer votre attention sur un point. L'option `-am` (que nous avons utilisée dans l'exemple précédent) combine deux autres options rencontrées plusieurs fois : `-a` (pour ajouter automatiquement tous les fichiers modifiés à ce commit) et `-m` (pour préciser un message de commit). La plupart des outils en ligne de commande permettent de combiner plusieurs options de cette façon, en les faisant précéder d'un tiret simple. Une restriction cependant : une seule de ces options peut prendre un argument (en l'occurrence, `m`) et elle doit figurer en dernier. Cette combinaison particulière d'options est très pratique, car elle permet dans de nombreux cas de committer plusieurs changements avec une seule commande.

Une fois ce commit ajouté, la branche de travail et la branche `master` divergent : la nouvelle branche `nouvelle-page-accueil` possède un commit que la branche `master` ignore.

Le diagramme de la **FIG 3.3** représente plus ou moins ce que Git voit de l'état actuel de notre repository. Les deux branches se superposent parfaitement à l'exception du commit qui n'existe que sur la branche `nouvelle-page-accueil`. C'est comme cela que les branches fonctionnent en réalité : la branche `nouvelle-page-accueil` pointe vers `b26b038`, la branche `master` pointe vers `18ee782`, et `b26b038` pointe vers son commit parent, `18ee782`.

On peut également voir les deux branches comme des copies logiques discrètes de l'intégralité de la chronologie, et qui ont accidentellement la particularité d'être quasiment identiques (**FIG 3.4**).

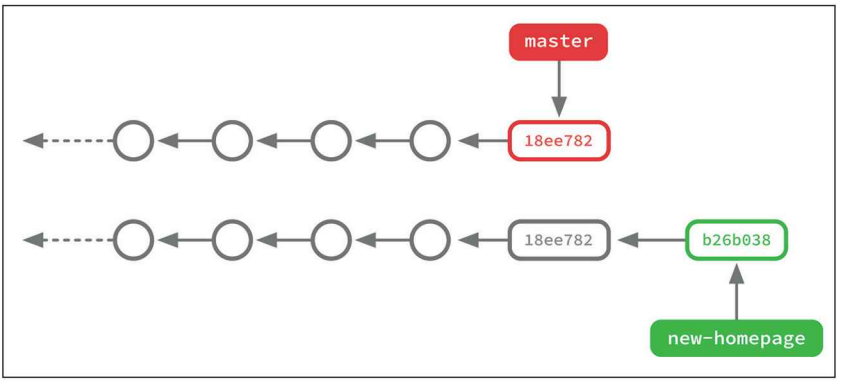


FIG 3.4 : Les deux branches peuvent être considérées comme des copies logiques discrètes qui sont, par coïncidence, quasiment identiques.

En termes techniques, Git crée le commit, mais déplace aussi le pointeur de la branche en cours vers le commit qui vient d'être effectué. Plus simplement, lorsque vous ajoutez un commit à une branche, le nouveau commit remplace le précédent à la tête de la branche. Grâce à la magie des commits parents, on peut revenir en arrière et retracer la lignée d'une branche jusqu'à son point de départ, et ainsi produire des graphiques présentant l'historique commun des deux branches. Mais en pratique, l'aspect le plus important d'une branche donnée est généralement le commit qui se trouve sur le dessus de la pile.

En effet, c'est sur le commit de tête de la branche que sera basé notre prochain commit. Travailler sur des branches Git s'apparente souvent à participer à un cadavre exquis (http://bkaprt.com/gfh/03-01/ ou https://fr.wikipedia.org/wiki/Cadavre_exquis_(jeu) en français), œuvre d'art collaborative inventée par les surréalistes au XXe siècle, et dans laquelle chaque contributeur ne voit que les derniers éléments ajoutés par son prédécesseur. Par exemple, un premier artiste commence un dessin sur le premier tiers d'une feuille de papier. Il replie ensuite la feuille de manière à couvrir la majeure partie de sa contribution puis la passe à quelqu'un d'autre qui remplit le deuxième tiers, et ainsi de suite. Le « layer tennis » est une version moderne du cadavre exquis, dans laquelle deux designers s'échangent un fichier

Photoshop en rajoutant à chaque fois un calque supplémentaire (http://bkaprt.com/gfh/03-02/).

Quand vous travaillez avec Git, essayez de ne pas vous soucier du système de commits, de branches et de chronologies. Tout cela peut être extrêmement utile et sera à portée de main quand vous en aurez besoin. Mais pour le moment, n'hésitez pas à vous concentrer sur le passage de votre dernier commit au suivant, étape par étape, incrément par incrément.

NOMMER LES BRANCHES

Il n'existe pas de règle définitive quant à l'utilisation des branches. Comme je l'ai indiqué, chaque équipe peut employer la fonction d'embranchement de manière entièrement unique.

Il existe tout de même quelques conventions, mais elles sont elles-mêmes pleines d'ambigüités. Par exemple, chaque repository Git possède une branche `master`, considérée par convention comme la branche « principale » ou « par défaut » de votre projet. Mais c'est à vous de définir ce que signifie exactement « principale » ou « par défaut » dans le contexte de votre travail.

Dans certains projets, en particulier les grands projets open source comme Ruby on Rails, la branche `master` sert à tout le travail de pointe qui sera publié dans la prochaine grande version, et d'autres branches parallèles sont créées pour finaliser et préparer le code qui sera intégré dans les versions intermédiaires du framework Rails. Une autre approche, plus courante pour les sites et les applications web, consiste à faire de la branche `master` le reflet de la version publiée du projet déployé sur les serveurs web, potentiellement plusieurs fois par jour.

Ces deux types de projets partagent souvent la même façon d'utiliser les branches thématiques. Bien qu'il n'y ait pas une manière unique de faire les choses, on observe une communauté de pratique dans la façon dont les équipes utilisent Git pour accomplir leurs tâches.

Tout d'abord, quelqu'un extrait la dernière version de la branche `master` puis, à partir de ce commit, crée une nouvelle branche qu'il nomme selon le travail qu'il prévoit de faire, comme dans l'exemple de la branche `nouvelle-page-accueil` que nous avons créée plus tôt. La branche `nouvelle-page-accueil` est un bon exemple de branche thématique : le thème est l'exploration d'un nouveau design pour la page d'accueil, et l'embranchement permet d'expérimenter librement sans craindre de perturber la version principale figurant dans `master`.

Une fois l'embranchement effectué, on est libre de travailler sur cette nouvelle idée ou fonctionnalité pendant un moment, en ajoutant des commits à la branche au fur et à mesure. Pendant ce temps, `master` continue d'évoluer de son côté et accueille de nouveaux commits qui ne se trouvent pas dans la nouvelle branche. Aucun aspect de l'historique commun de la nouvelle branche et de `master` ne changera suite au travail effectué dans chacune de ces branches, et toutes les modifications apportées ensuite sur une branche sont isolées de celles qui sont faites sur l'autre.

Par exemple, à l'approche de la nouvelle année, vous aurez besoin d'actualiser la mention de copyright du site alors que vous êtes en pleine refonte de la page d'accueil. Une fois que vous avez committé vos modifications dans la branche `nouvelle-page-accueil`, vous n'êtes qu'à une commande `git checkout` de revenir dans la branche `master` :

```
[nouvelle-page-accueil] $: git checkout master
Switched to branch 'master'

[master] $:
```

Une fois que vous avez modifié l'année et committé le changement, vous pouvez revenir facilement à votre travail à l'aide de la commande `git checkout nouvelle-page-accueil`.

Le nom que vous donnez à la branche doit donner une description logique de sa raison d'être. Imaginez une étiquette concise qui identifie le travail en cours. Une branche créée pour corriger un problème dans Chrome 32 pourra s'appeler `correctif-bug-`

`chrome32`, ou bien `correctif-bug-webfont-chrome32` pour être plus précis ou encore `correctif` si l'on veut être plus générique. Choisissez le degré de spécificité qui distinguera cette branche des autres branches de votre projet sans gaspiller de la place. L'espace utilisable dans les listes de branches est encore plus limité que dans les commit logs (bien que, contrairement aux commits, il soit possible de supprimer et de nettoyer les anciennes branches inutilisées). Il n'est pas indispensable que les noms de branche soient parfaitement descriptifs, il faut juste qu'ils soient assez descriptifs.

MERGE

Il arrive parfois qu'une branche soit simplement le lieu d'expérimentations jetables. C'est l'un des grands avantages de l'embranchement dans les systèmes de gestion de versions en général et dans Git en particulier : la création d'une branche est rapide et peu coûteuse, et vous n'êtes jamais tenu de réconcilier la version de travail d'une branche avec celle de `master`.

Mais la plupart du temps, les gens utilisent les branches pour travailler sur des aspects qu'ils prévoient d'intégrer ensuite à la copie principale. De même, comme `master` continue d'évoluer indépendamment des branches thématiques sur lesquelles vous travaillez, vous aurez besoin de synchroniser ces modifications (ou au moins une partie d'entre elles) avec celles de votre branche. L'objectif est que la version du site web enregistrée dans la branche soit aussi fraîche que possible : vous ne voulez pas présenter une version du site où la page d'accueil est toute neuve mais le reste date manifestement d'il y a six mois. Et vous voulez également avoir l'assurance que votre travail s'intègrera facilement à la branche `master` le moment venu.

La fusion (ou *merge*) combine deux (ou plus, mais généralement deux) branches différentes de votre projet en une version unifiée présentant les attributs des deux branches. D'un côté, vous avez la branche `master`, contenant la version de votre site qui se trouve actuellement sur le web. Un collègue de votre équipe découvre une erreur dans les informations de contact et crée

une nouvelle branche thématique nommée `maj-infos-contact` où il corrige l'erreur. Vous voulez maintenant une version de la branche `master` incluant les nouvelles informations de contact figurant dans `maj-infos-contact`.

Pour cela, on extrait d'abord la branche `master` dans la copie locale :

```
[update-contact-info] $: git checkout master
Switching to branch 'master'

[master] $:
```

On utilise ensuite la commande `git merge` pour insérer les modifications de l'autre branche dans la copie de `master` :

```
[master] $: git merge maj-infos-contact
Updating 286af1c..885e3ff
Fast-forward
```

Et voilà ! Vous possédez maintenant une version de `master` contenant tout ce qui s'y trouvait auparavant, ainsi que les nouvelles informations de contact. Prenons un peu de recul pour examiner ce qui vient exactement de se passer sous le capot.

De quoi est fait un merge

Du point de vue dont Git gère vos données, quand on dit qu'on veut obtenir une version de notre branche qui inclue tous les aspects d'une autre branche, deux critères doivent être satisfaits.

Tout d'abord, on veut que tous les commits effectués sur l'autre branche soient visibles dans notre commit log (bien que leur ordre exact soit flexible). Il faut avant tout que l'historique qui nous a conduits à ce moment soit maintenu, et que la chaîne puisse être tracée sur les deux branches afin d'inclure tout le travail qui a été effectué jusqu'à présent.

Ensuite, et surtout, nous avons besoin d'obtenir une copie des fichiers et dossiers du projet qui intègre toutes les modifications

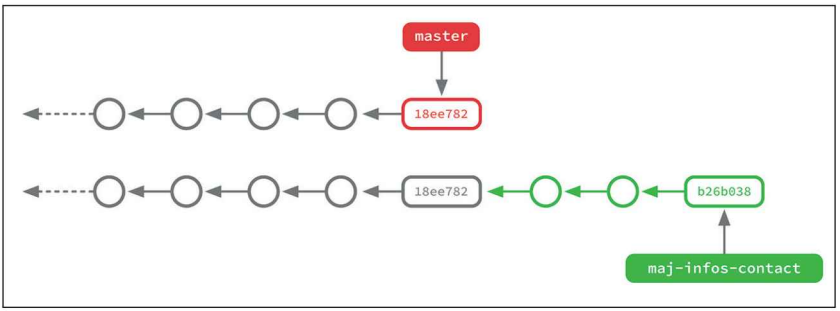


FIG 3.5 : Alors que la branche `maj-infos-contact` contient quelques commits supplémentaires, `master` reste inchangée. Un seul côté du merge a été modifié et l'état fusionné est donc identique à ce qui se trouve dans la branche `maj-infos-contact`.

apportées dans les deux branches. Vous ne serez pas surpris d'apprendre que le résultat d'un merge est un commit et que, comme nous l'avons vu, chaque commit représente un snapshot complet de l'ensemble du projet. Le commit obtenu après un merge ne fait pas exception, ce qui signifie que quelqu'un (ou quelque chose) doit être responsable de la construction de cette nouvelle version unifiée du répertoire du projet.

La plupart du temps, ce « quelque chose » responsable de la construction d'une copie fusionnée du projet n'est autre que Git, qui emploie pour ce faire plusieurs stratégies.

Fast-forward

Le type de merge le plus simple et le plus facile est appelé fast-forward, et correspond en effet exactement à une « avance rapide ». Dans l'exemple de la mise à jour de la page de contact, rien n'a changé dans `master` depuis l'embranchement ; tout ce qui diffère dans la branche `maj-infos-contact` correspond à des éléments ajoutés après le dernier commit sur la branche `master` (**FIG 3.5**).

Dans ce cas, `git merge` n'a rien à faire pour déterminer l'état post-fusion du projet, parce qu'un seul élément du merge a changé. L'état fusionné sera donc identique à celui de la branche `maj-infos-contact`. Il suffit simplement à Git de déplacer le

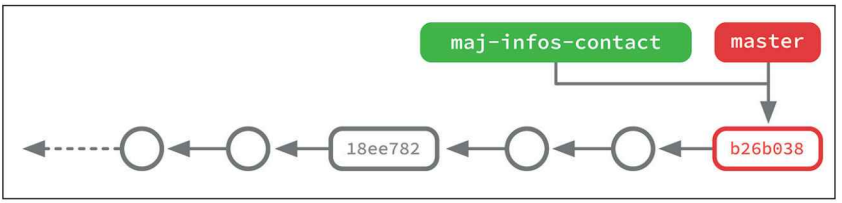


FIG 3.6 : Le signet de la branche `master` passe du commit en cours au commit de tête de l'autre branche.

signet de la branche `master` de son commit actuel vers le commit de tête de l'autre branche (**FIG 3.6**).

Après un fast-forward, les deux branches pointent simplement vers le même commit et deviennent à nouveau identiques à tous égards, à l'exception de leur nom.

Merge commits

Aussi simples et élégants que soient les fast-forwards, ils sont réservés aux cas où une seule des deux branches présente de nouveaux commits, ce qui, selon la taille de votre équipe et le rythme auquel votre projet évolue, ne se produit que rarement en général. Dans tous les autres cas, Git se rabat sur un « vrai merge », dans le cadre duquel il détermine quel doit être l'état combiné du projet, crée un snapshot représentant cette nouvelle version fusionnée puis ajoute un commit d'un type spécifique – un *commit de fusion* ou *merge commit* – pour parachever l'opération.

Pour reprendre notre exemple de correction de coordonnées, imaginons que les branches `master` et `maj-infos-contact` aient toutes les deux changé et contiennent chacune un nouveau commit ajouté après l'embranchement.

Comme les deux branches présentent des changements, Git doit travailler un peu pour que le merge des deux branches se déroule en toute sécurité. Tout d'abord, Git identifie les changements entre les commits de tête de chaque branche en recherchant le premier ancêtre commun des deux versions, avant de revenir sur ses pas pour comprendre ce qui a changé et dans

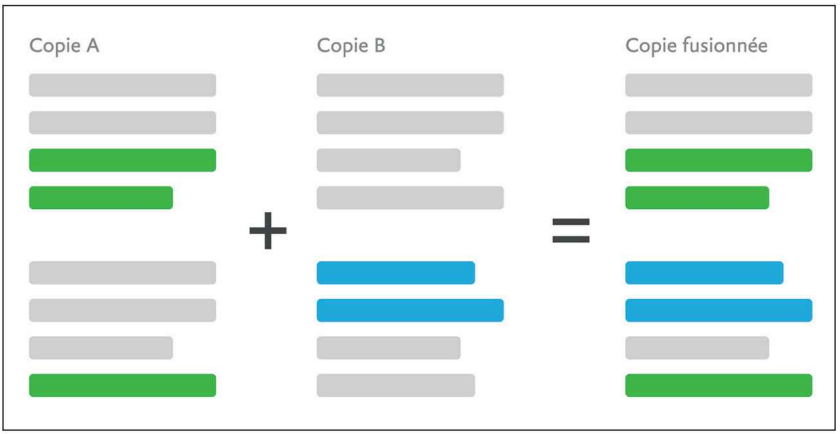


FIG 3.7 : Dans un merge simple, Git ne repère que les lignes modifiées des deux branches et les combine pour produire un troisième snapshot fusionné.

quel ordre, en utilisant l'ancêtre commun comme point de référence. Ensuite, Git compare chaque fichier modifié des deux branches au point de référence. Quand Git identifie une ligne modifiée dans l'une des branches, cette ligne est isolée pour être incluse dans la copie finale fusionnée. Tant que les branches ne contiennent pas des changements au niveau de la même ligne, Git peut tout fusionner automatiquement.

Une fois que le snapshot fusionné a été généré automatiquement, Git conclut l'affaire et la committe pour vous en générant un message de commit automatique : « Merge branch "maj-infos-contact" into master. »

Ce n'est bien sûr qu'une option par défaut et vous pouvez, en passant l'option `--no-commit` à la commande `git merge`, demander à Git de gérer et d'indexer la version fusionnée de votre travail, mais d'attendre que vous la committiez vous-même. C'est notamment utile si vous souhaitez ajouter votre propre message de commit, même si le message automatique suffit dans la plupart des cas. Vous pouvez aussi souhaiter intégrer d'autres changements au même commit, voire opérer des merges dans d'autres branches en un seul commit, en exécutant plusieurs fois la commande `git merge --no-commit`. Cela ne

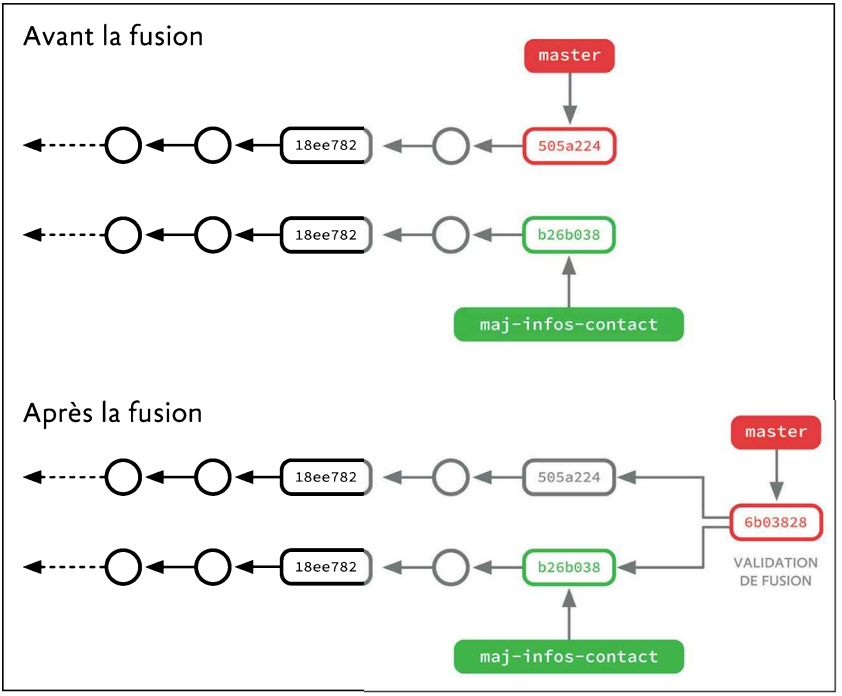


FIG 3.8 : Comme `master` et `maj-infos-contact` ont toutes les deux ajouté un commit supplémentaire depuis leur ancêtre commun `18ee782`, Git crée un merge commit pour les lier, en référençant les deux commits précédents comme parents.

vous fera pas gagner de temps et vous privera d'un merge commit signalant où et quand un merge a été effectué. Bref, à moins d'avoir une raison de procéder autrement, utilisez le merge conventionnel et automatique.

Les merge commits présentent des caractéristiques particulières. Par exemple, contrairement à un commit classique qui ne possède qu'un seul parent, les merge commits peuvent en avoir deux (ou plus mais il s'agit généralement de deux) (**FIG 3.8**). À bien des égards, les merge commits sont comparables aux autres commits et soumis aux mêmes règles.

Pour plusieurs raisons, il vaut mieux éviter qu'une branche thématique ne s'écarte trop de la dernière version de `master`.

En partant du principe que les branches ont une durée de vie courte et que votre objectif est de les fusionner dans `master`, les maintenir proches de `master` facilitera le merge final et réduira les risques de conflits de fusion.

C'est pourquoi, de temps à autre, vous allez fusionner les nouveaux commits apportés à `master` dans votre branche afin de maintenir cette dernière à jour :

```
(nouvelle-page-accueil) $: git merge master
Updating c7038f8..1c4b16a
Fast-forward
 Makefile                    |   7 ++
 Rakefile                    |   15 +++--
```

Idéalement, l'état final de chaque branche de travail doit uniquement différer de `master` sur des aspects propres à sa thématique. Par exemple, la branche `nouvelle-page-accueil` ne doit comprendre que les changements nécessaires à la nouvelle page d'accueil, sans toucher à d'autres aspects ni inclure de régressions vers une version antérieure de `master`. C'est la même chose pour `maj-infos-contact`, `ajout-webfonts` ou toute autre branche thématique en cours d'utilisation.

GÉRER LES CONFLITS DE FUSION

Comme on l'a vu, dans la plupart des cas, Git peut fusionner deux branches automatiquement. Mais il n'est pas toujours évident de déterminer comment deux branches doivent fusionner, auquel cas Git vous demandera de l'aide. C'est l'un des scénarios les plus agaçants de Git, celui qui effraie le plus la première fois qu'on le rencontre, mais qui n'est pourtant pas si terrible : le redouté *conflit de fusion* (ou *merge conflict*).

La plupart du temps, les conflits de fusion surviennent quand deux lignes d'un merge se superposent : autrement dit, si deux versions différentes tentent simultanément de modifier la même ligne d'un même fichier. Dans des circonstances normales, Git n'essaie pas de résoudre lui-même le conflit. Il fait ce

qu'il peut puis s'arrête et vous demande de terminer le merge commit vous-même.

Le processus de résolution d'un conflit de fusion est très semblable à celui d'un commit. Rappelons-nous qu'un merge commit reste un commit. Il a plusieurs parents pour signaler qu'il combine deux versions précédentes de votre projet en un tout unifié. En dehors de cela, il suit les mêmes règles que n'importe quel autre commit, y compris en ce qui concerne sa création : il faut d'abord indexer les changements à inclure avant de les committer.

Ce qui distingue les merge commits réside dans la quantité de travail effectué automatiquement par Git avant de demander votre intervention. Et c'est l'une des nombreuses raisons vicieuses pour lesquelles les nouveaux venus peuvent être rebutés : au moment où Git demande de l'aide, le merge commit est, bien souvent, quasiment déjà indexé. C'est comme si quelqu'un d'autre avait commencé un commit et vous avait demandé de le terminer — et c'est même précisément ce qui est en train de se produire.

Quand Git rencontre un état conflictuel sur l'une des branches, s'il peut déjà fusionner sans souci des fichiers qui ont été modifiés depuis le dernier commit de la branche en cours, les changements concernés sont automatiquement ajoutés à la zone d'indexation. Vous n'avez généralement pas besoin de vous soucier de ces changements ni d'intervenir. Ils peuvent être committés en toute confiance.

C'est là que le docteur vous dit que ça risque de piquer un peu : quand Git ne peut pas fusionner automatiquement deux copies du même fichier, il annote la copie de travail et vous demande d'examiner les différences afin de les sélectionner manuellement une à une. Et quand je dis « sélectionner manuellement », la réalité est la suivante : Git remplit chaque fichier conflictuel d'annotations sibyllines appelées *marqueurs de conflit* ; vous devez les examiner une à une en substituant au texte annoté la version que vous souhaitez obtenir après le merge.

Prenons un exemple. L'une de nos collègues, Meghan, directrice commerciale, a récemment été promue vice-présidente, et nous

travaillons sur une branche (`promotion-meghan`) pour ajouter son nouveau titre à la page de présentation de l'entreprise. Au même moment, un autre membre de notre équipe modifie la structure HTML de cette page de présentation dans sa propre branche (`noms-classes-page-presentation`), où figure l'ancien intitulé de poste de Meghan, et modifie tout le balisage qui l'entoure. Ces deux modifications semblent inoffensives mais ouvrent la voie à une catastrophe pour Git.

Partons du principe qu'on fusionne d'abord `noms-classes-page-presentation` dans `master` afin que la nouvelle structure HTML soit intégrée à la branche `master`. Ensuite, en utilisateur consciencieux de Git, on tente de mettre à jour la branche `promotion-meghan` en exécutant `git merge master` :

```
(promotion-meghan) $: git merge master
Auto-merging about.html
CONFLICT (content): Merge conflict in about.html
Automatic merge failed; fix conflicts and then
  commit the result.
```

Oh non, un conflit de fusion ! Et maintenant, quand on ouvre about.html, on trouve ce qui suit :

```
<div class="team-member">
  <h2>Meghan Trucmuche</h2>
<<<<<<< HEAD
  <p class="title">Vice présidente, Développement
  commercial</p>
=======
  <p class="job-title">Directrice commerciale</p>
>>>>>>> master
</div>
```

Les lignes remplies de chevrons sont les marqueurs de conflits et signalent les deux versions de la ligne conflictuelle, séparées ici par une ligne pleine de signes « égal ». En haut de cette section de conflit se trouve la version du fichier présente dans

notre branche actuelle, identifiée ici par le terme HEAD. HEAD est le nom que Git donne au « pointeur de tête de branche » ; il désigne « le commit se trouvant au sommet de cette branche », ce qui, dans notre cas, est synonyme de « commit actuellement extrait dans cette copie de travail ». Cette version contient le nouveau titre de Meghan mais l'ancien nom de classe CSS (`class="title"`).

C'est la version du bas, identifiée ici par son nom de branche (`master`), que l'on essaie de fusionner. Comme vous pouvez le voir, le nom de classe est à jour (`class="job-title"`), mais le titre en lui-même ne l'est pas. La différence est subtile, mais elle a suffisamment d'importance pour que Git ne déduise pas automatiquement quelle version est la bonne et ne sache pas comment les combiner. Git n'y connaît rien en intitulés de poste et ne sait pas écrire l'HTML. Il attend que vous, l'agent humain, interveniez pour produire la version fusionnée.

Pour résoudre ce conflit, il faut tout remplacer — les lignes encadrées par des chevrons et tout ce qu'elles contiennent — par la version du code que l'on veut obtenir à la suite du merge :

```
<div class="team-member">
  <h2>Meghan Trucmuche</h2>
  <p class="job-title">Vice présidente,
  Développement commercial</p>
</div>
```

Cet merge fait main intègre à la fois le nouveau nom de classe et le nouveau titre. Nous allons ensuite résoudre officiellement le conflit en ajoutant la nouvelle version à la zone d'indexation. C'est notre façon de dire à Git que nous approuvons formellement cette version.

```
(promotion-meghan *) $: git add about.html
```

Terminons le merge par un commit avec un message explicatif :

```
(promotion-meghan *) $: git commit -m "Fusionné branche »
  master dans promotion-meghan, conflits résolus"
```

Même si ce commit résout également un merge, cela reste un simple commit. Aussi stupéfiant que cela paraisse, vous n'avez aucunement l'obligation d'utiliser l'une des deux anciennes versions de cette ligne dans ce fichier. Vous pourriez décider de résoudre le merge en modifiant l'intitulé de poste de Meghan pour en faire la P.-D.G. (Ne vous étonnez pas si vous recevez un e-mail un peu surpris de la part de votre véritable P.-D.G, par contre.) La version du fichier qui est indexée quand vous réalisez le dernier commit est celle qui « remporte » le merge.

Comme vous pouvez le voir, le mécanisme de résolution des conflits est simple – peut-être même un peu trop simple. Si vous n'êtes pas rigoureux, vous pouvez archiver accidentellement un fichier rempli de marqueurs de conflits, parce que Git s'attend à ce que vous sachiez que c'est à vous de les supprimer. (S'il vous arrive effectivement d'archiver des marqueurs de conflit ou quoi que ce soit d'autre n'ayant pas sa place dans un fichier, pas de panique. Corrigez calmement les erreurs, indexez les modifications et committez-les. Git est souvent énigmatique, mais toujours cohérent.)

Une fois encore, si les conflits de fusion sont plus agaçants que véritablement effrayants, il vaut mieux les éviter autant que faire se peut en fusionnant régulièrement dans `master`. Cela peut sembler idiot dans la mesure où, dans notre dernier exemple, c'est précisément le merge dans `master` qui était problématique. Mais ce conflit de fusion précis était inévitable en raison de la nature des modifications apportées dans les deux branches. En fusionnant `master` maintenant, nous avons pu traiter ce conflit de fusion à un stade précoce, ne touchant qu'une ligne de code. Attendre plus longtemps, c'est prendre le risque que d'autres conflits s'y ajoutent et ne nous obligent à les résoudre péniblement un par un.

Si l'on tient à ce que la branche principale et les branches thématiques restent à jour les unes par rapport aux autres, ce n'est pas pour éviter les conflits mais plutôt pour les rendre plus faciles à gérer en limitant les différences entre deux branches. La plupart du temps, si vous réalisez vos merges de façon judicieuse, ce genre de choses ne se produit pas. Mais en dépit de tous vos efforts, des conflits de fusion se produisent tout de

même parfois. Le scénario de notre exemple — deux membres de la même équipe font, chacun de leur côté, des modifications légitimes mais conflictuelles — est toujours possible. Vous serez donc rassuré d'apprendre que Git a une solution et, comme toujours, cette solution est un commit.

4 REPOSITORIES DISTANTS

Jusqu'ici, toutes les modifications que nous avons effectuées résidaient dans un seul et même endroit : votre ordinateur. C'est plutôt pratique : alors que d'autres systèmes de gestion de versions maintiennent leur repository de versions committées uniquement sur un serveur et nécessitent donc de s'y connecter pour committer des changements, Git fonctionne hors ligne par défaut. Pour les projets en solo, vous pouvez ainsi bénéficier de toute la puissance de la gestion de versions sans avoir à accéder à un serveur ou à créer un compte quelque part.

Mais ce n'est pas pour travailler seul que les gens font appel à Git. Au contraire, c'est précisément pour collaborer.

Comme le dit l'adage (qui, comme le mot « bug », est souvent attribué à Grace Hopper), *les bateaux qui restent au port ne risquent rien, mais ce n'est pas pour cela qu'ils ont été construits*. Pour le moment, tous les commits que vous avez faits se trouvent en sécurité au port, dans votre machine. Il faut leur faire prendre la mer.

Un *repository distant*, par opposition à un repository local sur votre ordinateur, est une copie d'un projet Git qui se trouve ailleurs : sur une autre machine de votre réseau, sur l'ordinateur de quelqu'un d'autre, sur un service en ligne comme GitHub, bref, n'importe où ailleurs que dans le répertoire que vous contemplez en ce moment. En réalité, quand je parle de votre repository « local », je ne parle que de celui sur lequel vous êtes en train de travailler. Vous pourriez même demander à Git de pousser des modifications vers une autre copie locale stockée dans un autre répertoire de votre ordinateur : ce deuxième exemplaire serait considéré comme un repository distant.

Les repositories distants représentent l'une des abstractions les plus puissantes de Git. Contrairement aux branches, qui sont pour l'essentiel des copies virtuelles de votre projet, chaque repository distant correspond à une copie réelle et physique de votre repository, avec laquelle vous pouvez échanger des données. La plupart des opérations que vous aurez besoin de réaliser pour échanger des modifications avec un repository distant tiennent en deux verbes : *push* (pousser) et *pull* (tirer), qui produisent l'effet escompté.

VOTRE HUB GIT

La conception décentralisée de Git vous permet de pousser et de tirer des modifications entre deux ordinateurs, quels qu'ils soient : si vous le souhaitez, vous pouvez directement pousser les commits d'une branche résidant sur votre ordinateur vers une branche de l'ordinateur de votre collègue, et inversement. Et si cela semble formidable au premier abord, pour bien des équipes c'est un facteur de complexité sans grand avantage.

Celles-ci préfèrent généralement partager leur code via Git, à travers ce que j'appelle le *modèle de hub*. C'est une centralisation pour la bonne cause : vous et votre équipe conservez une copie commune du projet sur un serveur distant (le *hub*), accessible à tous les membres de l'équipe. Chaque collègue qui se joint au projet copie (ou *clone*) le repository du projet sur son ordinateur, effectue des modifications et des commits, puis utilise les

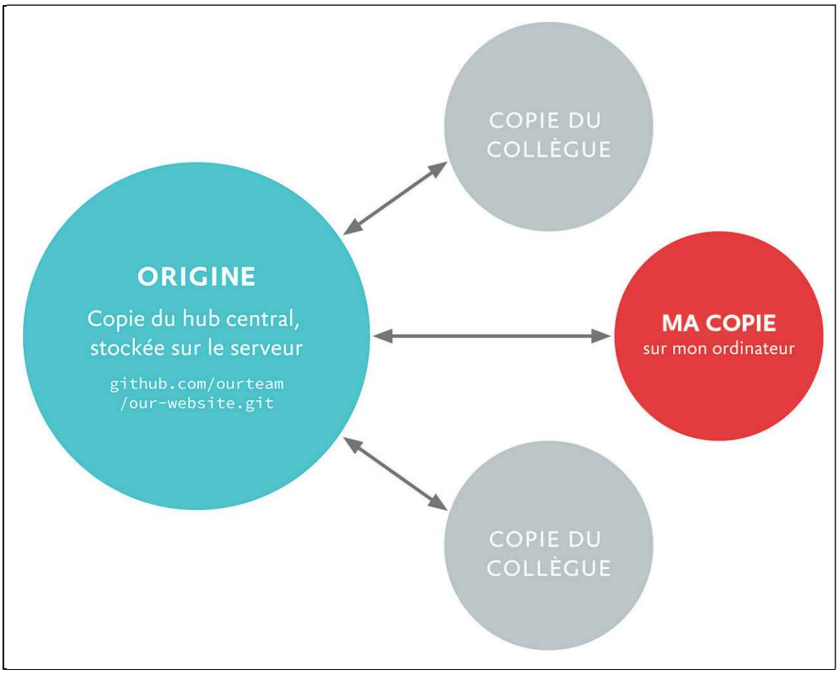


FIG 4.1 : Selon le modèle de hub, les membres de votre équipe synchronisent leur copie locale avec une copie centrale commune plutôt que les uns avec les autres.

commandes `git push` et `git pull` pour synchroniser le repository avec celui du serveur.

Il n'y a rien de spécial à savoir sur les repositories distants : ce ne sont que des instances du projet, stockées dans un endroit accessible permettant d'y pousser ou d'en tirer des commits. En théorie, Git ne considère aucun repository particulier comme le repository « canonique » d'un projet, même si, dans la pratique, les équipes disposent d'une copie distante commune (souvent hébergée sur GitHub) considérée comme principale, et que Git appelle, par convention, l'*origine*. Comme dans le cas des branches `master`, c'est à vous de décider ce que signifie « principal », et le repository d'origine sera ce que vous en ferez.

Le modèle de hub, en revanche, considère le repository d'origine comme canonique, et du point de vue des membres de votre équipe, vos changements ne sont pas véritablement archivés tant qu'ils n'ont pas été committés et poussés vers le serveur pour y être accessibles (**FIG 4.1**).

Le hub joue également le rôle de sauvegarde fiable du code au cas où la copie d'un contributeur serait endommagée ou perdue, ou si un utilisateur souhaite récupérer une copie de son travail sur une nouvelle machine. Au lieu de copier les fichiers d'un ordinateur à un autre, il est souvent plus simple de cloner le projet Git sur la nouvelle machine à partir du hub.

Cela présuppose naturellement que la copie du hub ne sera jamais perdue ni corrompue, mais la conception décentralisée de Git nous rend encore service ici. Bien que le hub soit la sauvegarde canonique de notre repository, chaque copie contient l'historique complet de votre projet. Pour que du travail soit vraiment perdu, il faudrait qu'il disparaisse de l'ordinateur de tous les contributeurs, ce qui est tout de même peu probable. Dans le cas (improbable) où le hub serait compromis, n'importe quel repository local peut être utilisé pour ressusciter un nouveau repository distant.

QU'ABRITE LE SERVEUR ?

Les repositories côté serveur sont ce que l'on appelle des repositories « vierges », comprenant uniquement les données réelles du repository (versions précédentes, branches), sans aucune copie de travail (et donc sans aucune zone d'indexation). On désigne généralement un repository Git vierge en ajoutant `.git` à son nom, comme dans notre hypothétique `notre-site.git`. L'architecture interne d'un repository Git vierge est quasiment identique à ce que vous trouveriez dans le répertoire masqué `.git` de votre répertoire de travail local : des sous-répertoires pour les objets, les pointeurs de branche et tout ce dont Git à besoin.

Notre repository Git côté serveur contient tous les commits qui ont été poussés, ainsi que son propre groupe de branches.

C'est ce groupe de branches distantes supplémentaire qui peut perturber les débutants, parce qu'on présuppose naturellement qu'il existe une relation directe entre une branche présente sur notre machine et son homologue sur le serveur. En effet, si c'est généralement le cas, cette relation n'est pas indispensable pour Git, qui ne l'applique donc pas strictement. Conforme à ses habitudes, Git vous pousse à gérer cette parenté souple entre branches locales et branches côté serveur d'une seule manière : en étant plus spécifique dans la rédaction de vos commandes.

Par exemple, pour pousser des modifications à partir d'une branche locale vers sa jumelle sur le serveur, il ne suffit généralement pas de lancer `git push` à la cantonade. Git préférera que vous disiez `git push <nomrepository><nombranche>`, même s'il nous paraît évident que le nom du repository distant et le nom de la branche peuvent être déduits du contexte.

OÙ SE TROUVE LE REPOSITORY DISTANT ?

L'emplacement du repository par rapport à votre repository local est ce qui en fait un repository distant. Autrement dit, un repository distant se trouve... ailleurs. Mais où, exactement ?

Pour vous, la plupart du temps, le repository distant se trouve sur GitHub. GitHub est le service d'hébergement de repositories Git le plus utilisé, et sa popularité est telle qu'il semble presque ridicule d'envisager des alternatives dans ce chapitre. Même si votre équipe n'héberge jamais ses projets sur GitHub, vous pouvez être sûr que vous interagirez avec un repository GitHub à un moment ou un autre de votre travail. Et en effet, le service de GitHub est à la fois très facile à utiliser et peu coûteux, voire gratuit si votre projet est open source ou au moins accessible au public. Des abonnements à bas prix sont proposés si le partage du code doit rester privé.

Il existe pourtant bien d'autres options, qu'il s'agisse d'autres services d'hébergement ou d'outils pour héberger vous-même des repositories Git. Si vous n'avez pas l'envie ou les capacités de gérer vos propres serveurs, un service d'hébergement comme GitHub représente la meilleure option : il se charge

des tâches pénibles pendant que vous vous consacrez à votre projet. Mais selon la nature de votre travail ou du client pour qui vous le faites, vous devrez peut-être conserver votre code source en interne.

Heureusement, bien que chaque service ait ses propres outils et interfaces pour créer des repositories distants, tous fonctionnent de la même façon une fois mis en place.

AJOUTER UN PREMIER REPOSITORY DISTANT

Vous pouvez passer l'URL d'un repository distant comme paramètre à chacune des commandes Git que je viens de mentionner, ce qui est parfait si vous n'avez besoin de pousser ou de tirer des commits qu'une fois et une seule. Mais dans la plupart des cas, vous travaillerez de façon répétée avec les mêmes repositories distants au cours de la vie d'un projet. Au lieu de désigner les repositories distants par leur URL, vous pouvez leur attribuer des noms qui permettront de les référencer.

À ce stade, la copie locale de notre projet se trouve naturellement sur notre ordinateur. Mais admettons que nous ayons également un repository Git distant (`notre-site.git`) stocké sur notre propre serveur, `gitforhumans.info`, et que nous voulions en faire l'origine de notre projet. Pour ce faire, nous allons utiliser la commande d'ajout de `git remote`. Rouvrez le terminal et saisissez la commande suivante :

```
(master) $: git remote add origin »
  https://gitforhumans.info/notre-site.git
```

J'en profite pour signaler que la commande `git remote` est d'un type nouveau pour nous : elle comprend des *sous-commandes*. Toutes les commandes que nous avons utilisées jusqu'ici tenaient en un seul mot (`git commit` par exemple), tandis que toutes les commandes liées à la configuration des repositories distants ont des *espaces de nommage* : ce sont des commandes en deux mots commençant par `remote` : `remote add`, `remote rm`, et ainsi de suite.

Si vous saisissez simplement `git remote`, sans sous-commande, vous demandez à Git de présenter une liste de tous les repositories distants ajoutés au projet, de la même manière que `git branch` dresse la liste de toutes les branches. Comme vous pouvez le voir, il n'y en a qu'un : `origin` :

```
$: git remote
origin
```

Notez que si vous avez commencé par cloner le projet sur votre ordinateur à partir d'un serveur distant à l'aide de la commande `git clone`, cette opération aura déjà été réalisée pour vous. Les repositories clonés depuis un serveur distant sont toujours préconfigurés avec ce serveur comme origine.

Tout comme la branche principale de votre projet possède un nom conventionnel (`master`), votre repository distant principal s'appelle par défaut `origin`. (Remarquez comme cette convention – simple mais efficace – renforce le rôle de « hub » du repository distant : sémantiquement, le repository distant est l'origine du code de votre projet, et tous vos repositories locaux sont des satellites en orbite autour du hub.)

Bien que le nom soit `origin` par convention, vous pouvez nommer vos repositories distants comme vous le souhaitez. Mais à moins d'avoir une excellente raison de le faire, il vaut mieux s'en tenir à la convention et conserver `origin` pour le repository distant principal de votre projet.

Comprendre les URL distantes

Git prend en charge trois protocoles réseau différents pour déplacer les commits et autres données sur le réseau : le protocole Git, SSH et HTTP (**FIG 4.2**). Au quotidien, les trois se comportent de la même façon. Les protocoles de Git se distinguent seulement par la manière dont on s'authentifie auprès du serveur et par le fait qu'ils prennent en charge la lecture et l'écriture, ou bien la lecture seule.

Protocole Git	git://gitforhumans.info/hello.git
HTTPS	https://gitforhumans.info/hello.git
SSH	git@gitforhumans.info:hello.git

FIG 4.2 : Chacun de ces exemples d'URL désigne le même repository — `hello.git`, sur le serveur nommé `gitforhumans.info` — à l'aide des trois options de protocole de Git. La plupart des services d'hébergement Git proposent au moins HTTPS et SSH.

SSH (Secure Shell)

Le protocole SSH de Git est parfaitement identique à celui que bon nombre d'entre nous utilisent chaque jour pour se connecter à des serveurs distants. De fait, tous les serveurs SSH auxquels vous avez accès peuvent sans doute être utilisés comme hôtes de repositories Git distants. Les repositories distants SSH supportent la lecture et l'écriture, et permettent d'utiliser toutes les méthodes d'authentification prises en charge par SSH.

Si Git ne dispose pas de protocole par défaut à proprement parler, SSH est si largement utilisé pour partager des repositories Git en ligne de façon sécurisée qu'il est devenu une sorte de norme — statut encore renforcé par le fait que Git n'exige aucun préfixe de protocole pour les URL SSH. Autrement dit, si vous omettez la portion protocole d'une URL, Git part du principe que vous utilisez SSH. Cela fait bien longtemps que le format d'URL par défaut de GitHub pour l'accès aux repositories privés (du type git@github.com:nomutilisateur/nomrepository.git) utilise SSH.

Un inconvénient propre au protocole SSH en limite l'utilité dans l'écosystème open source d'aujourd'hui : il ne fonctionne que pour les repositories privés, parce que SSH n'offre aucun moyen d'autoriser quelqu'un à accéder à des ressources sans authentification. (C'est un shell sécurisé, ne l'oublions pas.) Vous devrez donc faire appel à un autre protocole si vous souhaitez ouvrir un accès public à tout ou partie de vos repositories.

Heureusement, la plupart des hôtes Git prennent aujourd'hui en charge plusieurs protocoles, et vous pouvez utiliser HTTP

pour permettre au public de télécharger la dernière version de la branche `master` de votre super framework JavaScript, tout en utilisant SSH au sein de l'équipe pour pousser des commits dans ladite branche.

Les serveurs distants Git SSH, comme de nombreux serveurs SSH, permettent de se connecter avec un nom d'utilisateur et un mot de passe. Toutefois, on utilise plus couramment une authentification par clé publique : vous générez une paire de clés unique et sécurisée et vous chargez la clé publique dans votre compte sur le serveur Git (GitHub, par exemple), en conservant la clé privée sur votre ordinateur, en toute sécurité. Lorsque vous accédez à un repository distant à partir de ce serveur, Git (ou plutôt SSH pour le compte de Git) envoie votre clé privée de façon sécurisée pour qu'elle joue le rôle de badge d'identification. (La documentation d'aide de GitHub résume très bien le processus de création d'une paire de clés : http://bkaprt.com/gfh/04-01/).

Les débutants seront peut-être intimidés par la méthode des paires de clés, mais cette petite complexité supplémentaire offre à la fois sécurité et simplicité d'utilisation (une fois la configuration effectuée). Comme chaque utilisateur génère une paire de clés unique sur sa propre machine, les administrateurs de serveurs peuvent gérer avec précision à quels projets les membres ont accès. Les services d'hébergement comme GitHub et Bitbucket, en particulier, offrent d'excellents outils pour la gestion des utilisateurs et des clés.

HTTPS

Il s'agit naturellement du protocole HTTP que l'on utilise pour fournir du contenu sur le Web. De nos jours, de nombreux hôtes Git (en particulier GitHub) ont fait d'HTTPS le protocole par défaut de leurs URL, parce qu'elles sont plus faciles à utiliser (on peut s'authentifier sur un repository distant HTTPS avec un nom d'utilisateur et un mot de passe plutôt qu'une clé SSH), mais aussi parce qu'elles sont plus versatiles. Tandis que SSH doit être privé et doit autoriser l'accès en lecture et en écriture à vos repositories, HTTPS se montre plus souple. Vous pouvez autoriser n'importe quel internaute à récupérer

des modifications sur votre repository tout en limitant le push aux membres de votre équipe.

Protocole Git

Seul le protocole Git est unique à Git. Il est rarement utilisé de nos jours, essentiellement parce qu'il ne permet que la lecture. Il fut un temps où il était pratique pour proposer des repositories publics (sur GitHub, au hasard), et ce protocole complétait habilement SSH pour les projets nécessitant des accès publics et privés. Mais HTTPS constitue un meilleur choix aujourd'hui.

Lequel devez-vous utiliser ?

Dans le cas des projets purement privés (si vous travaillez sur un logiciel commercial plutôt qu'un projet open source, notamment), SSH est une excellente option et c'est la plus largement prise en charge. Cela dit, si vous souhaitez une expérience simple et homogène, je recommande d'utiliser HTTPS dans tous les scénarios qui le permettent. Si les clés SSH ne sont pas difficiles à gérer, elles restent tout de même plus complexes qu'un nom d'utilisateur assorti d'un mot de passe, et la possibilité de rendre les URL HTTPS publiques facilite leur partage.

Pour tout savoir sur les protocoles de Git, leurs avantages et leurs inconvénients, plongez-vous dans l'excellent ouvrage de référence de Scott Chacon intitulé *Pro Git* (en anglais), disponible gratuitement en ligne (http://bkaprt.com/gfh/04-02).

TRAVAILLER AVEC DES BRANCHES DISTANTES

Cela peut sembler évident, mais ce qui distingue principalement les branches ordinaires des branches distantes est que ces dernières se trouvent sur une autre machine. Quand vous utilisez des branches, vous vous préoccupez essentiellement de gérer différentes versions de votre travail se trouvant sur votre propre ordinateur, dans ce que j'appelle (et Git aussi, d'ailleurs) votre *copie locale*. Avec les repositories distants, vous gérez également différentes versions. En effet, vos interactions avec les repositories distants se feront pratiquement toujours dans le

contexte d'une branche. Une fois que vous avez committé une modification dans une branche de votre repository local, vous pouvez utiliser `git push` pour soumettre votre copie de la branche – et tous les commits que vous y avez ajoutés – au serveur. À chaque fois que vous avez besoin de rafraîchir votre copie d'une branche pour récupérer les dernières modifications de vos collègues, vous pouvez utiliser `git pull`.

Examinons quelques exemples d'utilisation concrets de ces commandes, en commençant par `push`.

Pousser des modifications

En travaillant sur le design de la nouvelle page d'accueil, nous avons découvert un bug dans un bloc de JavaScript. Une collègue propose de le corriger, mais il faut d'abord intégrer nos modifications à sa copie du projet. Pour cela, il faut pousser la branche `nouvelle-page-accueil` vers le serveur, où notre collègue pourra la récupérer.

La commande à utiliser ici est `git push <repository distant><branche>`. Une fois encore, Git s'attend à ce que l'on soit explicite et qu'on indique clairement vers quel repository distant (`origin`) on pousse quelle branche (`nouvelle-page-accueil`). C'est la première fois que l'on a accès à ce repository en particulier. Il est protégé par un mot de passe, et Git nous invite à saisir nos identifiants lors de la première tentative d'échange :

```
$: git push origin nouvelle-page-accueil
Username for 'https://gitforhumans.info': ddemaree
Password for 'https://ddemaree@gitforhumans.info':
Counting objects: 8, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 743 bytes | 0 bytes/s,
  done.
Total 8 (delta 1), reused 0 (delta 0)
To https://gitforhumans.info/notre-site.git
* [new branch]    nouvelle-page-accueil -> nouvelle-page-
accueil
```

Git fait plusieurs choses à notre place quand nous poussons des modifications, comme nous le détaille cette longue réponse emberlificotée.

Tout d'abord, dans les premières lignes suivant l'invitation à saisir le mot de passe, Git emballe et envoie les commits sur le réseau :

```
Counting objects: 8, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 743 bytes | 0 bytes/s,
  done.
Total 8 (delta 1), reused 0 (delta 0)
```

Il n'y a rien que nous ayons besoin de savoir dans ce bloc de texte : il indique simplement que Git est effectivement parvenu à emballer et envoyer nos données sur le serveur.

La ligne suivante a beaucoup plus d'intérêt :

```
* [new branch]    nouvelle-page-accueil -> nouvelle-page-
accueil
```

Ici, Git nous explique que le serveur distant a reçu notre branche appelée `nouvelle-page-accueil`, et qu'il s'en est servi pour créer une nouvelle branche sur le serveur, également appelée `nouvelle-page-accueil`. Git n'a pas besoin que les branches distantes portent le même nom que leurs homologues locales. Mais pour la santé mentale de tous, on préfèrera généralement que ce soit le cas.

Récupérer des modifications

C'est la fin de l'après-midi, et en revenant de la pause café, vous découvrez que votre collègue a soumis ses modifications et ainsi corrigé les bugs du JavaScript. Il est temps d'intégrer les modifications qu'elle a committées dans la branche `nouvelle-page-accueil` à notre copie de cette branche, en mettant cette dernière à jour grâce à la commande `git pull <repository distant><branche>`.

Fidèle à lui-même, Git nous demande d'être terriblement explicite et de préciser le nom du repository et celui de la branche :

```
$: git pull origin nouvelle-page-accueil
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
From https://gitforhumans.info/notre-site.git
 * branch    nouvelle-page-accueil   -> FETCH_HEAD
Updating fed3ac5..4f82376
Fast-forward
 carousel.js | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Comme dans le cas de `git pull`, la réponse comprend plusieurs lignes (en commençant par `remote:`) qui expliquent comment les données sont transférées entre les deux repositories, ce qui n'a pas grand intérêt. Passons directement au détail vraiment intéressant :

```
From https://gitforhumans.info/notre-site.git
 * branch    nouvelle-page-accueil   -> FETCH_HEAD
```

On pourrait s'attendre à ce que Git explique ici qu'il a récupéré les modifications de la copie de `nouvelle-page-accueil` présente sur le serveur pour les envoyer vers notre copie locale de la même branche. Pourtant, la petite flèche ASCII pointe vers autre chose : `FETCH_HEAD`. Pour comprendre cela, accordez-moi quelques instants afin que je vous explique les rouages secrets de `push` et `pull`.

Quand vous poussez ou récupérez une branche, deux choses doivent se passer, et les deux se retrouvent dans la réponse de `git pull`.

Tout d'abord, Git doit transférer un paquet d'objets (vos commits et les fichiers dont ils suivent les modifications) depuis ou vers le serveur. Toutes les lignes `remote` concernent cette partie du processus, et si je vous affirme aussi sereinement que

vous pouvez les ignorer, c'est parce qu'il est extrêmement rare que des problèmes surviennent à ce niveau. L'envoi de données entre deux machines présente un risque majeur : celui que les données d'une machine écrasent accidentellement les données de l'autre sans que cela ne se voie, et que des données soient perdues dans la manœuvre. L'une des grandes merveilles de l'architecture de Git est qu'il est quasiment impossible que des commits entrent en conflit, si bien que l'envoi et la réception d'objets sont extrêmement sûrs. Le risque principal est qu'une copie se retrouve avec trop de données, mais il est pratiquement impossible de perdre quoi que ce soit.

Une fois que tous les nouveaux commits sont en sécurité sur votre ordinateur, passons à la seconde partie, le merge :

```
Updating fed3ac5..4f82376
Fast-forward
 carousel.js | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Comme il n'y a eu aucun autre commit de notre côté depuis que nous avons confié la branche à notre collègue, Git peut réaliser le merge avec un simple fast-forward.

Git réalise cette jolie danse « copie-merge » à trois temps pour garantir la sécurité du travail committé dans notre copie de `nouvelle-page-accueil`. Si l'on peut copier des commits d'un ordinateur à l'autre en toute sécurité, comme nous l'avons vu, fusionner des branches crée parfois des conflits que Git ne peut résoudre seul. De plus, même si l'on utilise `git pull` pour fusionner une branche côté serveur dans l'une de nos branches locales, Git réalise tout le travail de merge du côté local, et doit donc copier la branche `nouvelle-page-accueil` du serveur vers un emplacement de notre ordinateur avant de tenter le merge. Cet emplacement, c'est `FETCH_HEAD`. C'est une branche temporaire créée par Git comme tampon afin de fusionner les modifications fraîchement récupérées.

Il est essentiel de se souvenir que le merge fait implicitement partie du `pull` (et aussi, puisqu'on en parle, du `push`). Ou, pour prendre les choses dans l'autre sens, pensez que `push` et `pull`

sont les variantes distantes du merge. Les deux commandes font exactement la même chose : elles déplacent une branche d'un ordinateur à un autre, puis la fusionnent avec une autre branche.

Une fois qu'on a récupéré les modifications depuis le serveur, notre copie de `nouvelle-page-accueil` est à jour et nous pouvons nous remettre au travail.

Résoudre les conflits de fusion... à distance

Comme nous l'avons vu, la récupération de modifications distantes se termine toujours par un merge. Et comme nous le savons aussi, les merges se terminent parfois par un conflit. Il faut savoir que les `push` et les `pull` sont plus sujets aux conflits que les autres types de merge, parce qu'ils impliquent généralement plus de gens et de changements, sur des périodes plus longues. Et le risque de conflit est sans doute à son comble dans la branche qui, dans la plupart des projets, change le plus souvent : la copie commune de `master` qui se trouve dans `origin`.

Dans le chapitre précédent, j'ai indiqué qu'il était préférable que chaque branche de travail destinée à être fusionnée dans `master` soit tenue à jour des dernières modifications apportées à `master`. Plus simplement, pendant que vous travaillez, vous devez penser à récupérer régulièrement la branche `master` du serveur afin de réduire le risque de conflits de fusion, et pour que les conflits, s'ils se manifestent quand même, restent minimes. Si vous ne l'avez pas déjà deviné, la commande pour ce faire est `git pull origin master`, qui fonctionne toujours de la même manière, quelle que soit votre branche de travail. Ici, nous allons essayer de récupérer les modifications d'`origin/master` pour les intégrer à notre copie de `nouvelle-page-accueil` :

```
(nouvelle-page-accueil) $: git pull origin master
From https://gitforhumans.info/our-homepage.git
 * branch    master   -> FETCH_HEAD
Auto-merging about.html
CONFLICT (content): Merge conflict in about.html
```

```
Automatic merge failed; fix conflicts and then
  commit the result.
```

Oups ! Une fois de plus, Git s'est pris les pieds dans une différence d'une ligne sur la page de présentation. Comme quand nous avons modifié l'intitulé de poste de Meghan, un commit de notre branche a modifié du texte dans l'en-tête (« À propos du site » a été remplacé par « Notre équipe ») et un commit fait dans `master` a modifié le balisage environnant. Il suffit d'ouvrir about.html pour voir la modification conflictuelle, encadrée par les marqueurs de conflit :

```
<<<<<<< HEAD
<h1 class="big-heading">À propos du site</h1>
=======
<h1>Notre équipe</h1>
>>>>>>> 4f2d3c939deaf8f2824d2be04cb59b3f8342aedb
```

La bonne nouvelle, c'est que le processus de résolution de conflit est exactement le même pour un `git merge` local que pour une tentative de `git pull`. Comme la dernière fois, il faut remplacer tout cela par la version du texte que nous souhaitons conserver dans cette branche :

```
<h1 class="big-heading">Notre équipe</h1>
```

Ensuite, indexons et committons la modification pour résoudre le conflit dans notre branche locale.

```
(nouvelle-page-accueil *) $: git add -A

(nouvelle-page-accueil *) $: git commit -m "Fusionné
origin/ »
  master dans nouvelle-page-accueil, conflits résolus"
```

Une fois ce commit effectué, notre branche est parfaitement à jour par rapport au `master` du serveur. Vous pouvez maintenant pousser ces modifications, y compris le merge commit que

nous venons de créer, vers la copie de la branche qui se trouve sur le serveur, ou bien continuer de travailler.

Pendant que nous y sommes, permettez-moi d'attirer votre attention sur une nouvelle notation que j'ai utilisée dans le message de commit. Les noms des branches distantes prennent souvent la forme `nomderepository/nomdebranche`, comme dans `origin/master` (qui désigne la copie de `master` résidant sur le repository distant `origin`) ou `testserver/bugfix` (la branche `bugfix` sur le repository distant `testserver`). Bien que les branches distantes correspondent pratiquement toujours à (ou suivent) une branche présente sur votre propre machine, il s'agit techniquement de branches différentes, et cette notation avec slash est un excellent moyen de faire la distinction sans avoir à dire, comme je viens de le faire, « la copie de `master` qui se trouve sur le repository distant `origin` ».

Faire face au rejet (de push)

Pendant que nous poursuivions notre travail sur le design de la nouvelle page d'accueil, la collègue qui avait corrigé le JavaScript a trouvé et corrigé un autre bug dans notre code. Elle a committé et poussé son correctif sur la branche distante, mais a dû partir en réunion avant de pouvoir nous informer qu'elle avait modifié notre branche.

Pendant ce temps, nous avons aussi essayé de pousser des modifications vers la branche, et voici ce qui s'est passé :

```
(nouvelle-page-accueil) $: git push origin nouvelle-page-
accueil
To https://gitforhumans.info/page-accueil.git
   ! [rejected]    nouvelle-page-accueil -> nouvelle-
page-accueil
    (non-fast-forward)
error: failed to push some refs to ' https://
  gitforhumans.info/page-accueil.git'
hint: Updates were rejected because the tip of your
  current branch is behind
hint: its remote counterpart. Integrate the remote
  changes (e.g.
```

```
hint: 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git
  push --help' for details.
```

Aïe. Pourquoi est-ce que Git rejette les changements que vous essayez de pousser ? En règle générale, les repositories Git côté serveur ne disposent pas de copies de travail, de zones d'indexation et encore moins d'utilisateurs humains capables de résoudre les conflits de fusion. Et effectivement, l'absence de copie de travail signifie que les repositories distants ne peuvent tout simplement pas fusionner des branches si cela nécessite plus qu'un simple fast-forward. C'est ce que nous indique la réponse à `git push` :

```
! [rejected]    nouvelle-page-accueil -> nouvelle-page-
accueil
  (non-fast-forward)
```

Heureusement, on peut facilement corriger cette situation en récupérant les modifications depuis le serveur et en essayant de les pousser à nouveau. Le fast-forward consiste à déplacer le pointeur `HEAD` d'une branche du commit où il se trouve actuellement vers l'un de ses descendants directs. Quand vous récupérez des modifications, le résultat de l'opération est un merge commit, qui se trouve être un descendant direct du commit de tête actuelle, et peut donc être traité par un fast-forward. Magie !

Pour faire court : si vous voulez éviter ce genre de rejet ou toute autre incartade de Git, faites toujours un `pull` avant un `push` pour être sûr que votre copie locale est à jour. Récupérer des modifications est rarement dangereux et presque toujours avantageux.

SUIVRE LES BRANCHES

Par défaut, rien ne lie les copies locales et distantes d'une branche donnée. Bien qu'elles portent le même nom, et même si nous

savons qu'elles représentent logiquement le même travail, Git ne sait pas encore que la branche `nouvelle-page-accueil` locale et la branche `nouvelle-page-accueil` du serveur sont liées, et c'est pourquoi nous devons toujours indiquer les branches distantes auxquelles s'appliquent les commandes `git pull` et `git push`. Certes, comme partout ailleurs dans Git, cette obligation d'être explicite peut sembler agaçante, mais c'est aussi un puissant atout. Vous pouvez potentiellement récupérer dans `nouvelle-page-accueil` les modifications provenant de n'importe quelle branche située sur n'importe quel repository distant. Vous pouvez exécuter la commande `git pull maniks-computer nouvelle-page-accueil-avec-sass` — `maniks-computer` étant la machine de votre collègue Manik et `nouvelle-page-accueil-avec-sass` une branche pour la conversion des styles CSS en Sass : ça marchera.

Cela dit, il est intéressant d'indiquer à Git que les versions locales et distantes d'une branche sont liées en précisant qu'une branche locale suit son homologue distante. Par exemple, quand une branche est configurée pour effectuer un suivi, il suffit de lancer `git push` ou `git pull` sans aucun autre argument pour pousser et récupérer des modifications. Git comprendra ce que vous voulez dire et agira en conséquence.

La manière la plus simple d'établir une relation de suivi consiste à inclure l'option `--set-upstream` (ou `-u`) quand vous invoquez `git push`.

```
(nouvelle-page-accueil) $: git push -u origin nouvelle-
page-accueil
Branch nouvelle-page-accueil set up to track remote
branch
  nouvelle-page-accueil from origin.
Everything up-to-date
```

Vous n'aurez à le faire qu'une seule fois par branche locale, et si vous oubliez de le faire la première fois que vous poussez, aucun souci. Vous pouvez le faire à tout moment, même sans avoir de nouvelle modification à pousser (ce que nous indique Git ici en annonçant « Everything up-to-date »).

MAÎTRISER FETCH

Une autre commande Git liée aux repositories distants mérite qu'on s'y attarde. À première vue, `git fetch` semble incroyablement similaire à `git pull`. Mais si `git pull` va récupérer les modifications d'une seule branche, `git fetch` récupère tout le contenu d'un repository distant en une seule opération.

Si vous exécutez la commande `git fetch origin`, vous observerez un résultat très familier :

```
$: git fetch origin
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
From https://gitforhumans.info/our-homepage.git
   9eb7cf6..fed3ac5  master     -> origin/master
```

On retrouve d'abord le gloubi-boulga de copie d'objets qu'on a déjà vu plusieurs fois. Mais si l'on regarde ce qui suit, on voit qu'il s'est passé quelque chose de plus qu'une simple copie de données provenant du serveur, quelque chose de très différent des merges et des fast-forwards auxquels on est habitué. Plus spécifiquement, Git a enregistré une copie de la branche `master` du serveur dans une branche spéciale, en lecture seule, de notre copie locale : `origin/master`.

Une partie du travail de `git fetch` consiste à vous permettre de travailler hors ligne. Quand je dis que `git fetch` travaille sur des repositories complets, je veux dire ceci : par défaut, Git récupère un snapshot de chaque branche d'un repository distant pour que vous puissiez effectuer une comparaison, un merge ou autre opération sur ces branches sans avoir besoin d'être constamment en ligne. Quand Git a été développé en 2005, avant que les smartphones et le Wi-Fi dans l'avion ne deviennent monnaie courante, il fallait récupérer une copie complète des documents d'un projet pour pouvoir travailler dans un café ou pendant un vol. Mais on n'avait pas forcément envie de prendre le temps de fusionner chaque branche du serveur avec

son homologue locale. (Ne serait-ce que parce que les changements de vos branches locales ne sont pas forcément prêts à être fusionnés, ou parce qu'il pourrait y avoir des conflits.)

La solution choisie par Git consiste à garder une trace de l'état de chaque branche de vos repositories distants à l'aide d'un système de branches en lecture seule avec espace de nommage, basé dans votre copie locale du repository. J'ai un peu menti en disant tout à l'heure que `origin/*` n'était qu'une notation permettant d'identifier les branches distantes. `origin/master` est aussi une branche réelle, enregistrée dans votre copie locale du repository. Après un `fetch`, vous obtenez des copies de chaque branche du repository distant, y compris de celles qui n'ont aucun équivalent local dans votre copie (parce qu'elles ont été créées par d'autres membres du projet par exemple).

Pour des raisons de sécurité et de rapidité, Git n'essaie d'utiliser le réseau que pour déplacer les commits, et fait tout le travail réel sur votre ordinateur. Ainsi, au lieu d'essayer de comparer les données de votre ordinateur avec celles du serveur, Git fait une copie du contenu du serveur et s'en sert de référence pour vos comparaisons et vos merges. La branche `origin/master` représente la branche `master` du repository distant `origin` et pointe vers le commit qui était en tête de la branche la dernière fois que vous l'avez récupéré sur le serveur.

Ces copies hors ligne spéciales de vos branches distantes peuvent compliquer les choses au lieu de les simplifier. Par exemple, on peut avoir trois branches différentes appelées `master` : la branche `master` locale, la branche `master` du repository distant et la branche `origin/master` locale qui est censée — mais ce n'est pas garanti — être en synchronisation avec la branche `master` du repository distant. Heureusement, les branches comme `origin/master` sont en lecture seule et ne servent qu'à représenter une copie de ce qui se trouve sur le serveur. Quand on exécute `git fetch`, on peut généralement partir du principe que chaque branche hors ligne est une représentation exacte de sa jumelle sur le serveur.

EXTRAIRE UNE BRANCHE EXISTANTE

Dans la plupart des équipes avec lesquelles j'ai travaillé, les différentes branches appartenaient pour l'essentiel à leurs créateurs respectifs qui se chargeaient également de les fusionner dans `master` une fois le travail terminé. Mais bien des projets dépassent les compétences d'une personne et s'étalent sur plusieurs jours, et vous ne serez peut-être pas le premier à être invité à travailler sur une branche en particulier. Vous serez peut-être même amené à rejoindre une branche sur laquelle un collègue travaille déjà et à laquelle beaucoup de gens contribuent. Comment ajouter un commit à la branche de quelqu'un d'autre ?

Dans un premier temps, extrayons cette branche. Pour cela, nous allons utiliser `git fetch` pour récupérer des copies de toutes les branches se trouvant actuellement sur le serveur :

```
[master] $: git fetch origin
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
From https://gitforhumans.info/notre-site.git
  9eb7cf6..fed3ac5   master     -> origin/master
  9eb7cf6..fed3ac5  nouvelle-page-accueil -> origin/
    nouvelle-page-accueil
```

Une fois que l'on a récupéré les dernières modifications d'`origin`, toutes les branches du serveur sont maintenant à notre disposition sur notre ordinateur, même hors ligne. Nous aurons besoin d'être connectés pour pousser nos propres changements sur le serveur, mais en attendant, tout ou presque est faisable localement.

On peut, par exemple, demander à Git une liste de toutes les branches qui se trouvaient sur le serveur la dernière fois que nous avons lancé `git fetch`. Même si, par défaut, la commande `git branch` ne renvoie que les branches qui se trouvent dans votre copie locale, vous pouvez ajouter le marqueur `--remote`

(`-r`) pour lui demander de lister également toutes les branches connues de Git et se trouvant sur vos repositories distants :

```
$: git branch --remote
origin/plus-gros-le-logo
origin/master
origin/nouvelle-page-accueil
```

Toutes ces branches peuvent être extraites et utilisées, ou fusionnées dans l'une de vos branches. `git pull origin master` est en fait un raccourci pour un `git fetch` suivi d'un `git merge origin/master`.

Nous allons ensuite extraire la branche sur laquelle nous voulons travailler, astucieusement baptisée `plus-gros-le-logo`. Inutile d'inclure le préfixe `origin/` ; si vous extrayez une branche distante pour la première fois, Git va d'abord regarder si vous possédez une branche locale sous ce nom puis, dans le cas contraire, va automatiquement en créer une pour suivre la branche distante.

```
[master] $: git checkout plus-gros-le-logo
Branch plus-gros-le-logo set up to track remote
  branch plus-gros-le-logo from origin.
Switched to a new branch 'plus-gros-le-logo'

[plus-gros-le-logo] $:
```

Nous avons évoqué l'origine de la gestion de versions et sa mise en pratique dans nos projets à l'aide de Git. Nous savons comment faire des commits, créer et fusionner des branches, et synchroniser nos modifications avec d'autres ordinateurs (et donc, par extension, d'autres personnes). Au fil du temps, nous avons bâti un historique de notre projet.

Nous allons maintenant explorer ce que nous pouvons faire de tous ces commits. En avant !

5 HISTORIQUE

Git est un excellent outil pour synchroniser des modifications entre plusieurs machines, et c'est essentiellement dans ce but qu'on l'utilise : pour que tous les membres d'une équipe sachent où en sont les autres en ce moment. La plupart du temps, on se soucie uniquement de la version en cours — ou des quelques versions en cours — alors que Git s'applique avec le plus grand soin à conserver et suivre chaque version d'un projet, et que ces milliers d'autres commits sont toujours là, ne demandant qu'à être explorés.

Chaque commit ajouté à votre repository contribue à l'historique du projet, et c'est pourquoi il faut s'efforcer d'appliquer les commits les plus judicieux et sensés possible. Dans ce dernier chapitre, nous allons aborder les outils utilisés par Git pour inspecter l'historique du projet et en cerner toute l'utilité.

LIRE LE LOG

La manière la plus simple d'inspecter l'historique de votre projet consiste à examiner une liste ordonnée de commits. L'outil principal de Git pour afficher cette liste est la commande `git log`. Les services d'hébergement comme GitHub proposent des outils web pour parcourir les commits antérieurs : ils font la même chose que `git log`, mais produisent un agencement plus lisible pour l'utilisateur. L'intérêt d'apprendre à se servir de la commande `git log` est que, comme le reste de l'interface en ligne de commande de Git, elle fonctionne toujours de la même façon, quel que soit l'ordinateur ou le service d'hébergement. Et que contrairement à une recherche de commit sur GitHub, elle fonctionne hors ligne.

Par défaut, invoquer `git log` présente une liste de tous les commits du projet depuis le commit de tête actuel, et ce jusqu'au début du projet, dans l'ordre chronologique inverse :

```
$: git log
commit 45b1ec87cd2fde95a110dfe3028e93d25c9af186
Author: David Demaree <david@demaree.me>
Date:   Fri Dec 26 16:28:41 2014 – 0500

    styles.css renommé en main.css

commit bf8144d4690d3f6052dc7f42135e3e9944b96b5a
Author: David Demaree <david@demaree.me>
Date:   Thu Dec 25 13:24:25 2014 – 0600

    Commit initial
```

Les lignes commençant par `commit` indiquent un commit, qui occupe plusieurs lignes. La longue chaîne de lettres et de chiffres est l'identifiant du commit. On peut lire en dessous l'auteur (`Author`) du commit, ainsi que la `Date` à laquelle il a été ajouté. Enfin, on lit le message de commit, indenté sous les métadonnées.

Voilà l'historique qui s'est accumulé au fil des modifications et des commits du projet. C'est pour ces logs que les messages de commit existent, et on comprend pourquoi ils doivent rester brefs. Idéalement, vous devez pouvoir avoir une idée de l'évolution d'un projet simplement en parcourant le résultat de `git log` et les messages du log.

L'exemple précédent montre le produit par défaut du log. Mais Git peut vous en dire autant — ou aussi peu — que vous le voulez sur vos commits, dans quasiment tous les formats. L'option `--pretty` permet de choisir un format parmi plusieurs options prédéfinies, ou de spécifier le vôtre à l'aide d'une chaîne. Le format intégré `oneline` affiche uniquement l'identifiant de commit et le message sur une même ligne :

```
$: git log --pretty=oneline
45b1ec87cd2fde95a110dfe3028e93d25c9af186 styles.css »
  renommé en main.css
 bf8144d4690d3f6052dc7f42135e3e9944b96b5a Commit »
  initial
```

Vous trouverez une liste complète des formateurs de log et la syntaxe pour définir le vôtre sous forme d'une chaîne en consultant la documentation de Git (http://bkaprt.com/gfh/05-01/). De plus, Atlassian a publié un tutoriel complet mais accessible où il présente toutes les options de mise en forme du log, et explique rapidement à quoi servent certains formats (http://bkaprt.com/gfh/05-02/).

Préciser le point de départ

Comme on l'a appris plus tôt, le concept d'historique de Git repose sur la filiation : un commit contient une référence vers un ou plusieurs commits parents, qui pointent vers leurs commits parents, et ainsi de suite jusqu'au début du projet. `git log` semble afficher un historique de notre projet dans l'ordre chronologique inverse, mais cet ordre n'est en réalité qu'un effet secondaire. En effet, Git suit simplement la chaîne de filiation des commits pour vous indiquer d'où provient le commit en cours.

Par défaut, la liste présentée commence par le commit de tête de votre branche actuelle. Si vous avez extrait `master`, vous voyez toute la filiation du premier commit de `master`. Vous pouvez toutefois lui demander d'utiliser n'importe quel commit ou branche comme point de départ. Par exemple, on lui demande ici de présenter l'historique de notre branche `nouvelle-page-accueil` :

```
$: git log nouvelle-page-accueil
```

Afficher une plage de commits

Vous pouvez même préciser des plages de commits : autrement dit, vous pouvez demander toutes les entrées de log entre deux commits donnés, afin de voir uniquement ce qui a changé entre deux points précis de votre projet.

C'est très intéressant pour voir une liste de ce qui différencie une branche thématique depuis sa séparation de la branche principale. Cette fois, nous allons demander à `git log` d'afficher tous les commits ajoutés à `nouvelle-page-accueil` qui n'ont pas encore été fusionnés dans `master`, en ajoutant l'option `--oneline` pour rendre le résultat plus lisible :

```
$: git log --oneline master..nouvelle-page-accueil
bce44eb Boutons navigation plus gros
056c8fd Nouvelle image de fond sur zone héros
7e53652 Chargement fontes asynchrone
```

La plage est définie sous la forme `début..fin`, `anciennebranche..nouvellebranche` ou pour être vraiment pédant, `branche..brancheavecdautrescommits`. Comme vous le voyez, `git log` ne se soucie pas de la chronologie, et vous savez que rien n'empêche `master` d'avoir des modifications qui n'ont pas encore été fusionnées dans une branche thématique comme `nouvelle-page-accueil`. Pour dire les choses le plus simplement possible, `git log branche-a..branche-b` affiche une liste de tous les commits de `branche-b` qui ne figurent pas dans `branche-a`. Dans l'exemple précédent, trois commits de

`nouvelle-page-accueil` n'ont pas encore été fusionnés dans `master`.

Tout l'intérêt réside dans le fait qu'on peut demander à `git log` de procéder dans l'autre sens, et d'afficher une liste des commits de `master` qui ne figurent pas dans `nouvelle-page-accueil`.

```
$: git log --oneline nouvelle-page-accueil..master
5514d53 Corrigé bug JS sur page produit
4af326c Ajouté support Microsoft Edge
```

Cela fonctionne également avec les branches distantes, et vous pouvez donc savoir si une copie d'une branche est en retard par rapport à celle du serveur. Là, je demande à `git log` d'afficher une liste des commits du serveur que je n'ai pas encore récupérés sur ma branche locale, en appliquant une chaîne de format personnalisée afin de connaître l'auteur de chaque commit :

```
$: git log --pretty='format:%h - %an: %s' »
  nouvelle-page-accueil..origin/nouvelle-page-accueil
635ce39 – Susan Oliver: Modifs importantes sur mentions
légales 65ae00e – Stewart Colbert: Belles promesses en JS
```

Si l'une des bornes de l'intervalle de commits est la HEAD actuelle (autrement dit, le commit actuellement extrait dans votre copie de travail), vous pouvez laisser un blanc. Ici, c'est `nouvelle-page-accueil` qui est extrait, et l'on veut voir une liste des commits provenant de `master` :

```
[nouvelle-page-accueil] $: git log --oneline ..master
5514d53 Corrigé bug JS sur page produit
4af326c Ajouté support Microsoft Edge
```

C'est exactement le résultat que nous avions obtenu en précisant `nouvelle-page-accueil..master` auparavant. Comme c'est `nouvelle-page-accueil` qui est extrait, Git déduit qu'il s'agit de l'autre terme de la comparaison que nous souhaitons effectuer, et économise ainsi un peu nos doigts.

Filtrer le log

Enfin, comme si ça ne suffisait pas, vous pouvez passer des options de filtrage à `git log` pour limiter la liste des commits et afficher uniquement un certain nombre de commits récents, les commits ajoutés dans une période donnée ou les commits ajoutés par un membre spécifique de l'équipe. La commande suivante présente par exemple seulement les commits dont je suis l'auteur, qui proviennent de l'un de mes repositories, contiennent le mot « Heroku » et ont modifié le fichier `Gemfile` :

```
$: git log --author=Demaree --grep=heroku »
  --oneline Gemfile
94d8ecb Modif Gemfile pour supprimer heroku
ccc5266 Merge prep heroku dans master
```

Et nous ne faisons qu'effleurer la surface des possibilités de `git log`. Pour en savoir plus sur ce puissant outil, le tutoriel d'Atlassian offre une excellente synthèse des possibilités (http://bkaprt.com/gfh/05-03/).

TOUT SAVOIR SUR LES ID DE COMMIT

L'identifiant unique d'un commit donné compte parmi les choses les plus utiles à chercher à l'aide de `git log`. Les ID de commit de Git ont plusieurs usages, mais le plus important reste le plus évident : identifier un commit, où commit signifie « la modification qui a cassé toutes les balises d'images, tu sais, la `65ae00e` ». Jusqu'à présent, nous n'avons vu les ID de commit que sous cette forme courte. La plupart du temps, pour des raisons que j'expliquerai, la forme courte suffit et vous n'aurez besoin de rien d'autre. Mais il arrive parfois que vous ayez besoin de voir les ID de commit sous leur forme entière, comme ceci :

```
65ae00edfe8a795199ed416a9d6df8c3cfe8bd0a
```

Quelle est la différence ? Et pourquoi Git utilise-t-il ces étranges chaînes de lettres et de chiffres pour identifier les révisions, au lieu de se contenter de numéros ?

Comme on l'a vu au chapitre précédent, si nous sommes nombreux à utiliser Git selon un modèle centralisé, Git est fait pour être décentralisé. Chacun de nos ordinateurs possède son propre exemplaire du repository, capable d'évoluer indépendamment des autres. Chacun peut apporter des modifications et des commits à une branche en étant hors ligne, et personne n'a besoin de savoir dans l'immédiat ce que font les autres, jusqu'à ce qu'il soit temps de synchroniser les copies locales avec la copie distante. Lorsqu'on effectue ces commits, Git a besoin d'affecter un nom ou un numéro d'identification unique à chacun d'eux, mais Git n'a aucun moyen de savoir à l'avance si un autre ordinateur a déjà utilisé ce nom ou ce numéro.

De plus, la conception de Git donne la priorité à la stabilité et à l'intégrité des données, devant tout le reste. Dans une présentation de 2007, Linus Torvalds a expliqué que les systèmes de gestion de versions devaient garantir la véracité des données dont ils avaient la charge, et a évoqué les fonctionnalités de Git qui assurent l'exactitude des données (http://bkaprt.com/gfh/05-04/) :

> *Si votre disque est corrompu, ou si vous avez le moindre problème, Git le remarquera. Je vous le promets, si vous confiez vos données à Git, vous pouvez être sûr que cinq ans plus tard, après avoir transféré le contenu de votre disque dur sur un DVD ou je ne sais quel support qui aura été inventé d'ici là, vous pourrez vérifier que vos données seront parfaitement identiques à celles que vous lui avez données cinq ans auparavant.*

Git résout les deux problèmes en créant et en utilisant des identifiants basés sur le contenu de chaque commit, plutôt qu'en affectant un nom ou un numéro arbitraire. Techniquement, les ID de commit ne sont pas tant des identifiants que des *sommes de contrôle*, sorte d'empreinte digitale numérique généralement utilisée pour committer des données transmises sur un réseau. Les versions logicielles sont souvent accompagnées d'une liste de sommes de contrôle. Cela permet par exemple aux

utilisateurs qui téléchargent une version préliminaire de Windows de vérifier que le fichier téléchargé est complet et qu'il n'a pas été altéré.

Quand vous ajoutez un commit, Git prend tout ce qui le constitue (nom de l'auteur, adresse e-mail, date et heure, message de commit, références aux commits parents et snapshot du projet) et lui applique une fonction de hashage afin de générer une chaîne de 40 caractères. On a ainsi la garantie quasiment absolue que la valeur obtenue identifie le commit de façon unique. Et c'est le cas si le même commit est effectué sur deux machines différentes. Deux commits identiques auront des hashes identiques, et donc des identifiants identiques, quel que soit l'ordinateur qui les a ajoutés sur le repository. À l'inverse, des commits qui présentent la moindre différence (ne serait-ce qu'un nom d'auteur différent) ont l'assurance de recevoir des identifiants distincts. Chaque hash identifie donc un seul commit, et ce de façon unique.

Si ces longs hashes facilitent la collaboration en permettant d'échanger des commits d'une machine à l'autre, ils sont également sources de problèmes pour nous. Leur lecture et leur écriture sont peu pratiques. Heureusement, même si vous n'indiquez qu'un fragment de l'ID de commit, Git est assez malin pour comprendre de quel commit il s'agit, dès lors que l'ID court comprend au moins quatre caractères et qu'il est unique dans votre repository.

Par exemple, l'ID de commit que j'ai présenté au début de cette section pourrait être raccourci à quatre caractères (65ae) sans qu'aucune confusion ne soit possible avec un autre commit de ce projet. Et en pratique, dans la plupart des repositories Git, un ID de sept caractères comme 65ae00e suffit à identifier de façon unique n'importe quel commit, même si le repository en contient des dizaines de milliers. C'est pourquoi Git utilise fréquemment des identifiants courts dans ses réponses plutôt que leur forme longue.

Dans les rares cas où deux ID courts se chevauchent, Git est également capable de gérer la situation en leur ajoutant automatiquement des chiffres supplémentaires. Dans le projet du noyau Linux (http://bkaprt.com/gfh/01-02/) par exemple, qui est sans

doute le repository Git le plus ancien et certainement le plus vaste, il s'avère que sept caractères ne suffisent pas pour éviter la superposition d'identifiants – mais onze, si. Git modifie donc automatiquement son format d'identifiant court de manière à employer le plus petit nombre de caractères permettant une identification unique à l'échelle de tout le projet.

MESSAGES DE COMMIT

Git et les outils comme GitHub proposent différentes manières (dont nous détaillerons certaines dans ce chapitre) d'afficher ce qui a effectivement changé lors d'un commit. Mais un message de commit bien écrit peut vous éviter d'avoir recours à ces outils en résumant judicieusement (et brièvement) ce qui a changé.

Le message de log est sans doute la partie la plus importante d'un commit, parce que c'est la seule chose qui indique non seulement ce qui a changé, mais aussi pourquoi.

Quels sont les ingrédients d'un bon message ? Tout d'abord, il doit être court, et pas seulement parce que la brièveté est l'essence du trait d'esprit. La plupart du temps, les messages de commit sont présentés dans le contexte du log de Git, où il y a peu d'espace pour afficher du texte.

Voyez le commit comme le fil d'actualité de votre projet : le message de log est le titre de chaque commit. Avez-vous déjà parcouru les titres d'un journal ou, pour prendre un exemple plus contemporain, de Buzzfeed, et eu l'impression d'avoir eu un bon résumé de ce qui se passe dans le monde ? Un bon titre ne doit pas raconter toute l'histoire, mais il doit en dire suffisamment pour que vous sachiez de quoi il en retourne avant de la lire.

Si vous travaillez seul ou en étroite collaboration avec un ou deux collègues, l'intérêt du log peut paraître strictement historique, puisque vous aurez assisté à la plupart des commits. Mais dans un repository Git comptant de nombreux collaborateurs, le commit log vous permet de savoir ce qui s'est passé pendant que vous aviez le dos tourné.

Théoriquement, les messages de commit peuvent couvrir plusieurs lignes, et être aussi longs et détaillés que vous voulez. Git n'impose pas de limite à ce qui entre dans un message de commit et, en effet, si un commit nécessite un complément de contexte, vous pouvez ajouter des paragraphes supplémentaires, comme ceci :

```
MàJ version de Ruby on Rails pour sécurité

Passage à Rails version 3.2.11 pour corriger »
  bug de sécurité JSON.
Cf. http://weblog.rubyonrails.org/2013/1/8/ »
  Rails-3-2-11-3-1-10-3-0-19-and-2-3-15-have-been- »
  released/
```

Vous remarquerez que si ce message contient bien plus qu'une ligne de contexte, la première est importante, car c'est la seule qui sera visible dans le log :

```
commit f0c8f185e677026f0832a9c13ab72322773ad9cf
Author: David Demaree <david@demaree.me>
Date:   Sat Jan 3 15:49:03 2013 – 0500

    MàJ version de Ruby on Rails pour sécurité
```

Comme tout bon titre, la première ligne résume ici la raison du commit, et le reste du message entre dans le détail.

Rédiger des messages de commit dans votre éditeur de texte préféré

Bien que tous les exemples de ce livre vous fassent taper vos messages dans la ligne de commande à l'aide de l'argument `--message` ou `–m` de la commande `git commit`, vous préférerez peut-être les rédiger dans votre éditeur de texte favori. Git s'intègre avec élégance avec de nombreux éditeurs courants, qu'ils soient en ligne de commande comme Vim ou Emacs, ou sous forme d'applications graphiques plus modernes comme Atom, Sublime Text ou TextMate. Une fois un éditeur configuré, vous

pouvez omettre le marqueur `--message` ; Git propose alors une ébauche de message à l'autre programme en vue de son édition. Quand vous avez terminé, il suffit généralement de fermer la fenêtre pour que Git récupère le message que vous avez saisi.

Pour profiter de cette agréable intégration, vous devez d'abord configurer Git pour qu'il utilise votre éditeur (spécifiquement, l'interface en ligne de commande de votre éditeur, s'il en possède une). Ici, j'indique à Git de confier les messages de commit à Atom :

```
$: git config --global core.editor "atom --wait"
```

Chaque éditeur de texte propose un ensemble légèrement différent d'arguments et d'options à passer pour s'intégrer correctement à Git. (Comme vous le voyez ici, il a fallu passer l'option `--wait` à Atom pour que cela fonctionne.) La documentation d'aide de GitHub offre un excellent guide rapide pour intégrer plusieurs éditeurs courants (http://bkaprt.com/gfh/05-05/).

Règles et tag des messages de commit

Il y a peu de règles strictes pour la rédaction de messages de commit efficaces, mais les recommandations et les bonnes pratiques sont nombreuses et vous vous feriez rapidement des nœuds au cerveau à tenter de toutes les appliquer.

Pour vous faciliter la tâche, voici quelques conseils que je recommande de suivre systématiquement.

Soyez utile

Le but d'un message de commit est de résumer un changement. Mais le but de ce résumé est d'aider votre équipe et vous-même à comprendre le déroulement de votre projet. Les informations que vous mettez dans un message doivent donc être utiles à ceux qui le liront.

Même s'il peut paraître amusant d'utiliser les messages de commit pour vous défouler — sur un bug, sur Git ou sur votre propre maladresse — évitez de faire de la littérature. Retenez-vous de laisser un message comme « aaaaaaah fichus bugs !!! ».

Respirez profondément, faites-vous un café ou une camomille, bref, faites ce que vous voulez pour retrouver votre calme. Rédigez ensuite un message décrivant ce qui a changé dans ce commit, aussi clairement et succinctement que possible.

Outre une description claire et brève du changement, quand un commit a trait à des informations qui se trouvent dans un autre système (par exemple parce qu'il corrige un bug figurant dans votre outil de suivi), on ajoute couramment le numéro du ticket ou du bug, comme ceci :

```
Remplacé l'écouteur onReady jQuery par JS simple; »
  corrige #1357
```

Certains outils de suivi de bug (dont celui qui est intégré à chaque projet GitHub) peuvent même être raccordés à Git, de manière à ce que les messages de ce type signalent automatiquement le bug `1357` comme fermé dès leur merge dans `master`.

Soyez (assez) précis

En tant que développeur logiciel en rémission, je comprends la tentation de remplir le message de commit (de même que les e-mails, les rapports de situation et les réunions) de détails techniques. J'adore les détails techniques. Mais si certains détails sont essentiels pour comprendre une modification, il y a toujours une raison plus globale — et plus succincte. De plus, il n'y a généralement pas la place d'indiquer chaque détail d'une modification si l'on veut préserver la lisibilité du commit log dans la fenêtre du terminal. Trouver un moyen plus simple de décrire votre modification, ce n'est pas seulement la rendre plus compréhensible pour vos collègues, c'est aussi un excellent moyen de gagner de la place.

Une bonne règle consiste à faire tenir la portion « objet » de votre message de commit sur une ligne, soit environ 70 caractères. Si vous avez d'autres informations importantes à inclure dans le message mais qu'elles n'ont pas besoin de figurer dans la ligne d'objet, vous pouvez tout de même les ajouter dans un paragraphe supplémentaire.

Soyez cohérent

Quelle que soit la façon dont vos collègues et vous-même décidez d'écrire les messages de commit, votre log sera plus pratique si vous vous efforcez de tous suivre les mêmes règles. Les messages de commit sont trop courts pour nécessiter un guide de style élaboré, mais vous gagnerez tous à prendre le temps de mettre en place quelques conventions ou de créer une petite page Wiki contenant des exemples de bons (et de mauvais) messages.

Utilisez la voix active

Le commit log n'est pas une liste de choses statiques, c'est une liste de changements. C'est une liste d'*actions* qui ont produit des versions successives de votre travail. S'il peut être tentant d'utiliser un message de commit pour étiqueter une version du travail (« Version 1.0 », « livrable du 24 janvier », etc.), il existe d'autres manières de le faire — et de meilleures. Et puis on a vite fait de se retrouver dans des situations embarrassantes :

```
# Dernière modif de la page d'accueil avant le lancement
  du nouveau site
$: git commit -m "Version 1.0"

# Dix minutes plus tard, après avoir trouvé une coquille
  dans la CSS
$: git commit -m "Version 1.0 (pour de vrai)"

# Quarante minutes plus tard, après avoir trouvé une
  autre coquille
$: git commit -m "Version 1.0 (et puis m…)"
```

Décrire le changement n'est pas seulement le format le plus correct d'un message de commit, c'est aussi la règle la plus facile à suivre. Au lieu de vous poser des questions abstraites et de vous demander si un commit est la version publiable d'une chose ou d'une autre, concentrez-vous sur quelque chose de simple : j'ai fait une chose, et la voici.

Tous ces commits « Version 1.0 » pourraient ainsi être décrits de façon plus simple et plus précise :

```
$: git commit -m "MàJ page d'accueil pour lancement"
$: git commit -m "Corrigé coquille dans screen.scss"
$: git commit -m "Corrigé coquille dans la page
Présentation"
```

Je recommande également de choisir un temps verbal et de vous y tenir, pour des questions de cohérence. J'ai tendance à utiliser la forme verbale la plus courte possible pour décrire les commits : *Corrigé coquille dans la page Présentation*, plutôt que *Correction d'une coquille* ou *J'ai corrigé*. Les autres formes sont correctes, mais elles sont un peu plus longues. Si un autre style fonctionne mieux pour votre équipe, foncez. Essayez simplement de le faire de façon uniforme. Que se passera-t-il si votre style de message de commit n'est pas uniforme ?

Votre repository Git s'effondrera sur lui-même et tout votre travail disparaîtra. Je blague ! L'être humain est faillible, on peut parfois se relâcher, et vos logs contiendront inévitablement des choses bizarres. Mais les règles de style présentées ici deviendront plus faciles à suivre au fil du temps. Efforcez-vous de rédiger les meilleurs messages de commit possible, et vous aurez des logs plus utiles et plus cohérents.

Pour en savoir plus sur l'art d'écrire des messages de commit, consultez l'article de Tim Pope intitulé « A Note About Git Commit Messages » (http://bkaprt.com/gfh/05-06/) et celui de Chris Beams, « How To Write A Git Commit Message » (http://bkaprt.com/gfh/05-07/).

FAIRE DE BONS COMMITS

Dans 24 Ways, Emma Jane Westby écrit que « les commits doivent contenir des idées complètes de travail abouti » (http://bkaprt.com/gfh/05-08/). Pour nous, simples mortels, le rôle d'un commit consiste à regrouper des modifications en portions logiques. Parfois, la logique qui sous-tend un groupe de

modifications donné est aussi simple que : « C'est le moment où moi, développeur, j'ai pensé qu'il était bon d'enregistrer mes progrès. » Mais il y a parfois plus à dire — et à comprendre — d'un changement.

Pour un outil logiciel aussi soucieux de la propreté et de la cohérence de vos données, Git est incroyablement flexible quant à ce que vous pouvez committer, et au moment où vous pouvez le faire. L'un des grands avantages de Git (qui est aussi source de confusion) est qu'il n'exige pas d'indexer ou de committer tout ce que vous avez modifié en une fois. Git vous permet de committer certains fichiers modifiés (voire une portion d'un fichier modifié), tout en laissant d'autres éléments ni indexés ni committés. Si vous apportez trois changements indépendants à une même feuille de styles, vous pouvez committer chacune de ces modifications séparément ou ensemble, comme ça vous arrange.

Imaginons que vous travailliez sur un projet dont vous avez modifié un fichier JavaScript et le fichier README, pour des raisons tout à fait différentes. Notre statut est le suivant :

```
[master] $: git status
# On branch master
#
# Initial commit
#
# Untracked files:
#   (use "git add <file>..." to include in what will
  be committed)
#
#     README.md
#     site.js
```

L'approche la plus simple consisterait à committer les deux fichiers en même temps avec un message de log commun comme « Ajouté l'écouteur onReady ; MàJ README ». Mais s'il vous semble plus judicieux de committer les deux modifications séparément en donnant plus de contexte à vos logs, Git vous permet de le faire assez facilement.

Nous allons d'abord indexer et committer l'un des deux changements :

```
$ git add site.js
$ git commit -m "Ajouté écouteur onReady"
[master 591672e] Ajouté écouteur onReady
 1 file changed, 3 insertions(+)
```

À la fin de l'opération, le README est toujours modifié et non indexé, prêt à être committé séparément :

```
$ git add README.md
$ git commit -m "MàJ README"
[master 96406dd] MàJ README
 1 file changed, 1 insertion(+)
```

Maintenant, si nous consultons le log, nous aurons un peu plus l'impression que chaque entrée — chaque commit, donc — représente une seule idée complète.

Ce n'est pas toujours évident à réaliser et, comme bien d'autres bonnes pratiques, les commits parfaits, englobant une unité de travail et accompagnés d'un message idéalement formulé, sont l'exception plus que la règle. Ne vous flagellez pas si vous devez ajouter un gros commit sous le titre un peu vague de « Corrigé l'en-tête ». Sachez simplement qu'il est possible de faire mieux et visez ce mieux la prochaine fois.

COMPARER DES COMMITS

Nous avons beaucoup parlé des versions, des statuts, et de la façon dont les modifications s'accumulent au fil du temps. Quand nous gérons notre travail à l'échelle du commit, nous sommes encouragés à penser au-delà de l'état actuel de notre travail et à envisager celui dans lequel il était hier, et dans lequel il sera demain. En écrivant des messages de commit, en choisissant ce que nous voulons inclure dans un commit, nous sommes amenés à réfléchir à la description de nos actions au moment même

où nous agissons. Et c'est ainsi qu'on adopte une approche plus réfléchie et plus pondérée de notre travail.

Surtout, Git nous demande de traiter chaque modification de notre projet — ou plus formellement, chaque transition vers un nouvel état représenté par un commit — comme un événement réel. Chaque commit représente non seulement un snapshot de notre projet mais (à l'exception de la première, bien sûr) aussi un changement par rapport à un commit précédent. À terme, quand on commence à penser et travailler en versions, il arrive un moment où l'on a besoin (ou envie) de comparer des versions pour voir, spécifiquement, ce qui a changé. Un message de commit peut vous donner un résumé, mais Git offre également un moyen pratique d'inspecter réellement les différences entre deux commits.

`git diff` (abréviation de « différence ») présente les modifications entre deux versions de votre projet ou deux versions d'un ou plusieurs fichiers. De ce point de vue, cette commande est très semblable à `git log`, et vous pouvez effectivement afficher les informations de différence dans le résultat de votre log si vous le souhaitez. En plus de comparer des versions committées, si vous avez des modifications non committées, vous pouvez utiliser `git diff` pour afficher tout ce qui distingue votre arbre de travail du dernier commit.

Ici, `git diff` montre une modification simple du fichier README que nous examinions tout à l'heure :

```
$: git diff
diff --git a/README.md b/README.md
index 0c0a11f..48fb805 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,3 @@
-# My Project
\ No newline at end of file
+# My Project
+
+This is a project managed by Git.
\ No newline at end of file
```

Avouons-le, ce n'est pas ce qu'il y a de plus lisible. Le résultat par défaut de `git diff` est généré par un outil de comparaison Unix appelé « diff », développé à l'origine dans les années 1970 et affiché par un programme de pagination baptisé « less », dont la mission consiste à afficher des textes plus longs que votre fenêtre de terminal. (Je précise que cet exemple est relativement simple. La plupart du temps, les diffs sont plus longs et plus complexes.)

Voici ce qui se passe dans ce diff : les lignes commençant par signes moins (-) sont celles que nous avons supprimées depuis le dernier commit, et les lignes qui commencent par des plus (+) sont celles qui ont été ajoutées.

Les diffs (tout comme Git dans sa globalité) se concentrent sur les lignes modifiées de vos fichiers, et la modification d'un seul caractère suffit pour que Git considère la ligne comme modifiée. De plus, de même que renommer un fichier se traduit, pour Git, par la suppression d'un ancien fichier et l'ajout d'un nouveau, la modification d'une ligne est envisagée comme une suppression suivie d'un ajout. Le diff le montre clairement : le titre principal est présent dans les deux versions, sous la forme d'une suppression et d'un ajout. Qu'est-ce qui a changé ? Dans la version committée, le titre n'était pas suivi d'un saut de ligne. (Oui, même l'ajout d'un saut de ligne suffit à ce qu'une ligne soit signalée comme modifiée.)

Les diffs peuvent être extrêmement utiles, mais à moins d'avoir l'habitude du format du diff Unix, ils sont l'un des rares outils de Git pour lequel je vous encourage vivement à utiliser un outil avec interface graphique, que ce soit une application sur votre ordinateur ou au sein d'un service d'hébergement comme GitHub. Dans le cas de projets hébergés sur GitHub ou GitHub Enterprise, chaque repository propose un outil de comparaison (accessible en ajoutant `/compare` à la fin de l'URL du projet) qui résume très bien les modifications sous une forme graphique et facile d'emploi (**FIG 5.1**).

Les utilisateurs de Mac peuvent également se tourner vers Kaleidoscope de Black Pixel (http://bkaprt.com/gfh/05-09/). Kaleidoscope est un outil de comparaison généraliste qui sert

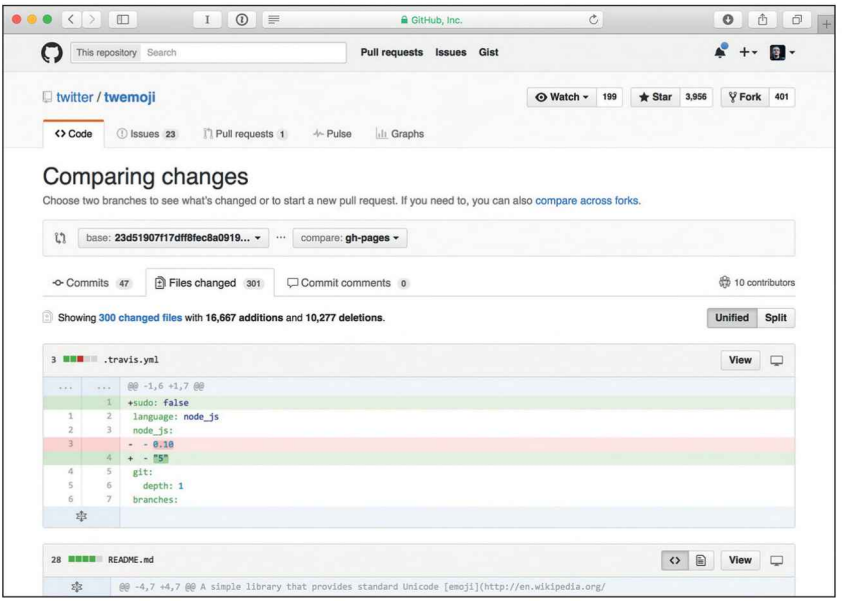


FIG 5.1 : Pour les projets hébergés sur GitHub, les vues comparées permettent d'afficher — et de partager — des diffs faciles à lire grâce à la coloration.

à comparer deux fichiers de n'importe quel type, qu'ils soient ou non gérés par Git. Cela dit, il offre une excellente intégration avec Git, grâce notamment à un outil de configuration simple qui incite Git à ouvrir les diffs dans Kaleidoscope via la commande `git difftool`.

Git propose un format de diff plus simple et plus lisible par des humains si l'on souhaite revenir à la ligne de commande : « diff stat », qui réduit tout un diff à une liste des fichiers qui diffèrent entre deux versions, balisés de manière à indiquer la nature de la modification.

Ici, je demande à `git diff` d'afficher les statistiques des différences entre le commit `HEAD` actuel du repository du Web Font Loader de Typekit et la précédente (`HEAD~1`) :

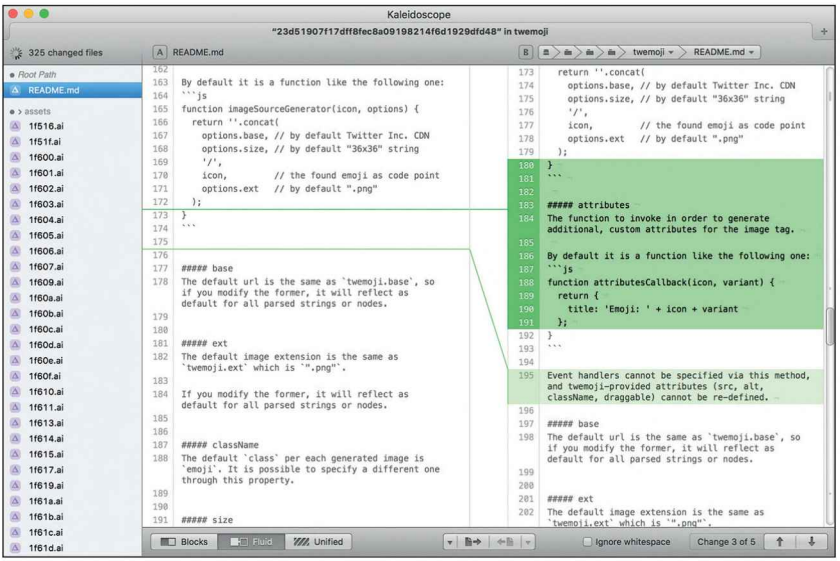


FIG 5.2 : Git peut être intégré à des applications de comparaison de fichiers comme l'excellent Kaleidoscope de Black Pixel, qui affiche les diffs sous une forme colorée et facile à parcourir.

```
$ git diff --stat HEAD~1
 CHANGELOG              | 3 +++
 lib/webfontloader.rb   | 2 + –
 webfontloader.gemspec  | 2 + –
 webfontloader.js       | 4 ++--
 4 files changed, 7 insertions(+), 4 deletions(-)
```

Bien qu'il soit plus concis que le diff complet, le diff stat offre un bon résumé des modifications de ce commit. Chaque ligne présente un fichier modifié lors du commit, comme `lib/webfontloader.rb`. Ensuite, séparées par un caractère pipe (|), sont énoncées les statistiques du fichier : deux modifications (un ajout et une suppression). Maintenant que nous connaissons Git, et sachant qu'il n'y a qu'un seul commit (donc peu de changements), on peut déduire qu'il s'agit d'une modification d'une ligne, un changement de numéro de version par exemple.

Ensuite, si l'on a besoin de plus d'informations, on peut demander un diff complet d'un fichier particulier (avec la commande `git diff HEAD~1 webfontloader.js`), d'un groupe de fichiers (en passant plusieurs noms de fichiers) ou du projet au complet. On peut également demander des statistiques à propos de commits effectués sur une période bien plus longue :

```
$: git diff -stat HEAD~15
 .travis.yml                        |   5 + -
 CHANGELOG                          |  12 ++
 README.md                          |  24 +--
 lib/webfontloader.rb               |   2 + -
 package.json                       |   5 + -
 spec/core/fontwatcher_spec.js      |   3 -
 spec/core/fontwatchrunner_spec.js  | 441 +++++++++--
  ------------
 src/core/domhelper.js              |  26 ++ -
 src/core/fontruler.js              |   2 + -
 src/core/fontwatcher.js            |  22 + -
 src/core/fontwatchrunner.js        |  33 +--
 webfontloader.gemspec              |   6 + -
 webfontloader.js                   |  42 ++--
 13 files changed, 207 insertions(+),  416
  deletions(-)
```

Même si cette liste semble résumer les changements d'une cinquantaine de commits d'un coup, il faut savoir que `git diff` (et sa vue stats) ne compare que deux commits à la fois. `HEAD~50` ne représente pas les cinquante derniers commits mais seulement le commit qui occupe la cinquantième place dans la chaîne de filiation. Pensons également que chaque commit représente un snapshot complet de votre projet, et que chaque commit s'appuie sur celui qui le précède. En toute logique, la liste des différences entre votre commit actuel et son cinquantième parent doit être similaire — du moins dans l'esprit — au résumé des cinquante derniers changements, dont l'effet devrait toujours se faire sentir dans votre commit actuel.

Si vous trouvez les statistiques utiles, vous pouvez même les inclure dans le résultat de `git log` à l'aide de l'option `--stat`. Ici, je demande à `git log` d'afficher un log incluant des statistiques, en appliquant un format personnalisé aux entrées du log, et en se limitant aux modifications survenues depuis l'avant-dernier commit.

```
$ git log --stat --pretty=format:"%h (%an) %s" »
  HEAD~1..
d08a7f2 (Bram Stein) Release 1.5.10
 CHANGELOG              | 3 +++
 lib/webfontloader.rb   | 2 + –
 webfontloader.gemspec  | 2 + –
 webfontloader.js       | 4 ++--
4 files changed, 7 insertions(+), 4 deletions(-)
```

ÉTIQUETER LES COMMITS

En plus de toutes les références que nous avons vues — identifiants de commit longs et courts, noms de branches et pointeur HEAD —, les commits peuvent recevoir des noms permanents et lisibles appelés *tags*. Les tags sont semblables aux branches dans la mesure où elles attribuent des noms lisibles par l'homme à des commits particuliers. Mais contrairement aux branches dont les noms, s'ils restent inchangés, flottent et désignent toujours la commit de tête à chaque modification de la branche, les tags désignent toujours un commit spécifique, afin de marquer des moments intéressants ou importants de l'histoire du projet.

Selon votre projet, il se peut que vous n'utilisiez pas du tout les tags, ou que vous vous en serviez énormément. Contrairement aux branches qui jouent un rôle central dans quasiment tous les usages de Git que j'ai pu observer, les tags n'ont pas de signification intrinsèque ni d'utilisation prédéterminée, et beaucoup de projets les ignorent totalement. Au cours du développement d'un site web ou d'une application, l'intérêt des tags peut dépendre de la façon dont vous publiez le code sur votre serveur de production. Beaucoup d'équipes déploient

simplement en récupérant les dernières nouveautés de `master` sur les serveurs. Elles déterminent quel code est rendu public en fixant des règles pour le commit merge et en instaurant des contrôles de qualité afin que tout ce qui se trouve dans `master` soit toujours prêt pour la production. Pour la plupart d'entre nous, les branches ne sont pas seulement plus simples, elles ont aussi plus de sens : le nom de branche `master` ne désigne pas seulement un commit, il référence le dernier commit d'un certain fil de travail. Les noms de branche changent moins souvent et impliquent moins de travail.

Les tags Git sont souvent utilisés pour les librairies logicielles et les frameworks livrés en versions numérotées. Par exemple, le code de la version 4.2.0 du framework Ruby on Rails correspond au tag `rel-4.2.0` sur leur repository Git (http://bkaprt.com/gfh/05-10/), qui pointe quant à elle vers le commit `7847a19`, dont le message, très éclairant, indique « Préparation à la publication de la 4.2.0 ». La version 4.2.0 officielle a la forme d'un paquet Ruby hébergé sur rubygems.org ; le tag sert de lien entre ce paquet et le commit utilisé pour le produire.

Pour étiqueter un commit, il suffit d'utiliser la commande `git tag` qui, vous en conviendrez, porte bien son nom. Elle prend toujours le nom du tag comme premier paramètre, et ce nom peut-être n'importe quelle chaîne de caractères. Nous allons donner le tag `fhqwhgads` au commit en cours de notre branche actuelle. (Si vous trouvez ça bizarre, sachez que j'ai travaillé dans une équipe où nous étiquetions les mises à jour bihebdomadaires de notre site web en utilisant les noms de nos boutiques préférées — `prada.0` par exemple.)

```
$: git tag fhqwhgads
```

Une fois le commit étiqueté, vous pouvez utiliser le nom `fhqwhgads` partout où vous utiliseriez l'ID de commit.

Si le commit qui nous intéresse n'est pas extrait au moment de l'opération, on peut préciser son identifiant dans la commande :

```
$: git tag fhqwhgads 8891c37
```

Comme rien ne peut être simple avec Git, il faut savoir qu'il existe deux types de tags. Celui que nous venons de créer est un tag léger. Il est stocké dans le repository et c'est juste un nom pointant vers un commit, comme dans le cas d'une branche.

Mais il existe également des tags *annotés* qui, outre un nom et une référence vers un commit, peuvent inclure un message similaire à un message de commit.

```
$: git tag fhqwhgads -a -m "Version Fhqwhgads (22 »
  dec 2014)"
```

Les tags, comme les branches, peuvent et doivent être partagés sur un repository distant, et vous pouvez les pousser vers ce repository distant de la même manière, en utilisant `git push` :

```
$: git push fhqwhgads
```

Les règles qui encadrent les tags sont peu nombreuses mais elles sont strictes, comme nous allons le voir.

Les noms des tags doivent être uniques

De même qu'il serait très problématique que deux versions différentes de votre projet portent le même nom, Git ne vous permet pas de créer un tag s'il en existe déjà une sous le même nom. Git rejettera donc un tag poussé s'il existe déjà sur le serveur.

En revanche, Git vous autorisera à donner à un tag le nom d'une branche, et inversement. Mais si vous donnez une instruction ambiguë, Git donnera la priorité à la branche et vous informera que ce n'est peut-être pas une bonne idée. Voici ce qui se passe quand, dans un repository contenant un tag et une branche toutes deux nommées `branche-2`, j'essaie d'extraire `branche-2` :

```
$ git checkout branche-2
warning: refname 'branche-2' is ambiguous.
Switched to branch 'branche-2'
```

Pour vous faciliter l'existence, évitez de donner des noms identiques à vos branches et vos tags. Beaucoup d'équipes qui utilisent les tags ajoutent un préfixe à leur nom pour les distinguer des branches. Notre tag `fhqwhgads` pourrait plutôt s'appeler `pub-fhqwhgads` pour le distinguer d'éventuelles branches `fhqwhgads`. Cela a également l'avantage d'indiquer ce à quoi le tag fait référence : ici, `pub` est une abréviation pour « publication ».

Les tags sont destinés à être permanents

Git autorise la modification de certaines choses, dont les tags. Plus précisément, Git vous permet de supprimer un tag et de le remplacer par un nouveau portant le même nom. (Bon à savoir : si vous étiquetez le mauvais commit par accident, ce qui peut arriver, vous pouvez utiliser `git tag -d <nomtag>` pour supprimer le tag puis en créer un nouveau pointant vers le bon commit.)

Cela dit, un tag est fait pour servir de surnom durable à un commit spécifique, et cette mission devient difficile si les noms des tags ou les commits qu'ils désignent changent. Une fois que vous avez poussé un tag vers un repository distant, en particulier si c'est un hub de collaboration que vous partagez avec d'autres personnes, essayez de ne plus le modifier. Vous serez parfois contraint de le faire, et il peut arriver que recréer un tag soit plus simple que de créer un nouveau tag sous un nouveau nom, mais d'après mon expérience, ces cas restent exceptionnels, et ils ne méritent pas d'envoyer un message à toute votre équipe pour expliquer que `pub-wombat.0` n'est pas forcément le commit qu'il est censé être.

VOYAGER DANS LE TEMPS AVEC git checkout

C'est déjà bien de pouvoir passer en revue ce que vous avez fait, mais Git vous permet véritablement de revisiter le passé en extrayant d'anciens commits à l'aide de la commande `git checkout` que vous utilisez pour changer de branche. Je parle bien d'extraire un commit, dans l'acception propre à la gestion

de versions : quand vous extrayez un commit (ou une branche, d'ailleurs), vous ne voyez pas seulement une version antérieure de votre travail, vous restaurez votre copie locale du projet dans la version que vous avez demandée. On extrait un commit comme on emprunte un livre à la bibliothèque. Et si vous ne voyez pas où votre arbre de travail se situe dans cette métaphore, rappelez-vous que, même si vous travaillez de façon progressive en ajoutant des commits à une branche plutôt qu'en revisitant les précédents, il y a toujours une version du projet qui est extraite : la branche sur laquelle vous travaillez, et à laquelle vous pouvez ajouter d'autres commits.

L'extraction d'un commit ne diffère de l'extraction d'une branche qu'à un seul égard : on ne s'attend pas vraiment à ce que vous ajoutiez d'autres commits après l'extraction. Cela ne veut pas dire que vous ne pouvez pas le faire. Je vais prendre un exemple pour expliquer cette distinction.

Vous recevez de vos utilisateurs des messages indiquant que quelque chose ne fonctionne plus, alors que, vous en êtes certain, elle fonctionnait dans un navigateur ou un appareil donné quand vous avez déployé votre projet il y a quelques semaines. Disons également qu'au moment où vous avez poussé le commit en production, vous avez attribué à ce commit le tag `pub-v1.0`.

La première chose à faire consiste à confirmer que le code que vous avez déployé à l'origine fonctionnait effectivement, en extrayant la version antérieure et en l'ouvrant dans un navigateur. Nous partirons du principe qu'il s'agit d'un site web statique que vous pouvez ouvrir directement dans un navigateur, mais si votre site nécessite une phase de compilation (avec Grunt, Middleman ou autre outil), cela devrait également fonctionner. Exécutez simplement la compilation ou la tâche après avoir extrait l'ancien site.

Pour cela, exécutez `git checkout` en précisant le tag ou l'ID du commit auquel vous voulez revenir :

```
$: git checkout pub-v1.0
Note: checking out 'pub-v1.0'.
```

Cette commande a fait ce que nous attendions d'elle : Git a restauré la version des fichiers et des répertoires de notre arbre de travail à laquelle nous essayons de revenir, qui correspond au commit `591672e`, également connu sous le nom de `pub-v1.0`. Nous pouvons maintenant ouvrir le site et confirmer qu'effectivement, il fonctionnait quand nous l'avons livré. À partir de là, nous pouvons poursuivre notre investigation en examinant le log et les commits ajoutés à `master` depuis celle-ci (`git log pub-v1.0..master`), ou passer en revue les véritables différences entre cette version de départ et la plus récente (`git diff pub-v1.0..master`). Si un certain commit semble avoir introduit un bug, vous pouvez l'extraire pour confirmer (ou infirmer) vos soupçons. Git propose même un outil (`git bisect`) qui effectue ce type de recherche binaire et trouve automatiquement le commit responsable d'un problème donné. Tobias Günther a rédigé une excellente synthèse sur l'utilisation de `git bisect` dans la chasse aux bugs pour A List Apart (http://bkaprt.com/gfh/05-11/).

Qu'est-ce que le statut de HEAD détaché ?

Quand vous extrayez un commit, par opposition à une branche, Git vous place dans l'état de « HEAD détaché » : le pointeur HEAD de votre ordinateur pointe vers un commit particulier mais pas une branche. Vous êtes « détaché » dans le sens où vous ne travaillez sur aucune branche. En pratique, cela veut dire que vous pouvez ajouter des commits et qu'ils seront enregistrés, mais vous n'aurez aucun nom de branche pour y faire référence.

Vous n'êtes pas réellement censé ajouter des commits quand vous êtes dans cet état détaché. La plupart du temps, Git s'attend à ce que vous extrayiez un commit antérieur pour relire ou tester du code, pas pour faire des modifications. (C'est à cela que servent les branches.)

Mais vous pouvez utiliser cette caractéristique à votre avantage : les commits effectués dans l'état détaché peuvent servir de bac à sable. Dans cet état, vous êtes libre d'apporter des modifications expérimentales et de les committer, puis de faire disparaître les commits ainsi effectués sans aucun impact sur les branches,

tout simplement en réalisant une nouvelle extraction. Il y a même encore moins de risque qu'une modification erronée se fraye un chemin dans la copie commune du projet ou même en production. En effet, à moins que vous ne les déplaciez dans une branche en exécutant `git branch` ou `git checkout -b`, les commits réalisés dans l'état détaché sont sans domicile jusqu'à ce que vous décidiez de créer une branche pour les contenir. Si vous voulez créer une nouvelle branche pour accueillir les commits que vous créez, vous pouvez le faire (maintenant ou plus tard) en utilisant `-b` avec la commande `checkout`. Par exemple :

```
$: git checkout -b nom_nouvelle_branche
HEAD is now at 591672e... Release v1.0
```

Pour sortir de l'état de `HEAD` détaché, il suffit d'extraire une branche. Dans ce cas, on revient à `master` :

```
$: git checkout master
Previous HEAD position was 591672e... Release v1.0
Switched to branch 'master'
```

`git checkout` traite toutes les références qui ne sont pas un nom de branche comme un commit, même si vous la demandez en utilisant une référence qui inclut le nom de la branche : vous pouvez ainsi vous retrouver dans l'état détaché par inadvertance. Comme nous l'avons vu au chapitre précédent, si vous utilisez `git checkout plus-gros-le-logo` pour extraire la branche nommée `plus-gros-le-logo`, alors vous avez extrait une branche. Toutefois, si vous demandez d'extraire `origin/plus-gros-le-logo` (une référence à une branche distante), vous extrayez le commit qui se trouve actuellement à la tête de la branche mais pas la branche.

MOT DE LA FIN

S'il y a une expression que je n'ai pas utilisée dans ce livre, c'est *graphique orienté acyclique* (GOA). Un graphique orienté acyclique est un type de structure de données dans laquelle chaque nœud pointe vers d'autres nœuds, où les références s'appuient les unes sur les autres pour former des chaînes d'informations et s'étendent sans fin, comme les racines d'un arbre, à mesure que nous travaillons et enrichissons le graphique à chaque nouveau commit.

Ces types de graphiques sont souvent utilisés pour visualiser les branches de Git, et on trouve couramment des schémas d'arborescence dans des tutoriels sur Git, même les plus basiques.

Pour être honnête, le GOA est un concept assez sophistiqué et la plupart des tutoriels Git ne s'aventurent même pas à en mentionner le nom, même s'ils les emploient à titre de support visuel.

Je mentionne les GOA ici, alors que ce long moment passé ensemble touche à sa fin, pour préciser quelque chose au sujet de la philosophie de ce livre. Quand on essaie d'expliquer Git, on se concentre souvent sur la vision globale du système : des réseaux entiers de repositories qui échangent des données, de vastes systèmes de branches issues les unes des autres.

Je ne dénigre pas ces tentatives : ces images sont réelles et elles sont spectaculaires. Comme œuvre de l'informatique, je trouve Git d'une extrême élégance.

Mais je trouve aussi que considérer ces choses comme des systèmes, c'est passer à côté de la plus grande qualité de Git : des gens comme vous, moi et vos collègues font évoluer des projets, modification après modification, et bâtissent des historiques. Le graphique n'a pas grande importance si votre objectif est d'enregistrer la prochaine version de votre projet ou de communiquer des modifications à votre équipe. Et si l'historique auquel nous collaborons via Git peut être modélisé sous la forme d'un graphique, il peut aussi être présenté comme une liste d'incidents — une histoire, autrement dit.

Certes, un repository Git n'est pas le meilleur endroit pour raconter une histoire, et la ligne de commande de Git n'est pas la façon la plus naturelle de le faire. Au tout début de cet ouvrage, j'ai décrit l'interface de Git comme une « abstraction percée ». Git essaie, sans toujours y parvenir, de nous éviter d'avoir à comprendre les innombrables complexités qui sous-tendent l'exécution d'une commande. Quand il n'y parvient pas, il nous pousse à apprendre ce qui se passe en coulisses.

Mais les histoires que nous racontons ensemble sont aussi vraies et belles que les structures d'informations élaborées par les créateurs de Git pour les contenir. Et maintenant, je l'espère, vous serez armé des connaissances nécessaires pour raconter ces histoires sans crainte.

CONCLUSION

Dans ce livre, nous sommes partis de la difficile modification des écrits gravés dans la pierre pour arriver à des astuces sur l'utilisation d'un HEAD détaché. En chemin, nous avons éclairci quelques aspects de Git : de quoi sont faits les commits, en quoi un commit est une version complète de votre travail, et comment les commits, accompagnés des repositories distants, des branches et autres, s'accumulent pour former le vaste paysage sauvage des choses dont — bonne nouvelle ! — vous devrez désormais vous soucier au quotidien. Et je comprendrais qu'une fois rendu au terme de cet ouvrage qui n'est sans doute pas le plus concis de la série A Book Apart, vous ayez encore quelques questions sur Git.

C'est bien normal ! Nous n'avons fait qu'effleurer la surface de ce dont Git est capable — et c'était l'intention. Le but n'était pas tant que vous terminiez cet ouvrage en connaissant toutes les commandes de Git que de vous transmettre la façon dont Git pense, afin que vous sachiez que Git n'est ni méchant, ni magique, et encore moins effrayant. Ce n'est qu'un outil et, si vous l'utilisez correctement, il vous rendra service.

Surtout, vous pourrez utiliser les commandes et les fonctions que nous avons abordées dans ce livre comme des blocs de construction pour élaborer les processus Git qui vous satisferont, comme des tremplins pour apprendre de nouvelles astuces. Selon la nature de votre travail, vous découvrirez peut-être que les connaissances contenues dans ce livre suffisent amplement à mener vos tâches à bien, ou vous vous sentirez armé pour poser des questions plus précises sur l'utilisation optimale que vous pourrez faire de Git à l'avenir.

RESSOURCES

Référence des commandes

Voici une liste rapide de toutes les commandes Git référencées dans ce livre, ainsi que quelques autres. Les arguments indiqués entre crochets (`[truc]`) sont facultatifs.

git config [--global] <clé><valeur>

Met à jour les paramètres de Git en modifiant la préférence identifiée par `<clé>`, comme `user.email`, pour lui donner la `<valeur>` indiquée, par exemple david@demaree.me. Le marqueur `--global` enregistre les préférences dans un fichier de votre répertoire d'accueil, donc Git les applique à tous les projets de votre ordinateur. Autrement, elles sont enregistrées et appliquées uniquement à un projet spécifique.

git init

Crée un nouveau projet Git à l'intérieur du répertoire de travail courant. Ainsi, si vous vous trouvez dans un répertoire nommé `mon-super-projet` contenant un site web sur lequel vous travaillez, la commande `git init` transformera ce dossier en repository Git tout neuf, prêt à l'emploi.

git clone <URL> [répertoire]

Copie un projet Git existant, situé à une URL donnée, sur votre ordinateur sous la forme d'un nouveau répertoire. Par défaut, le répertoire portera le même nom que le repository Git de l'URL : le repository `https://gitforhumans.info/rails.git` sera copié dans un dossier nommé `rails`, mais vous pouvez indiquer votre propre nom de répertoire comme argument si vous le souhaitez.

git status [-s] [chemin/vers/truc]

Renvoie le statut de votre copie de travail : identifie les fichiers qui sont modifiés mais non indexés, ou ajoutés mais non committés. Le marqueur facultatif `--short` ou `-s` produit une

version raccourcie du statut. Par défaut, `git status` fournit le statut de tout ce qui se trouve dans votre projet, mais vous pouvez indiquer un répertoire ou un chemin de fichier pour limiter les résultats.

git add [--all] nomfichier.txt

Ajoute un fichier modifié à la zone d'indexation en vue de son inclusion dans le prochain commit.

git rm dossier/nomfichier.txt

Commande raccourcie qui supprime le fichier du chemin indiqué, puis indexe la suppression pour le prochain commit. Si vous avez déjà supprimé le fichier ailleurs (dans Finder par exemple), la commande se contente d'indexer le changement.

git mv ancienchemin.txt nouveauchemin.txt

Autre raccourci qui déplace le fichier se trouvant sur `ancienchemin.txt` vers `nouveauchemin.txt`, puis indexe la modification.

git reset nomfichier.txt

Opposé de `git add` : une fois que vous avez indexé une modification de `nomfichier.txt`, vous pouvez utiliser `git reset` pour le désindexer.

git commit [-a] [— "Votre message"]

Ajoute un commit comprenant toutes les modifications indexées à l'aide de `git add`. L'option `--all` (ou `-a`) est pratique : elle permet d'indexer automatiquement toutes les modifications apportées à la copie de travail. Vous pouvez utiliser l'argument `--message` (`-m`) pour préciser votre message de commit. Si vous le laissez vide, Git ouvre votre éditeur de texte par défaut (ou celui que vous avez indiqué dans les paramètres de Git).

git branch [-r|-a]

Affiche une liste de toutes les branches. Par défaut, cette commande présente uniquement les branches de votre copie locale du repository. L'option `-r` peut présenter toutes les branches

récupérées dans les repositories distants ; `-a` affiche à la fois les branches locales et distantes.

git branch ‹nombranche› [‹commit›]

Si vous passez un nom de branche comme argument à la commande `git branch`, celle-ci crée une branche sous ce nom qui démarre au commit actuel (ou au commit que vous aurez spécifié si vous avez indiqué un ID de commit).

git checkout [-b] ‹nombranche-ou-commit›

Met à jour votre copie de travail afin qu'elle corresponde à la branche ou au commit indiqué. En pratique, vous fait basculer sur cette branche/commit. Si vous extrayez une branche, Git la définit comme la branche en cours pour vous permettre d'y ajouter des commits. Si vous extrayez un commit ou un tag, Git vous « détache » de toutes les branches : vous pouvez ajouter des commits, mais ils ne seront accessibles que via leur identifiant.

git merge ‹autrebranche›

Fusionne `autrebranche` dans la branche actuelle, s'il n'y a pas de conflit. En cas de conflit, Git copie et indexe la plus grande portion possible de l'autre branche, et balise les fichiers conflictuels pour vous permettre de résoudre le problème vous-même avant de committer.

git remote add ‹nom›‹URL›

Ajoute un repository distant sous l'URL et le nom indiqués dans les paramètres de votre projet Git local.

git remote rm ‹nom›

Retire le repository distant des paramètres de votre projet ainsi que toutes les branches de suivi distantes que vous pouvez avoir récupérées sur le serveur. Notez que cela ne supprime le repository distant qu'au niveau de vos paramètres locaux : les machines de vos collègues et le serveur ne seront pas affectés.

git push ‹nomrepositorydistant›‹nombranche›

Pousse l'état actuel de `nombranche` dans le repository distant nommé `nomrepositorydistant`.

git pull ‹nomrepositorydistant›‹nombranche›

Récupère l'état actuel de `nombranche` se trouvant dans le repository distant vers votre copie locale, et tente de le fusionner dans votre branche actuelle.

git fetch ‹nomrepositorydistant›

Copie tout le contenu du repository distant vers votre copie locale. Quand vous exécutez `git pull`, le fetch se produit automatiquement.

git log [—oneline] [—pretty] [‹nombranche-ou-commit›]

Présente une liste des commits dans l'ordre inverse, en commençant par la tête en cours (ou par le commit que vous aurez désigné par un nom de branche ou un ID). Vous pouvez utiliser l'option `--pretty` pour personnaliser le résultat ; `--oneline` est un raccourci pour le format de résultat le plus courant : un ID de commit court suivi du message de commit sur chaque ligne.

git diff [—stat] [‹nombranche-ou-commit›]

Génère un « diff », c'est-à-dire une représentation visuelle des différences entre deux commits. L'option `--stat` produit une vue résumée qui dresse la liste des fichiers modifiés, en indiquant le nombre de lignes ajoutées et supprimées dans chacun d'eux.

git tag [-a] [-m] ‹nomtag› [‹commit›]

Étiquette un commit sous le nom fourni, que vous pourrez ensuite utiliser comme nom permanent et convivial pour le commit en question. Le marqueur `-a` indique à Git de créer un tag annoté, qui inclut des informations sur la date et l'auteur de sa création, ainsi qu'un message descriptif (comme un commit). (Autrement, Git crée un tag « léger » qui fait référence à un commit sans créer aucune autre information.) Si vous créez

un tag annoté, n'oubliez pas d'inclure l'argument `--message/–m`, comme vous le feriez pour un commit.

git tag -d <nomtag>

Vous ne devriez pas avoir besoin de supprimer un tag, mais si cela se produit, vous pouvez le faire en passant l'option `-d` (pour « delete ») dans la commande `git tag`.

git tag -l

Produit une liste de tous les tags de votre repository.

git push --tags <nomrepositorydistant>

À titre de protection contre le partage accidentel d'un tag que vous n'êtes pas prêt à partager, Git ne pousse aucun tag en l'absence de l'option `--tags`.

Pour une liste exhaustive de toutes les commandes de Git et des informations complètes sur leur utilisation, consultez la documentation sur le site de Git (http://bkaprt.com/gfh/06-01/).

Applications Git recommandées

Dans ce livre, j'ai choisi de me concentrer sur l'interface en ligne de commande de Git afin de démontrer au mieux le mode de pensée de Git, et je recommande de commencer par là. Toutefois, on trouve d'excellentes applications pour Windows et pour Mac qui vous feront gagner du temps une fois que vous aurez pris vos marques.

GitHub Desktop. Que vous hébergiez ou non votre code sur GitHub, les applications de bureau pour Mac et Windows comptent parmi les meilleures, et elles sont gratuites. Vous pouvez indexer et committer des modifications, créer des branches et basculer de l'une à l'autre, échanger des données avec des repositories distants via push et pull, le tout visuellement. Et si vos données sont effectivement sur GitHub, l'application de bureau facilite la création des « pull requests » et l'ouverture d'une vue comparée (http://bkaprt.com/gfh/06-02/).

Tower. Pour les utilisateurs Mac avancés qui sont prêts à dépenser 69 €, Tower offre de nombreuses options et fonctionnalités supplémentaires. Tandis que GitHub Desktop se concentre sur les fondamentaux, Tower peut également traiter la résolution des conflits de fusion, la sélection fine de commits et bien plus encore (http://bkaprt.com/gfh/06-03/).

SourceTree. Plus complexe et plus puissant que les applications de GitHub, mais dépourvu de certaines excellentes fonctionnalités de Tower, SourceTree (qui est gratuit) est un excellent choix pour quelqu'un qui veut bénéficier de possibilités supplémentaires sans dépenser d'argent (http://bkaprt.com/gfh/06-04/).

De nombreux outils de code populaires intègrent une prise en charge de Git ou vous permettent de l'ajouter via des plug-ins, ce qui vous permet de committer des modifications sans quitter l'application. Atom, Coda, Sublime Text, TextMate, BBEdit, Xcode et Visual Studio Code sont tous immédiatement compatibles avec Git.

Services d'hébergement de Git

GitHub. Le plus grand — et quasiment le seul — nom de l'hébergement Git. Si vous travaillez avec du code, il y a fort à parier que vous avez déjà eu affaire à GitHub, parce que tout le monde s'en sert. Au-delà de son omniprésence, GitHub reste sans doute le meilleur choix pour la plupart des utilisateurs : l'entreprise continue d'investir dans des outils et des ressources pour faciliter la collaboration via Git (notamment à l'aide des Pull Requests), et produit de nouveaux outils intéressants comme les Pages GitHub (hébergement web bâti sur un repository Git). GitHub est payant lorsque les projets hébergés par une personne ou une organisation sont privés. Les projets publics, dont n'importe qui peut récupérer ou télécharger le code, mais dans lesquels seule votre équipe peut pousser des modifications, sont toujours gratuits. Il existe encore une édition « entreprise » qui est très chère mais que vous pouvez exécuter sur vos propres serveurs pour exercer un contrôle maximal sur vos données (github.com).

Bitbucket. Bitbucket n'a pas l'élégance de GitHub, mais il présente un avantage certain pour les amateurs et les petites entreprises : les individus et les petites équipées peuvent héberger rapidement un nombre illimité de repositories privés. Si Bitbucket ne dispose pas de l'immense communauté de GitHub, j'utilise personnellement les deux : GitHub pour les projets publics et les travaux collaboratifs, et Bitbucket pour les petits projets personnels (bitbucket.org).

Beanstalk. Spécialisé dans les repositories payants et privés, Beanstalk propose quelques fonctionnalités très appréciées des développeurs web, et notamment un outil de déploiement intégré qui met automatiquement à jour vos serveurs web après que du nouveau code a été poussé dans votre repository (beanstalkapp.com).

Enfin, si vous êtes habile avec la ligne de commande et que vous voulez avoir un contrôle total sur vos données (ou que rien n'est assez nerd pour vous), il n'est pas si difficile de monter votre propre hébergement. Le protocole par défaut de Git est SSH, donc n'importe quel serveur Linux peut, en théorie, être configuré pour accueillir des repositories Git. L'équipe de DigitalOcean propose un guide pratique pour la mise en place d'un serveur Git simple sur l'un de leurs serveurs virtuels (http://bkaprt.com/gfh/06-05/).

Encore plus de Git

« Meet the Command Line ». Animés par Dan Benjamin, ces vidéos (en anglais) sont une excellente ressource pour les novices de la ligne de commande. Elles les aideront à acquérir rapidement de l'aisance dans l'utilisation du terminal (http://bkaprt.com/gfh/06-06/). Et puisqu'on parle de vidéos, les tutoriels Git de Pluralsight par James Kovacs couvrent tous les aspects utiles, de l'installation de Git aux thèmes plus complexes de l'embranchement et du rebasage (http://bkaprt.com/gfh/06-07/).

GitHub Training. Ce microsite propose deux « cours » en ligne sous forme de diaporamas (en anglais) : l'un s'adresse aux vétérans de la ligne de commande, l'autre aux utilisateurs d'applications graphiques. Vous y trouverez aussi un PDF de

référence rapide couvrant l'essentiel des commandes de base, et même un simulateur Git basé dans le navigateur, où vous pourrez vous entraîner à saisir des commandes (http://bkaprt.com/gfh/06-08/).

Tutoriels Git d'Atlassian. Excellente ressource (en anglais) pour les utilisateurs débutants et vétérans de Git, ces tutoriels couvrent un large éventail de sujets dans une langue claire et enrichie d'illustrations. Leur guide de comparaison des différents processus Git explore en particulier le modèle de hub dont j'ai parlé dans ce livre (http://bkaprt.com/gfh/06-09/).

Git for Type Designers. Frank Grießhammer a compilé un excellent guide de référence rapide (en anglais) sur les processus Git de base, avec des remarques portant spécialement sur l'utilisation de Git avec les outils de dessin de caractères (http://bkaprt.com/gfh/06-10/).

Pro Git. L'ouvrage open source de Scott Chacon (en anglais) est la référence ultime des humains qui veulent vraiment savoir comment fonctionne Git (http://bkaprt.com/gfh/06-11/).

Des repositories Git à connaître

Repository Git suivant l'intégralité du Droit allemand. Comme son gestionnaire l'indique, « tous les citoyens allemands peuvent facilement accéder à une version actualisée de leurs lois sur Internet. Mais il n'est pas facile ni même possible de suivre le processus législatif, l'évolution historique et les modifications des lois. » Publier une copie des lois via Git, c'est permettre à chacun de voir comment le Droit évolue au fil du temps tout simplement en consultant le commit log (http://bkaprt.com/gfh/06-12/).

Code de conduite de Vox Media. Vox a ouvert la source de son code de conduite, un document vivant qui, comme l'écrit Mandy Brown, « va évoluer et se développer au rythme de notre équipe et des apports de la communauté. » Le code source du site web du code de conduite est hébergé sur GitHub ; les membres de la communauté peuvent signaler des problèmes, proposer des correctifs ou simplement consulter le commit log pour examiner son évolution dans le temps (http://bkaprt.com/gfh/06-13/).

Tacofancy. « Repository de recettes de tacos communautaire et orienté objet » créé et entretenu par Dan Sinker, Tacofancy propose des recettes complètes de tacos et d'ingrédients (http://bkaprt.com/gfh/06-14/).

What is Code? En 2015, Paul Ford, auteur, programmeur et utilisateur prolifique de GitHub (http://bkaprt.com/gfh/06-15/) a écrit pour Bloomberg Businessweek un article intitulé « What is Code? » (http://bkaprt.com/gfh/06-16/), vaste essai explicatif abordant, entre autres, GitHub et Git. Le code source de la version interactive et ludique de cet article est accessible à tous sur GitHub (http://bkaprt.com/gfh/06-17/).

REMERCIEMENTS

Ce livre n'aurait pas vu le jour sans l'aide et les encouragements de l'extraordinaire équipe d'édition et de production de A Book Apart. Quand son auteur est devenu un jeune parent stressé accumulant les nuits blanches, Katel Le Dû a réussi à faire avancer les choses avec un calme olympien. Les conseils rédactionnels de Caren Litherland ont aidé *Git par la pratique* à prendre forme et transformé en livre ce qui n'était qu'une pile d'idées sans queue ni tête. Jason Santa Maria, Rob Weychert et Ron Bilodeau ont rendu mon travail aussi beau que lisible sur papier et à l'écran. Enfin et surtout, merci à Jeffrey Zeldman de nous avoir embarqués dans cette galère.

Un salut tout spécial et un whisky pour David Yee, notre éditeur technique, qui a posé les bonnes questions et repéré les passages confus avec intelligence, empathie et juste ce qu'il faut de pédanterie.

Un immense merci (et encore plus de whisky) à Mandy Brown qui a vu un livre en moi, m'a offert l'opportunité de l'écrire pour A Book Apart et, plus de deux ans après lorsqu'il a enfin été achevé, en a signé l'avant-propos.

Merci aux amis et aux collègues qui m'ont apporté de judicieux retours sur mes premières ébauches, ainsi que des encouragements plus que nécessaires, généralement sous la forme d'un « au fait, ton livre, tu l'as terminé ? » : Tim Brown, Paul Hammond, Morgan Kelly, Sally Kerrigan, Liz Galle, Jake Giltsoff, Lucy Knisley, Bram Stein, Kevin Stewart et Elliot Jay Stocks.

À tous les designers qui se sont plaints de Git sur mon fil Twitter, et qui ont été la source d'inspiration de ce livre, notamment Ethan Marcotte, Frank Chimero et Susan Robertson, et en particulier Mat Marquis et Dave Rupert qui ont été les premiers à demander explicitement à A Book Apart de publier un ouvrage tel que celui que vous avez entre les mains : c'est pour vous tous que je l'ai écrit.

Aux petits génies qui sont derrière Git et GitHub : Linus Torvalds, Junio Hamano, Scott Chacon, Chris Wanstrath, PJ Hyett, Tom Preston-Werner et tous les autres qui ont apporté du code,

des mots ou les deux, pour que nous puissions partager du code et des mots les uns avec les autres : tout ça, c'est de votre faute ! Merci.

À Jeff Veen, Bryan Mason, Ryan Carver, Greg Veen, Matthew Rechs et toute l'équipe de Typekit, qui m'ont montré comment résoudre les problèmes les plus délicats et m'ont supporté pendant les deux dernières années que j'ai passées à résoudre celui-ci.

Enfin, tout mon amour et ma plus grande gratitude à mon épouse Jody pour son affection, sa patience et son soutien, et à notre petite fille June, qui est si impatiente de manger ce livre.

RÉFÉRENCES

Les URL raccourcies sont numérotées dans l'ordre et les URL complètes sont indiquées ci-dessous à titre de référence.

Introduction

00-01 http://frankchimero.com/writing//two-sentences-about-getting-older- and-working-on-the-web/

00-02 http://www.joelonsoftware.com/articles/LeakyAbstractions.html

Chapitre 1

01-01 http://www.pcworld.idg.com.au/article/129776/after_controversy_torvalds_begins_work_git_/

01-02 https://github.com/torvalds/linux

Chapitre 3

03-01 https://en.wikipedia.org/wiki/exquisite_corpse

03-02 http://www.layertennis.com/

Chapitre 4

04-01 https://help.github.com/articles/generating-ssh-keys/

04-02 https://git-scm.com/book/ch4-1.html

Chapitre 5

05-01 http://git-scm.com/docs/pretty-formats

05-02 https://www.atlassian.com/git/tutorials/git-log/formatting-log-output

05-03 https://www.atlassian.com/git/tutorials/inspecting-a-repository

05-04 https://www.youtube.com/watch?v=4XpnKHJAok8&t=56m20s

05-05 https://help.github.com/articles/associating-text-editors-with-git/

05-06 http://tbaggery.com/2008/04/19/a-note-about-git-commit-messages.html

05-07 http://chris.beams.io/posts/git-commit/

05-08 http://24ways.org/2014/dealing-with-emergencies-in-git/

05-09 http://www.kaleidoscopeapp.com/

05-10 https://github.com/rails/rails/tree/v4.2.0

05-11 http://alistapart.com/article/git-the-safety-net-for-your-projects

Ressources

06-01 https://git-scm.com/docs

06-02 https://desktop.github.com

06-03 http://www.git-tower.com

06-04 http://sourcetreeapp.com/

06-05 https://www.digitalocean.com/community/tutorials/how-to-set-up-a-private-git-server-on-a-vps

06-06 https://www.pluralsight.com/courses/meet-command-line

06-07 http://training.github.com/

06-08 https://www.atlassian.com/git/tutorials

06-09 http://git-scm.com/book

06-10 https://github.com/bundestag/gesetze

06-11 https://github.com/voxmedia/code-of-conduct

06-12 https://github.com/sinker/tacofancy

06-13 https://github.com/ftrain

06-14 http://www.bloomberg.com/whatiscode

06-15 https://github.com/BloombergMedia/whatiscode

INDEX

A

Allen, David 15
Atom 30, 116

B

Beams, Chris 120
Black Pixel 124
branche 64
  distante 93
  extraire 105
  merge 72
  nommer 70
  ouvrir 64
  suivre 101

C

Chacon, Scott 93
Chimero, Frank 8
clone 41
collaboration distribuée 18
commande 31
commit 42
  comparaison 122
  de fusion 75
  de tête 75
  deuxième 48
  étiqueter 128
  ID 112
  message 115
  parent 61
comparaison de commits 122
configuration de Git 38
conflit de fusion 78, 104
coordination des modifications 16
copie de travail 14
corbeille 53

D-E

diff 123
Dropbox 21
éditeur de texte 116
Emacs 116
embranchement 62

F

fast-forward 74
fichier
  ajouter 45
  renommer 54
  supprimer 51
fonctions de Git 37
fusion
  commit 75
  conflit 78, 98

G

gestion de versions 13
Git
  configuration 38
  fonctions 37
  installation 28
  langage 37
  statut 44
git add --all 57
Git Bash 29
git checkout 131
git fetch 103
GitHub 85, 88, 125, 143
graphique orienté acyclique 135

H

HEAD détaché 133
historique de projet 107

Hopper, Grace 13, 84
HTTPS 92
hub 85

I-K

ID de commit 112
indexation 47
installation de Git 28
Kaleidoscope 124

L

layer tennis 69
ligne de commande 31
    navigation 34
Linux 17, 21

M

marqueur de conflit 79
merge commit 75
message de commit 115
modèle de hub 85
modification
    coordonner 16
    pousser 94
    récupérer 95

N

navigation
    ligne de commande 34
    répertoire 34
nouveau projet 40

O

options 38

P

Pope, Tim 120
projet
    cloner 41
    historique 107
protocole
    Git 91
    SSH 91

R

rejet de push 100
repository distant 88
Retour vers le futur 24, 31
Ruby on Rails 129

S

Spolsky, Joel 9
statut Git 44
Sublime Text 30, 116
système de gestion de versions 13

T

tag 117
TextMate 116
Torvalds, Linus 17

V

version
    gestion 13
    séquentielle 61
Vim 116

W

Westby, Emma Jane 120

À PROPOS DE A BOOK APART

Nous couvrons les sujets émergents et essentiels du design et du développement web avec style, clarté et surtout brièveté — car les professionnels du design et du développement n'ont pas de temps à perdre !

À PROPOS DE L'AUTEUR

Photographie de Ryan Carver

David Demaree est développeur web, designer, conférencier et chef de produit, et vit aux abords de New York. Senior Product Manager pour Adobe Typekit, sa mission consiste à élaborer des outils permettant à tous de trouver et d'utiliser les meilleures polices typographiques pour tous les contextes. David a participé à de nombreuses conférences sur le design et la technologie aux États-Unis, en Europe et en Australie, et il rédige des articles techniques sur Medium.